

Physics-Informed GNN for Analog Circuit Simulation

Scientific work for obtaining the academic degree

Master of Science (M.Sc.)

at the TUM School of Computation, Information and Technology of the Technical University of Munich

Supervisor Prof. Dr. Robert Wille
Dr. Lorenzo Servadei
Chair of Design Automation

Advisor Dr. Arun Venkitaraman
Sony AI

Submitted by Zhongkai Hu

Submitted on July 10, 2026

Abstract

Sizing an analog operational amplifier still means running a SPICE simulation for every candidate design and that simulation is the main bottleneck in the design loop. This thesis trains a graph neural network to predict the result of that simulation straight from the netlist, without running SPICE. The model reads the sized transistor-level netlist as a graph and predicts the node voltages, the branch currents, the per transistor transconductance and output conductance and the DC gain.

On a three-stage Miller-compensated amplifier the model predicts the node voltages to 6.42 mV, about 0.36% of the supply range, the currents and small-signal parameters to roughly five to eight percent per device. It does this on an unfiltered dataset that keeps the poorly biased and non-amplifying circuits most other work removes, so it learns the whole design space and not just the well-behaved part.

The main question is whether known circuit physics, added to the training loss, helps the model. It helps only when the physics is exact. Of all the relations tried, only Kirchhoff's current law improves the model. Every approximate device equation makes it worse, the square-law and sub-threshold formulas, the current-mirror and differential-pair equalities and the single-pole bandwidth relation. These formulas that are already only approximations and on top of that they use fixed parameters that are not fixed in the real device behavior, so even a formula with the right shape would still use the wrong numbers. An approximate law is not a softer version of a good prior. It pulls the model away from the correct values. Kirchhoff's current law works because it is not a device equation at all. It is an universal current conservation, that holds in any region and any process. Its benefit is largest when data is scarce and it lowers the current imbalance in the predictions.

Another important finding is about where to put the physics. When a law is exact, it is better built into the architecture than added to the loss. Built into the architecture the law cannot be broken and the model does not waste any capacity learning to satisfy it, so that capacity goes to the prediction instead. The current head works this way. It predicts one current per device and shares it across the terminals. The soft loss is only needed where this hard form is not possible.

The graph representation also helps across topologies. One model can handle several at once at no extra cost and a model pretrained on some topologies and fine-tuned on a new one beats training from scratch at every data size.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Objectives	3
1.4	Contribution	3
2	Background and Related Work	5
2.1	Analog Operational Amplifiers	5
2.1.1	Circuit Function and Performance Metrics	5
2.1.2	DC Operating Point and Small-Signal Analysis	6
2.1.3	The Role of SPICE Simulation in the Design Loop	7
2.2	MOSFET Modeling in Modern Processes	7
2.2.1	From Square-Law to BSIM4	7
2.2.2	Operating Regions and Their Analytical Descriptions	8
2.2.3	Limits of Simplified Equations	9
2.3	Graph Neural Networks	10
2.3.1	Message Passing and Aggregation	10
2.3.2	Common Architectures	11
2.3.3	Graph Representations and GNN for Electronic Circuits	12
2.4	Physics-Informed Neural Networks	13
2.4.1	Foundations and the Soft-Hard Constraint Distinction	13
2.4.2	Applications to Circuits and Devices	14
2.5	Classical and Equation-Based Sizing Methods	14
2.5.1	Manual and Structured Design Methodologies	15
2.5.2	Constraint-Programming and Equation-Based Solvers	15
2.6	Machine-Learning Surrogates for Analog Sizing	16
2.6.1	Supervised Surrogates: Neural Networks, Gaussian Processes, and Trans- formers	16
2.6.2	Surrogate-Assisted Optimization Loops	16
2.7	Positioning of This Work	17
3	Methodology	19
3.1	Circuit, Process and Ground Truth	19
3.2	Circuit Representation	21
3.2.1	Graph Reconstruction	21
3.2.2	Node and Edge Features	22
3.3	Data	24
3.3.1	Data Generation	24
3.3.2	Dataset Statistics	25
3.3.3	Dataset Splits	27

3.4	Model Architectures	27
3.4.1	Backbone and Message Passing	28
3.4.2	Loop-Attention Component	30
3.4.3	Global Context	31
3.4.4	Prediction Heads	32
3.5	Training Objective	33
3.5.1	Supervised Losses and Normalization	34
3.5.2	Physics-Informed Terms	35
3.6	Experimental Protocol	38
3.6.1	Training Setup	38
3.6.2	Experimental Conditions	39
4	Results	41
4.1	Prediction Accuracy	41
4.2	Architecture Ablations	44
4.2.1	Message Passing	44
4.2.2	Attention Components	45
4.2.3	Backbone Structure	47
4.2.4	Prediction Heads	48
4.3	Physics Constraints	51
4.3.1	Choosing the Constraint	51
4.3.2	Data Efficiency	55
4.3.3	Physics Consistency	57
4.4	Generalization to Unseen Topologies	58
4.4.1	Joint Training Across Topologies	58
4.4.2	Adapting to an Unseen Topology	60
4.4.3	The Role of Physics in Transfer	62
5	Discussion and Conclusion	63
5.1	Summary of Findings	63
5.2	Correct Physics, Not Just Physics	64
5.3	A Harder Problem than Filtered Sizing	64
5.4	Limitations	65
5.5	Future Work	65
5.6	Conclusion	66
A	Appendix 1	67
A.1	Full Dataset Statistics	67
A.2	Training Details	68
A.3	Detailed Ablation Accuracy	69
A.4	DC-Gain Formula	71
A.5	Data Efficiency Runs	72
A.6	Approximate Physics Constraints	73
A.6.1	Constraint Definitions	73
A.6.2	The SMAXT Transconductance Formula	73
A.6.3	Formula Accuracy in Detail	74
A.7	Predicting the Unity-Gain Bandwidth	74
A.7.1	The Data Problem	75
A.7.2	The Approaches	75
A.7.3	The Cost to the Other Quantities	76
A.8	Full Fine-Tuning Results Across Topologies	76

A.9 Explored Alternatives: Compressed IV-Surface Embeddings as Device Features	77
Bibliography	80

Chapter 1

Introduction

Designing a modern analog operational amplifier can take an experienced engineer weeks of iterative work. While digital design has been transformed by automation, analog sizing remains largely a manual work. The simplified equations that traditionally guided this process have lost much of their predictive power in advanced process nodes, where short-channel effects and complex device behavior play a significant role [1].

Designers lean heavily on SPICE simulation. Every sizing is verified by a full operating-point and AC analysis to check gain, bandwidth, stability, noise and power. Exploring even a moderately sized design space can require dozens of simulations [2]. As a result, simulation itself has become the bottleneck. The natural question is whether machine learning can help with for SPICE, and if, what kind of model is best suited for this task.

1.1 Motivation

Researchers tried to address this by training machine-learning models that approximate SPICE behavior [2–4]. Most of these works target the sizing problem, given target specifications, find device dimensions and bias currents that meet them [5, 6]. Models based on neural networks, random forests or Gaussian processes are trained offline on simulated design points and then used inside an optimization loop to replace the expensive simulation calls [4, 7].

Two assumptions limit the applicability of these models. The first is that all transistors operate in saturation and in strong inversion, often enforced by filtering the training data or by adding constraints during optimization [2, 8]. This restriction simplifies the learning problem but excludes large parts of the design space and limits the resulting models. The second is that a circuit can be reduced to a flat feature vector, which discards the structural information encoded in the netlist itself.

Circuit's performance comes from how devices are interconnected. Feedback loops, current mirrors, cascode stacks and bias networks, each contribute behavior that depends on topology and on individual sizing. Graph neural networks (GNNs) benefit from these kind of structure, treating devices as nodes and connections as edges [9]. Recent works have applied them to parasitic-aware sizing, placement guidance and design-space exploration, showing that respecting topology improves both prediction accuracy and sample efficiency [10, 11]. These models are trained entirely with the data only, with no mechanism to enforce their

predictions to obey basic circuit laws.

A separate line of work on physics-informed neural networks addresses precisely this gap of knowledge. By embedding physics equations directly into the training loss, the model is guided by the known physics. They generalize better, train on less data and produce solutions that respect physical constraints [12]. Many have begun to apply this idea to electrical circuits, formulating circuit equations as residual losses [13, 14].

This thesis goes the opposite way to the sizing work and combines the other two directions. Rather than going from specifications to device sizes, the goal is to take a sized circuit and predict its operating point. The model works on the circuit graph and is constrained by the physical laws the operating point must satisfy. This is a narrower target than predicting circuit-level specifications, but it is the foundation those specifications are built on.

1.2 Problem Statement

The task is to predict the operating point of an analog operational amplifier from its sized netlist, together with the small-signal quantities and DC gain that follow from that operating point. Concretely, the model is trained to predict the node voltages $\mathbf{v}$, branch currents $\mathbf{i}$, the small-signal parameters $\mathbf{g}_m$ and $\mathbf{g}_{ds}$ for each transistor and the DC gain. Formally, the model learns a mapping

$$f_{\theta} : G \mapsto (\mathbf{v}, \mathbf{i}, \mathbf{g}_m, \mathbf{g}_{ds}, A_{DC}) \quad (1.1)$$

parameterized by neural network weights θ , where G denotes the input circuit graph and it outputs the predicted DC and small-signal quantities. The graph G encodes both the netlist topology and the per-device and global parameters, which includes device dimensions, terminal connectivity, supply voltages, bias voltages and bias currents.

The primary benchmark circuit used throughout this work is the three-stage folded-cascode operational amplifier with single Miller capacitor compensation, referred to as `fan_smc`, taken from the open-source AnalogGym benchmark suite [15] and implemented in the SKY130 process design kit. To assess generalization, four additional op-amps from the same benchmark suite are used for cross-topology training and fine-tuning experiments. Training data is generated by sampling device sizes and running NGSpice to obtain ground-truth values for all predicted quantities. The dataset size is deliberately kept modest in order to evaluate whether the inclusion of physics constraints reduces the number of simulations required or improves the accuracy of the predicted quantities, which is one of the central questions being investigated.

The scope is restricted to DC operating point and small-signal prediction. Frequency-dependent AC metrics such as unity-gain bandwidth, phase margin and the gain amplitude margin were explored, but were not accurate enough (Appendix A.7). Layout parasitics, process variation and topology synthesis are likewise out of scope. Success is evaluated by comparing the predicted node voltages, branch currents, small-signal parameters and DC gain against NGSpice ground truth across a held-out validation set, with particular attention to accuracy in the low-data regime where physics constraints are expected to provide the greatest benefit.

1.3 Objectives

There are four main objectives that address the gap identified in the previous sections.

The first objective is to develop a graph neural network architecture that operates directly on the transistor-level netlist of an analog operational amplifier and predicts the operating point together with the small-signal quantities and DC gain. The architecture must respect the structural information of the netlist, which are device connectivity and the differences between transistors, nets and passive components.

The second objective is to design and integrate physics constraints into the training of this model, with particular emphasis on Kirchhoff's current law as a mechanism for enforcing consistency between branch currents. The integration should preserve the data only supervision, while adding a useful physical learning signal on top

The third objective is to evaluate empirically whether the inclusion of physics constraints reduces the data requirements or improves the prediction accuracy compared to a purely data-driven approach.

The fourth objective is to evaluate the generalization of the model across multiple op-amp topologies, by training on a set of topologies from the AnalogGym benchmark suite [15] and zero-shot or fine-tuning on a held-out topology. This tests whether the combination of graph representation and physics constraints supports efficient transfer across circuits.

1.4 Contribution

A graph neural network architecture for operating point, small-signal and DC gain prediction in analog operational amplifiers. The architecture operates directly on the transistor-level netlist of an op-amp, encoded as a graph with transistors, nets and passive components. The goal is to predict the operating point together with the per-transistor small-signal parameters and the DC gain. The architecture is described in detail in Section 3.4.

A physics-informed training based on Kirchhoff's current law. The training loss includes a soft KCL constraint that penalizes violations of current conservation at every internal node of the netlist. The formulation is described in Section 3.5.2 and its effect on prediction accuracy is reported in Section 4.3.2.

An analysis of the conditions under which simplified analytical device equations conflict with BSIM4 ground truth. Loss terms derived from MOSFET drain-current relationships in the saturation, linear and cutoff regions, from current-mirror equations and from analytical formulas that estimate AC performance metrics from the DC operating point, were investigated as candidate physics constraints. The study identifies which constraints provide useful learning signals and which actively hurt model performance. The findings are reported in Section 4.3.1 and explains the choice of KCL as the surviving constraint.

An empirical analysis of the data-efficiency benefits of KCL constraint under unfiltered training data. The training data is unfiltered and includes poorly biased and edge-case design points, a harder problem than model trained only on well-behaved circuits (Section 3.3.2). Results in Section 4.3.2 vary the training set size to isolate the contribution

of the KCL loss term, demonstrating that including physical knowledge provides the various accuracy improvements.

A study of cross-topology generalization through training on multiple op-amp topologies and fine-tuning on a held-out topology. The model is trained on four topologies from the AnalogGym benchmark suite and fine-tuned on the fifth unseen topology. This demonstrate that the combination of different graph represented topologies and physics constraints supports efficient transfer across circuits. The held-out topology and the size of the fine-tuning dataset, alongside the main accuracy and ablation results, are specified in Section 4.4.2.

Chapter 2

Background and Related Work

This chapter provides the technical background and prior-works in this related field of work. The first half introduces the fundamentals of this thesis. The analog operational amplifier as the target circuit, the BSIM4 device model that defines the ground truth, graph neural networks physics informed neural networks as the model class. The second half analyses the existing literature on machine learning methods for analog circuit design. The chapter closes by positioning the contributions of this thesis against the prior work.

2.1 Analog Operational Amplifiers

The operational amplifier is one of the most widely used electronic component in analog circuit design. It appears in signal conditioning, filtering, data conversion, voltage regulation, etc., where voltage amplification is required [16, 17]. This section introduces the aspects of op-amp design that are relevant for the rest of the thesis. The performance metrics that characterize an op-amp, the analysis steps that produce those metrics and the role of the circuit simulation.

2.1.1 Circuit Function and Performance Metrics

An operational amplifier consists of one or more cascaded gain stages, typically including a differential input stage, one or more intermediate stages and an output stage [16]. A two-stage amplifier is shown in Figure 2.1 as a stage-level block diagram and as the corresponding transistor-level schematic. The structure and the choice of devices determine the performance metrics. For the present work, The most important are the DC gain, the unity-gain bandwidth, the phase margin and gain margin. Other metrics such as the slew rate, the common-mode rejection ratio, the power-supply rejection ratio and the input-referred noise are equally important in practice.

Each performance metric depends on the device sizes, the bias conditions and the topology of the amplifier. In the small-signal analysis, the metrics can be approximated by closed-form expressions involving the per-transistor transconductances g_m , output conductances g_{ds} , the parasitic capacitances of the devices and the nets [17]. These small-signal parameters are derived from the DC operating point of the circuit, which is the set of node voltages and branch currents at which the amplifier sits when no input signal is applied. The DC operating point is therefore the foundation on which the rest of the performance characterization rests and hence the primary prediction target of the model developed in this work.

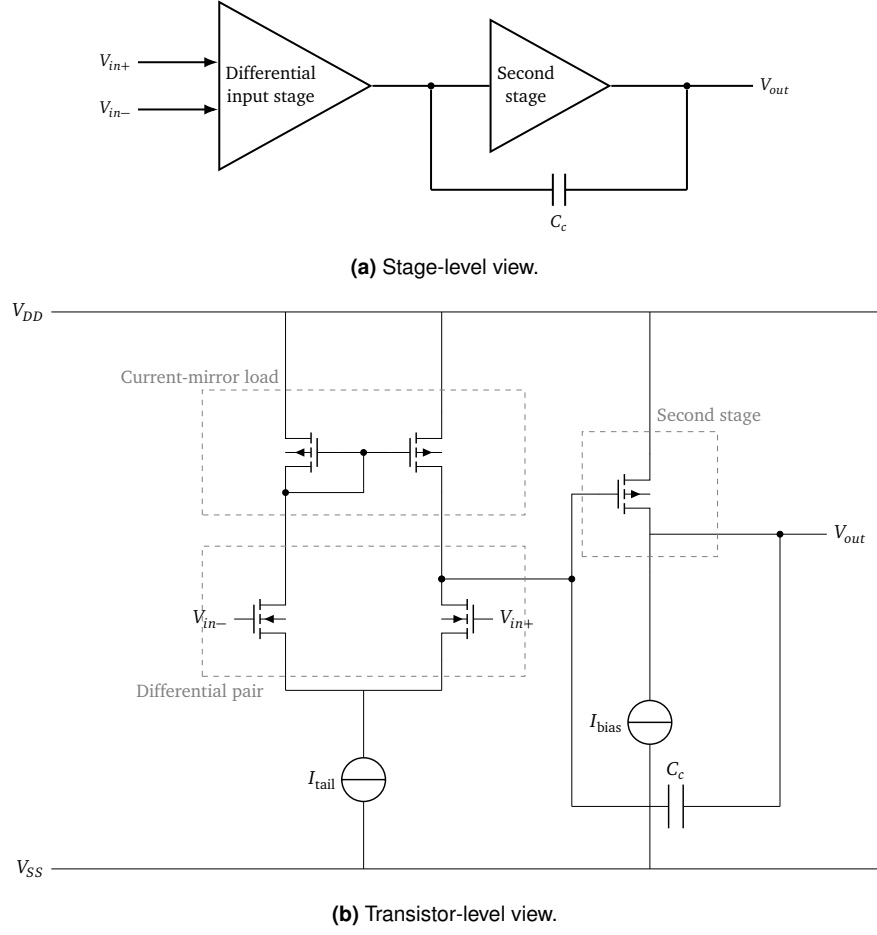


Figure 2.1: A two-stage operational amplifier with Miller compensation, shown at the stage level in (a) and at the transistor level in (b). The differential input stage with current-mirror load provides the first stage of voltage gain. The second stage provides the remaining gain and the capacitor C_c gives Miller compensation.

2.1.2 DC Operating Point and Small-Signal Analysis

The DC operating point of an analog circuit is the solution to the nonlinear system of equations formed by Kirchhoff's law at every node and by the device current-voltage relationships of every transistor and resistor in the circuit [16]. For a circuit with N nodes, the unknowns are the N node voltages and the equations relate unknowns through the linear resistor currents and the nonlinear drain-current behavior of the MOSFETs. Solving this system of equations requires numerical methods, typically Newton-Raphson iteration, which is nowadays performed by SPICE-like circuit simulators.

Once the DC operating point is known, the small-signal parameters, namely transconductance and output conductance of every transistor can be derived. Explicit form of these parameters is given in form of the equations in Section 2.2.2. Given these parameters, the DC gain of the amplifier can be approximated with closed form equation by combining the per-transistor transconductance and output-resistance contributions into stage-level expressions [17].

The frequency-domain or AC metrics, including the unity-gain bandwidth, the phase margin and gain margin, depend additionally on the parasitic capacitances of the devices and layout parasitic capacitances. [16]. These metrics are out of scope for the present work. The reasons are discussed in section 4.3.

2.1.3 The Role of SPICE Simulation in the Design Loop

A modern SPICE simulator solves the DC operating point of a circuit, performs small-signal AC analysis to obtain the frequency response (unity-gain bandwidth, phase and gain margins) and supports a wide range of additional analyses including noise, sensitivity, Monte Carlo, corner analysis, etc. The fidelity of these analyses rests on the underlying device models, whose form and limitations are the subject of Section 2.2.

The cost of this accuracy is computational. A single DC operating point simulation of a slightly complex op-amp can take milliseconds to seconds, as it depends on the convergence behavior of the Newton iteration and the complexity of the device models. This cost is acceptable when the designer runs a small number of simulations, but it becomes the bottleneck when simulation is used in an automated sizing or optimization loop that requires thousands or tens of thousands of evaluations [2, 4]. The motivation for surrogate models, including the model proposed in this thesis, comes directly from this cost. A surrogate that approximates the SPICE DC operating point with comparable accuracy but at a fraction of the cost can replace SPICE, enabling the designer to explore more of the design space within the same time.

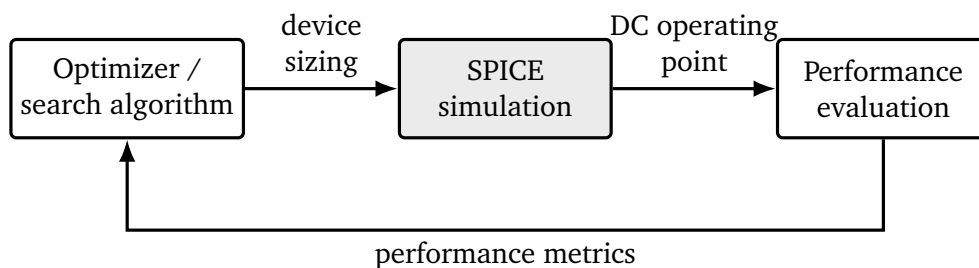


Figure 2.2: Simulation in the automated sizing loop. An optimizer gives a device sizing, a SPICE simulation evaluates it, and the resulting performance metrics guide the next iteration.

The simulator used throughout this thesis is NGSPICE [18], an open-source SPICE implementation that supports the BSIM4 device model and is compatible with the SkyWater 130 nm process design kit used also by the AnalogGym benchmark suite [15]. All ground-truth data for training and evaluation is generated by NGSPICE simulations on the netlists provided by the benchmark.

2.2 MOSFET Modeling in Modern Processes

The accuracy of any circuit simulation and of any surrogate that aims to replace it, is limited by the fidelity of the underlying model. This section explains the differences between the simple analytical equations and the complex empirical models used in modern process and why it is important for the physics-informed approach.

2.2.1 From Square-Law to BSIM4

The classical description of MOSFET behavior is the square-law model, in which the drain current in saturation is proportional to the square of the gate overdrive voltage [16]. This

model is compact and useful for developing design in long-channel devices, where the channel length is large enough that the physical effects can be captured by a small number of parameters. It remains the model designers use to reason about the first-order behavior of an op-amp.

As process technology scaled to smaller dimensions, the square-law model became progressively less accurate. Shrinking channel lengths introduced a range of physical effects that the simple model does not represent, such as channel-length modulation, velocity saturation, mobility degradation, drain-induced barrier lowering and various short-channel effects [17]. Capturing these effects analytically is not feasible with a single closed-form expression, hence the circuit simulation started to rely on device models that combine both physical derivation and empirical fitting. The Berkeley Short-channel IGFET Model (BSIM), in its fourth revision BSIM4 [19], is the industry-standard compact model implemented in SPICE simulators. It is also the model used in this thesis to generate all data. BSIM4 contains hundreds of parameters and is calibrated by the foundry (e.g. SKY130 process) to reproduce measured device behavior across different bias, geometry, temperature and process corners.

2.2.2 Operating Regions and Their Analytical Descriptions

In the analytical domain, a MOSFET is divided into three regions, cutoff, saturation and triode, which are determined by the relationship between the terminal voltages and the threshold voltage V_{th} . The gate-source voltage V_{GS} , the drain-source voltage V_{DS} and the gate overdrive $V_{ov} = V_{GS} - V_{th}$ form the square-law equations of the drain current I_D in the three regions is

$$I_D = \begin{cases} 0, & V_{GS} < V_{th} \quad (\text{cutoff}), \\ \mu C_{ox} \frac{W}{L} \left[V_{ov} V_{DS} - \frac{V_{DS}^2}{2} \right], & V_{DS} < V_{ov} \quad (\text{triode}), \\ \frac{1}{2} \mu C_{ox} \frac{W}{L} V_{ov}^2 (1 + \lambda V_{DS}), & V_{DS} \geq V_{ov} \quad (\text{saturation}), \end{cases} \quad (2.1)$$

where μ is the carrier mobility, C_{ox} is the gate-oxide capacitance per unit area, W and L are the channel width and length, and λ is the channel-length modulation (CLM) coefficient. In the cutoff region the channel is not formed and the drain current is idealized as negligible. In the triode region the device is strongly inverted and the small drain-source voltage makes the channel behave like a voltage-controlled resistor. In the saturation region the channel pinches off near the drain and the drain current depends primarily on the gate overdrive, while remaining only weakly dependent on V_{DS} through the channel-length modulation term [16], as illustrated in Figure 2.3.

As illustrated in Figure 2.4, the transconductance and output conductance are obtained by differentiating the saturation-region drain current at the DC operating point. The transconductance g_m and the output conductance g_{ds} are [16]

$$g_m = \left. \frac{\partial I_D}{\partial V_{GS}} \right|_Q = \mu C_{ox} \frac{W}{L} V_{ov} (1 + \lambda V_{DS}), \quad g_{ds} = \left. \frac{\partial I_D}{\partial V_{DS}} \right|_Q = \frac{1}{2} \mu C_{ox} \frac{W}{L} V_{ov}^2 \lambda, \quad (2.2)$$

where Q denotes evaluation at the operating point. These two quantities are used to approximate the small-signal gain and frequency response of the amplifier.

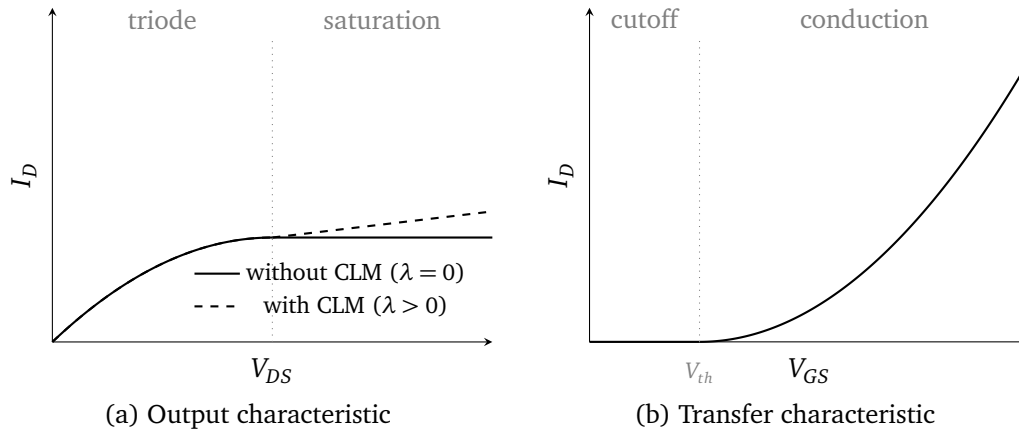


Figure 2.3: Idealized MOSFET characteristics. (a) Drain current I_{DS} versus V_{DS} at fixed V_{GS} , showing the triode and saturation regions. Without channel-length modulation (CLM) the saturation current is independent of V_{DS} , the $(1 + \lambda V_{DS})$ term adds the finite slope that influences the output conductance g_{ds} . (b) Drain current versus V_{GS} at fixed V_{DS} , showing the cutoff region below the threshold voltage V_{th} and the square-law curve above it.

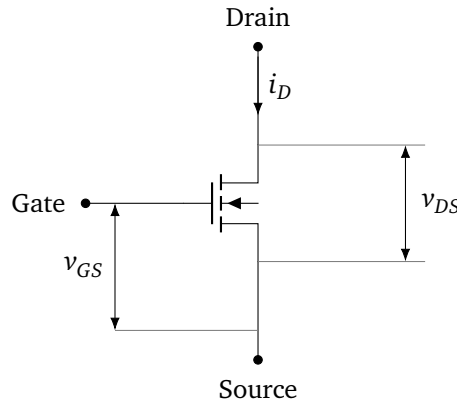


Figure 2.4: Terminal voltages and drain current of an n-channel MOSFET. The small-signal transconductance $g_m = \partial i_D / \partial v_{GS}$ and output conductance $g_{ds} = \partial i_D / \partial v_{DS}$ are obtained by differentiating the drain current at the DC operating point.

These regional equations are the basis for almost all hand analysis and for many equation-based sizing methods, because they make the relationship between bias conditions and device behavior explicit. They are also natural candidates for physics constraints investigated later in this thesis.

2.2.3 Limits of Simplified Equations

The simplified regional equations and the BSIM4 model do not describe the same device behavior. The analytical equations omit or oversimplify effects that become significant as MOSFETs shrink. As a result, they can deviate substantially from the behavior that BSIM4 and SPICE actually produce.

Channel-length modulation is a concrete example. The saturation expression in (2.1) represents it through the single linear factor $(1 + \lambda V_{DS})$ with a constant coefficient λ and this term is the only source of the output conductance g_{ds} in (2.2). If λ were zero the device

would behave as an ideal current source in saturation, with a drain current independent of V_{DS} and a vanishing g_{ds} . The entire predicted output conductance therefore rests on this one constant. However, in a short-channel device, the dependence of the drain current on V_{DS} is not described by a single constant. The true behavior varies with the operating point and with device geometry, so the value of λ that fits the device in one circuit does not fit it in another [16].

The same pattern affects the other parameters. The carrier mobility μ is treated as a constant, but in a real device it changes with the vertical field set by the gate voltage and the lateral field set by the drain voltage [17]. The simplified expression represents quantities that vary with the operating point by fixed constants. This is one of the reasons the results drift away from BSIM4 as device dimensions shrink. Another reason is that BSIM4, as stated in the previous Section 2.2.2, consists of hundreds of parameters, which are not modeled in the simplified analytical equations. On top of that the relationships built from these expressions inherit and amplify the error. For example, the assumption that two identically drawn transistors in a current mirror carry equal currents is sensitive to threshold-voltage variation and to drain-voltage differences that the simplified model does not capture [16].

On the other hand, not all relationships are affected this way. The conservation of current at a node, also known as Kirchhoff’s Current Law, holds regardless of the device model and is exact under BSIM4. It requires that the currents entering and leaving every node sum to zero.

This has a direct consequence for physics-informed modeling. A physics-informed training scheme injects prior knowledge by penalizing a model for violating a physical relationship. If the relationship also holds for the true device behavior, the penalty should be large to provide a stronger learning signal. If it is a simplified equation that the true behavior does not satisfy, the penalty should be tuned, as it might otherwise push the model away from the correct answer. Determining which analytical relationships remain trustworthy, and which do not, is examined in Section 4.3.1.

2.3 Graph Neural Networks

A circuit is naturally a graph. Devices and nets are nodes and their interconnections represent the edges between them. Graph neural networks are the class of models designed to learn directly on this kind of structured data. They also are the model family on which the method in this thesis is built. This section introduces the message-passing mechanism of graph neural network, the way circuits can be represented as graphs and how can GNN be used for circuit.

2.3.1 Message Passing and Aggregation

A graph neural network operates on a graph where nodes and edges carry feature vectors. It learns representations by repeatedly exchanging information along the edges. In the message-passing formulation, each layer updates the representation of every node by aggregating information from its neighbours and combining it with the node’s own embedding [20]. For the feature vector $h_v^{(l)}$ of node v at layer l , $\mathcal{N}(v)$ for the set of neighbours of v and e_{uv} for the features of the edge between u and v , one message passing layer can be written as

$$h_v^{(l+1)} = \gamma^{(l)} \left(h_v^{(l)}, \bigoplus_{u \in \mathcal{N}(v)} \phi^{(l)}(h_v^{(l)}, h_u^{(l)}, e_{uv}) \right), \quad (2.3)$$

where $\phi^{(l)}$ is a learnable message function, $\oplus$ is a permutation-invariant aggregation over the neighbours and $\gamma^{(l)}$ is a learnable update function. Stacking L such layers allows information to propagate over paths of length L , so that after several layers a node's representation exchanges information with large neighbourhood around it, as illustrated in Figure 2.5.

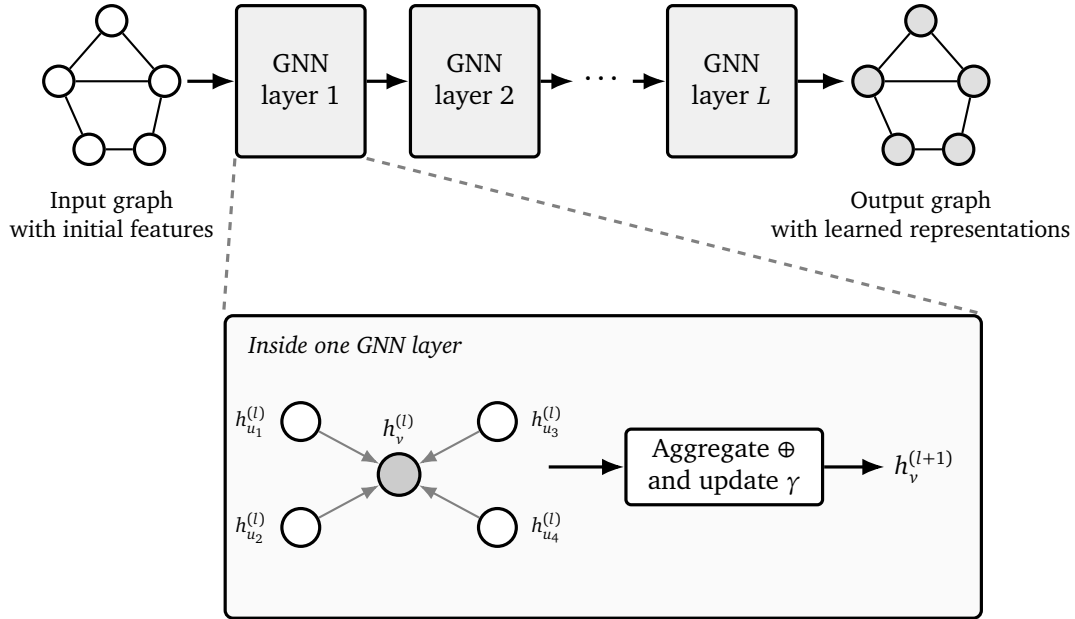


Figure 2.5: A graph neural network as a layered pipeline (top) and the operation inside one layer (bottom). The input graph carries an initial feature vector on every node. L message-passing layers update these features. For the node $h_v^{(l)}$, information are aggregated from the neighbours $h_{u_1}^{(l)}, \dots, h_{u_4}^{(l)}$ by a permutation-invariant operator $\oplus$. Then it is combined with the node's own embedding by an update function γ to produce the new representation $h_v^{(l+1)}$.

The aggregation $\oplus$ must be permutation-invariant, because the neighbours of a node have no inherent ordering. Common choices are the sum, the mean or the maximum and the choice depends on the network's task [21].

2.3.2 Common Architectures

Different GNN architectures uses the general message-passing equation (2.3) with different message and aggregation functions. The graph convolutional network uses a normalized neighbourhood averaging followed by a linear and a nonlinear transformation [22], which for a layer with weight matrix $W^{(l)}$ is written as

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{d_v d_u}} W^{(l)} h_u^{(l)} \right), \quad (2.4)$$

where d_v is the degree of node v and σ is a nonlinearity. The graph isomorphism network uses a sum aggregation with a learnable self-weight, followed by a multilayer perceptron. It

is known for its power to distinguish different graph structures [21]:

$$h_v^{(l+1)} = \text{MLP}^{(l)} \left((1 + \epsilon^{(l)}) h_v^{(l)} + \sum_{u \in \mathcal{N}(v)} h_u^{(l)} \right). \quad (2.5)$$

The graph attention network weights each neighbour by a learned attention coefficient α_{vu} , so that the aggregation emphasizes more relevant neighbours. In the GATv2 formulation, the difference lies in the scoring function, which applies the nonlinearity before the learnable projection, yielding a more expressive, input-dependent attention [23]:

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu} W^{(l)} h_u^{(l)} \right), \quad (2.6)$$

with the attention coefficient

$$\alpha_{vu} = \text{softmax}_u \left(a^\top \text{LeakyReLU} \left(W^{(l)} [h_v^{(l)} \parallel h_u^{(l)}] \right) \right), \quad (2.7)$$

where $[\cdot \parallel \cdot]$ denotes concatenation, $W^{(l)}$ and a are learnable. The LeakyReLU is applied before the projection by a , which is what distinguishes GATv2 from GATv1. The GraphSAGE family generalizes the aggregation to functions such as mean, pooling or recurrent aggregators and adds neighbour sampling to scale to large graphs [24].

A practical limitation of these architectures is that performance degrades as depth increases, because repeated aggregation tends to make node representations converge, also known as over-smoothing. The DeeperGCN family addresses this with a generalized, parameterized aggregation together with residual connections [25]. Its generalized message aggregation replaces the fixed sum or mean with a differentiable family controlled by a parameter, for example the softmax-weighted form

$$\bigoplus_{u \in \mathcal{N}(v)} m_u = \sum_{u \in \mathcal{N}(v)} \frac{\exp(\beta m_u)}{\sum_{w \in \mathcal{N}(v)} \exp(\beta m_w)} m_u, \quad (2.8)$$

where m_u is the message from neighbour u and β interpolates between mean-like behaviour at $\beta \rightarrow 0$ and max-like behaviour at large β . This makes it possible to train deeper graph networks without the usual over-smoothing effect.

2.3.3 Graph Representations and GNN for Electronic Circuits

The choice of how to map a circuit netlist to a graph strongly influences what the model can learn. The most direct representation would be to make every device and every net a node, with edges connecting a device to the corresponding net. This bipartite device-net representation preserves the full circuit topology. Alternative representations either collapse nets into edges, trading topological fidelity for a simpler graph or introduce heterogeneous node and edge types to distinguish device classes.

Graph-based models have been applied across several circuit problems. They have been used to predict layout parasitic capacitances and resistances before the physical layout is produced before the layout is produced [10], to guide analog placement [11] and to predict performance from circuit-graph representation [9].

These works differ in their graph construction and in their prediction targets, but they share the same foundation, a graph carries information that flat, structure-free feature vectors discard. They use that structure differently, for the parasitic and placement work focuses on which components are physically near or connected to each other, while the topology-level work focuses on representing the circuit as a whole.

2.4 Physics-Informed Neural Networks

A physics-informed neural network is a neural network whose training is biased toward solutions that respect known physical laws. The bias can be introduced by penalizing violations of those laws during training, by constructing the network so that the laws are satisfied by construction, or by a combination of the two. This section introduces the original physics-informed neural network formulation, the distinction between soft and hard constraints, and how the approach has been applied to circuits and devices.

2.4.1 Foundations and the Soft-Hard Constraint Distinction

The original formulation of physics-informed neural networks targets the solution of forward and inverse problems for partial differential equations. A neural network is used to represent the unknown solution as a function of space and time. Then the model's output is evaluated against the physics equation, and the resulting residual is added to the loss as a penalty. The total training objective is of the form [12]

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{physics}}, \quad (2.9)$$

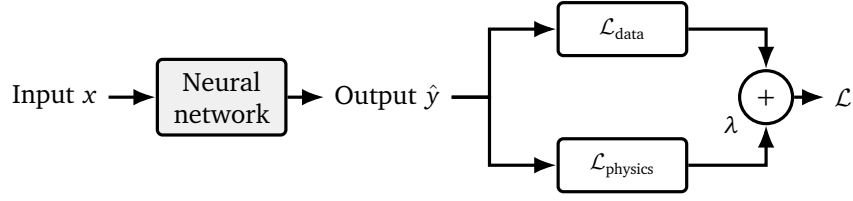
where $\mathcal{L}_{\text{data}}$ is a supervised loss that measures the agreement between the network's predictions and observed data, $\mathcal{L}_{\text{physics}}$ is a residual loss that measures the violation of the physical law and $\lambda \geq 0$ is a weight that balances the two terms. For a partial differential equation of the form $\mathcal{F}[u](x) = 0$, where $\mathcal{F}$ is a differential operator and u is the unknown solution represented by the network u_θ , the physics term is typically

$$\mathcal{L}_{\text{physics}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \|\mathcal{F}[u_\theta](x_i)\|^2, \quad (2.10)$$

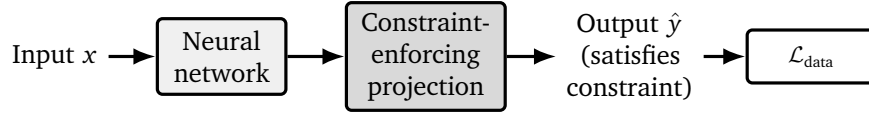
evaluated at N_c points $\{x_i\}$ in the domain. Training minimizes a combination of the data-fitting error and the violation of the underlying equation. The resulting network approximates a function that fits the data and approximately satisfies the equation. This template has since been adopted well beyond partial-differential-equation settings. Whenever a physical relation can be evaluated on a network's predictions, its residual can be added to the loss [26].

A design choice is whether a physical law is enforced softly or hardly [26], illustrated in Figure 2.6. In a soft formulation the law appears as a penalty term in the loss, as in (2.9). The network is rewarded for satisfying the law but is not required to do so exactly and any differentiable physical relation can be turned into such a penalty. The penalty gives a learning signal that biases the network toward physically consistent predictions.

In a hard formulation the law is built into the architecture itself so that the model's prediction satisfies it by construction. Hard constraints guarantee exact satisfaction of the law but require an architecture that can express it in closed form. This is not always possible or desirable when the law itself is only an approximation.



(a) Soft constraint: physics enters the loss



(b) Hard constraint: physics enters the architecture

Figure 2.6: The two ways of incorporating a physical law into a neural network. (a) In a soft formulation, the network's output is evaluated against both a data loss $\mathcal{L}_{\text{data}}$ and a physics loss $\mathcal{L}_{\text{physics}}$. The law is respected only approximately. (b) In a hard formulation, a constraint-enforcing projection is built into the architecture, so that the output satisfies the law by construction and only the data loss is needed.

2.4.2 Applications to Circuits and Devices

The physics-informed model has been applied to a many of problems in circuits and electrical devices. One work uses physics-informed neural networks for circuit analysis with an emphasis on dielectric material modelling, where the physics is described by differential equations that capture how the circuit evolves over time [14]. Another work proposes a physics-informed Fourier neural network for circuits, using a Fourier-series-structured network trained with physics-informed losses to generalize across different RLC circuit topologies and component values via transfer learning [13]. These works show that the physics-informed idea is transferable from partial-differential-equation into the electrical and circuit domain.

They also share the same underlying structure, which is a fully connected network operating on flat feature representation and the physics are enforced as an analytical relation between those features. The topology of the actual circuit is not embedded in the model architecture.

2.5 Classical and Equation-Based Sizing Methods

Before the machine-learning surrogates, analog sizing problem was solved almost exclusively through equation-based methods. It was done either by hand or with the help of optimization tools that used closed-form device and circuit models. These methods remain widely used today, because they make circuit behavior explicit and they require no training data. This section reviews the two approaches to equation-based sizing. The manual and structured design Methodologies, and the constraint-programming and equation-based solvers that automate the same process.

2.5.1 Manual and Structured Design Methodologies

The traditional approach to analog sizing is hand calculation based on the simplified device equations introduced in Section 2.2. A designer chooses an architecture, identifies the most important transistor parameters that control each performance specification and solves the resulting equations to obtain a candidate sizing. The simplified equations make this analysis tractable. The saturation-region drain current, the small-signal transconductance and output conductance and the matching relationships used in current mirrors are all available in closed form [16]. However, these methods does not scale, since the number of degrees of freedom in a moderate op-amp can run into the tens and on top of that, the simplified equations diverge from the simulated behavior, so the candidate sizing must be tuned through repeated simulation before it meets desired specification.

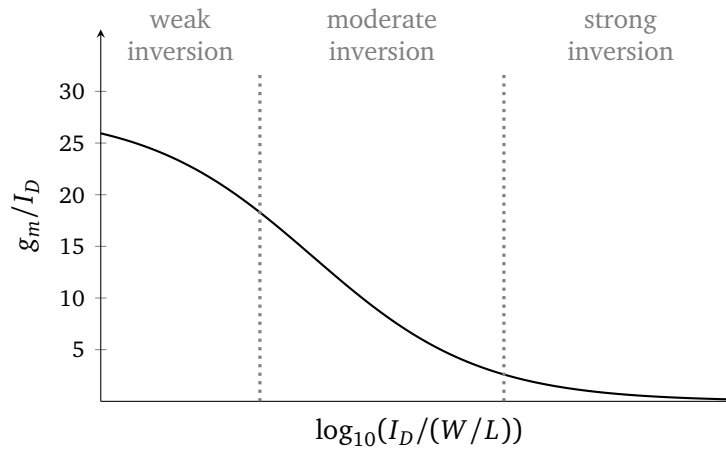


Figure 2.7: The g_m/I_D curve, the central plot in the manual design methodology. At low current density the device operates in weak inversion with a high g_m/I_D , at high current density it operates in strong inversion with much lower g_m/I_D . The methodology uses simulated or measured g_m/I_D characteristics to pick an inversion level for each transistor and read off the required device sizes.

To structure this process and reduce the trial-and-error component, the gm/Id methodology was developed, to replace closed-form analytical relations with tables of measured or simulated device characteristics, indexed by gm/Id [17]. The shape of the g_m/I_D plot is illustrated in Figure 2.7. It goes from a high transconductance per unit current in weak inversion to a much lower value in strong inversion. The designer chooses first the inversion-level for each transistor and read off the required device sizes from the tables, rather than solving the simplified equations and then correcting them. The tables capture the actual device behavior and therefore avoid the deviation from BSIM4. The cost is that the methodology operates at the level of individual transistors and small subcircuits. And circuit-wide effects such as the frequency response and the interactions between stages still require simulation to verify, and the lookup tables themselves must be regenerated whenever the technology changes.

2.5.2 Constraint-Programming and Equation-Based Solvers

The second line of work automates the equation-based method by modeling analog sizing as a constraint- or numerical-optimization problem using the simplified device and circuit equations. The circuit defines a system of equations and a solver searches the design space for a device sizing that satisfies them. The COPRICSI framework is a example, formulating

the sizing of basic op-amp stages as a constraint optimization problem [27]. Hierarchical libraries of performance equations have been developed to support such solvers, decomposing the op-amps into groups with explicit per-group performance expressions that combine into circuit-level performance metrics [27].

These methods share the strengths and limitations of the equation-based approach. Their solutions require no training data and has a clear analytical meaning. However, they are also bound by the same fidelity problem as manual hand analysis. The constraints they enforce are simplified equations rather than BSIM4 behavior, so a sizing that satisfies the analytical constraints still requires simulation verification and refinement. The goal of this thesis is also to accelerate analog sizing, but by replacing the simulator inside the optimization loop with a learned surrogate rather than replacing simulation itself with an analytical model.

2.6 Machine-Learning Surrogates for Analog Sizing

The cost of repeated SPICE evaluation discussed in Section 2.1 has motivated many works to replace the simulator, or parts of it, with a learned surrogate. The surrogate is trained on a set of simulated examples and is then used in place of SPICE, with the aim to obtain a reasonable answer at a fraction of the cost. This section reviews two types of surrogate model: one used to approximate the circuit’s behavior, the other embedded in an optimization loop.

2.6.1 Supervised Surrogates: Neural Networks, Gaussian Processes, and Transformers

The most direct application of machine learning to analog sizing is to train a supervised regression model that maps a candidate sizing to its simulated performance metrics. Early surrogates were built on Gaussian-process regressors, using the posterior to acquire both the mean and the uncertainty estimate of the sizing [4]. The same overall structure has been adopted with fully connected neural networks, which scale better to high-dimensional sizing spaces and to larger training sets, at the cost of the explicit uncertainty estimate [3]. A studies across several benchmark circuits report similar accuracy and the model choice was mainly affecting the training cost, the data efficiency and the ease of integrating the surrogate into an optimization loop [3].

More recent work explores stronger architectural priors for the surrogate. Transformer-based models have been used to predict transistor sizes, with precomputed lookup tables of device characteristics (g_m/I_D plot) as additional inputs to the model [28]. Large language model agents have also been proposed as automated sizing assistants that interact with simulators and reason about circuit topologies [5]. These approaches all share a supervised-surrogate core. A model is trained to approximate the relationship between a sizing and its performance. They treat the candidate sizing as a flat vector, with no explicit structure encoding how the devices are connected.

2.6.2 Surrogate-Assisted Optimization Loops

A separate line of work focuses on how the surrogate is used inside an outer optimization rather than replacing the SPICE itself. The general idea, often called surrogate-assisted optimization, runs the surrogate and the simulator in an alternating loop. The cheap surrogate

is used to propose promising candidate sizings and the expensive simulator then evaluates them to obtain their true performance. Each simulated result is added back to the training set and the surrogate is retrained, so it becomes progressively more accurate in exactly the regions of the design space that the search is exploring [2, 4]. The same pattern has been combined with reinforcement-learning agents that choose the next sizing to evaluate, treating the sizing process as a sequential decision problem and the simulator as the environment that returns the resulting performance [6, 8]. Studies report substantial reductions in simulator evaluations when surrogates are used in this assisted fashion [7].

2.7 Positioning of This Work

The previous sections covered four areas of work relevant to this thesis. Equation-based sizing methods, both manual and automated, rely on simplified device equations, hence a candidate sizing obtained from these methods typically requires simulation verification and refinement (Section 2.5). Machine-learning surrogates accelerate this verification by replacing the simulator with a learned regression model, using Gaussian processes, neural networks, transformers and language model agents (Section 2.6). Graph neural networks have been applied to circuits at the layout, placement and topology levels, exploiting the circuit structure that flat representations discard (Section 2.3). Physics-informed methods have been applied to electrical and device problems, enforcing analytical physical relations through residual terms in the loss (Section 2.4).

The contribution of this thesis is illustrated in the following Figure 2.8.

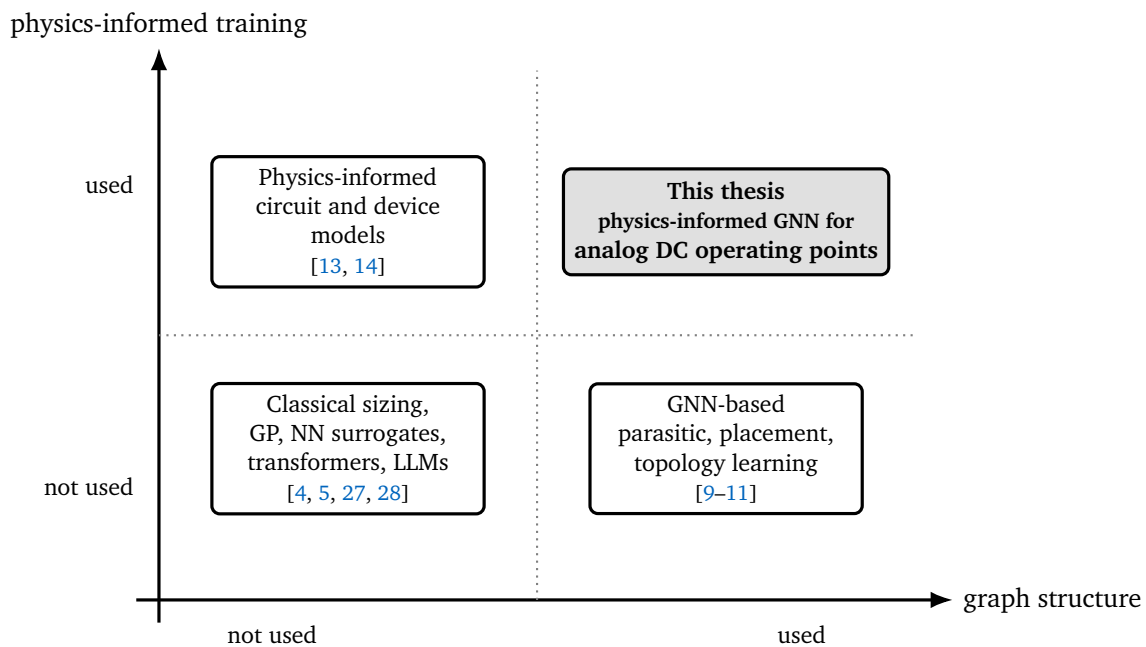


Figure 2.8: Positioning of this work. Each quadrant lists the works from the similar area reviewed in this chapter. The combination of a graph representation with a physics-informed training objectives, in the top-right quadrant, is what this thesis addresses.

First, the surrogate’s prediction target is the DC operating point with the per-transistor small-

signal parameters g_m and g_{ds} . This is a lower-level target than the performance metrics predicted by most learned surrogates and it has two consequences. It allows the DC gain and other performance quantities to be computed analytically from the predicted parameters, and allow the surrogate to slightly correct them. It also exposes the predicted values to conservation laws and other approximate analytical relations, which would not be possible if the surrogate predicted performance directly.

Second, the model is a graph neural network whose structure follows the device-net representation of the circuit. Every device terminal and net is a node and with intra-device connections to preserve device identity. This is a more detailed representation than the device-as-node and topology-level graphs used in earlier graph-based circuit work (Section 2.3.3). This representation is also motivated by the conservation law being enforced, which acts on the edges incident to each net node.

Third, the training objective combines a supervised data loss with a physics-informed soft penalty on the violation of Kirchhoff's current law at every internal net. Section 2.2 argued that physics constraints must be chosen carefully, because the simplified device equations deviate from the BSIM4 behavior that defines the ground truth. This means the analytical equations need to be explored and analysed before they are used as a training loss.

This thesis develops an approach that combines these three main choices. The graph neural network's representation matches the circuit, the prediction target is at the level of operating-point quantities and a soft physics-informed penalty enforces current conservation.

Chapter 3

Methodology

This chapter describes the methodology developed in this thesis. The goal is a learned surrogate that, given sized two-stage and three-stage operational amplifiers, predicts the operating point, the per-transistor small-signal parameters and DC gain. The surrogate is a graph neural network operating on a device-net representation of the circuit, trained on SPICE generated data with a supervised loss and physics-informed soft and hard constraints.

Section 3.1 describes the circuit topology, the process and the simulator used as ground truth. Section 3.2 defines the graph representation of the circuit, including node and edge features. Section 3.3 explains the data statistics and how it is generated. Section 3.4 specifies the different model architectures. Section 3.5 formalizes the training objective, including the data loss and the physics used for training. Section 3.6 states the experimental protocol used for evaluation.

3.1 Circuit, Process and Ground Truth

The methodology is evaluated on five three-stage operational amplifiers from the AnalogGym benchmark suite [15]. A simpler two-stage design was used only in early testing and is not part of the main evaluation. Using several topologies rather than a single architecture tests whether the surrogate generalizes across circuits of different structure and complexity. The five topologies are summarized in Table 3.1.

Identifier	Full name	Compensation scheme
fan_smc	Fan single Miller compensation	classic three-stage
sau_cfcc	Sau cross-feedforward cascode comp.	feedforward
peng_tcfc	Peng transconductance feedforward comp.	feedforward
leung_nmcf	Leung nested Miller with feedforward	feedforward, nested
leung_nmcnr	Leung nested Miller, no nulling resistor	nested, no resistor
two_stage	Two-stage Miller compensation	classic two-stage

Table 3.1: The six operational amplifier topologies used for evaluation and testing.

The five three-stage topologies share similar three gain stages but differ in how they are compensated for stability. The Fan single-Miller-compensated amplifier is the classic three-stage design, using a single Miller capacitor for dominant-pole compensation. The Sau cross-feedforward cascode-compensated amplifier and the Peng transconductance-feedforward-compensated amplifier introduce feedforward paths around the gain stages. The Leung

nested-Miller amplifier with feedforward combines nested Miller compensation with a feed-forward path and the Leung nested-Miller amplifier without a nulling resistor uses nested compensation without the series resistor. The sixth topology is a two-stage Miller-compensated amplifier, as a structurally simpler starting point.

Each topology has a set of design parameters, the transistor widths, lengths and passive device values. Choosing values for these parameters within their ranges and running the simulator determines the circuit’s quantities of interest. The surrogate is trained and evaluated on these samples, with the ranges and sampling described in Section 3.3.1.

All six topologies are implemented in the SkyWater SKY130 open-source process design kit, which provides BSIM4 compact models for the n-channel and p-channel transistors [15, 19]. These models define the device behaviour that the simulator uses for the simulation. The simulation settings are summarized in Table 3.2.

Setting	Value
Process design kit	SkyWater SKY130 (open-source)
Device flavor	1.8 V core (nfet_01v8, pfet_01v8)
Process corner	typical-typical (TT) only
Temperature	NGSPICE default ($\approx 27^\circ\text{C}$)
Device mismatch	disabled
Supply voltage	fixed at 1.8 V

Table 3.2: Simulation settings used to generate the ground-truth data.

The SKY130 process offers several device flavors for different supply voltages. All circuits here use the 1.8 V core devices. Simulations are run at a single process corner, typical-typical (TT) and device mismatch, which is the small random parameter variation between identical transistors, is disabled, so that every transistor behaves identically. With these values fixed, the only quantity that varies between samples is the device sizing.

Simulator NGSPICE returns the internal node voltages, the terminal currents, the per-transistor small-signal parameters g_m and g_{ds} (Figure 2.4) and the circuit-level metrics such as DC gain, unity-gain bandwidth, phase margin, and gain margin. These quantities form the ground-truth data.

The DC gain is the gain at low frequency and the unity-gain bandwidth is the frequency at which this gain falls to unity (0 dB). The phase margin and gain margin, illustrated in Figure 3.1, are stability metrics for the amplifier in feedback. The phase margin is the distance of the phase above -180° at the unity-gain frequency and the gain margin is the amount by which the gain has already fallen below 0 dB at the frequency where the phase reaches -180° . Larger margins indicate a more stable amplifier.

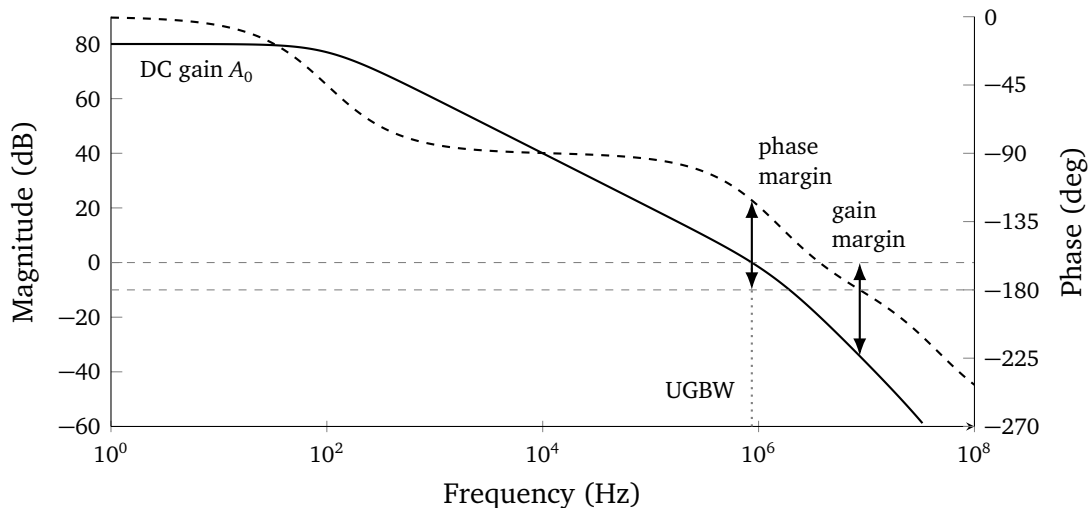


Figure 3.1: Open-loop frequency response of an operational amplifier. The solid curve is the magnitude and the dashed curve is the phase response.

3.2 Circuit Representation

The model operates on a circuit netlist graph. Every device terminal and every net becomes a node, edges connect each terminal to the corresponding net and additional edges connect the terminals of the same device together. This representation preserves the exact connectivity and detail information of the circuit. This section describes how a netlist is mapped to a graph and what features the nodes and edges carry.

3.2.1 Graph Reconstruction

The graph has two kinds of nodes. Each device contributes one node for every terminal it possesses. For example, a transistor gives four terminals (gate, drain, source and bulk) and a resistor gives two terminals (positive and negative). Each net contributes one net node, including the supply rails V_{DD} and V_{SS} as well as the input and output nets. A circuit with N device terminals and D nets therefore produces a graph with $N + D$ nodes.

As for the edges, there are two types. A net-terminal edge connects a net node to the corresponding device terminal node. An intra-device edge connects the terminals of the same device to one another, so a four-terminal transistor contributes six intra-device edges and a two-terminal passive contributes one. All edges are undirected.

Each device in the netlist is first expanded into its terminal nodes, each net is created as a net node, a terminal-net edge is added for every terminal-to-net connection and the intra-device edges are then added on top. Figure 3.2 shows the result for a small circuit. The transistor M_1 contributes four terminal nodes, the resistor R_D contributes two and these connect through terminal-net edges to the net nodes V_{in} , V_{out} , V_{DD} , and V_{SS} , while the dashed intra-device edges link the terminals of same device together.

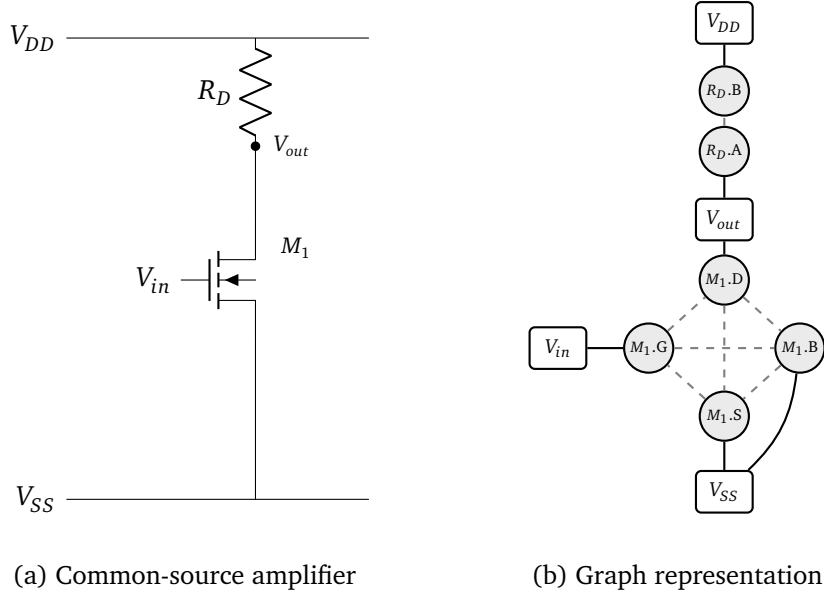


Figure 3.2: Mapping a circuit to its graph representation. (a) A common-source amplifier. (b) The corresponding graph. Each device terminal and each net is a node, solid edges connect each terminal to a net and dashed edges are the intra-device connections. The transistor M_1 contributes four terminal nodes and the resistor R_D two. The supply rails V_{DD} and V_{SS} appear as ordinary net nodes.

3.2.2 Node and Edge Features

Each node carries a feature vector x formed by concatenating four parts: per-terminal physical properties, global quantities, which are broadcasted to every node, a one-hot encoding of the node type and a one-hot encoding of the net role. These give a 26-dimensional input per node. The composition is summarized in Table 3.3.

Component	Dim	Notes
Device properties	4	per-terminal, padded (Table 3.4)
Global context	5	broadcast to every node (Table 3.5)
Node type	12	one-hot over node categories (Table 3.6)
Net role	5	one-hot, net nodes only (Table 3.7)
Total	26	concatenated at the input

Table 3.3: Composition of the node feature vector x .

The first four entries (Device properties) hold the physical properties of the terminal, which depend on the device it belongs to, as listed in Table 3.4. For a MOSFET, M is the multiplier, the number of identical devices connected in parallel. These are log-normalized, zero-padded to a common width and net nodes carry no physical properties.

Node kind	Raw features	Dim
MOSFET terminal (g/d/s/b)	$[W, L, W/L, M]$	4
Resistor / capacitor (p/n)	[value]	1
Voltage / current source (p/n)	[dc]	1
Net node	—	0

Table 3.4: Per-terminal physical properties (the first four entries of x), log-normalized and zero-padded to a common width.

The next five entries are global context broadcasted to every node, listed in Table 3.5. The supply voltage, the common-mode voltage, the two differential input voltages and a reference current, all normalized, giving every node access to the global conditions of the sample.

Entry	Quantity
V_{DD}	supply voltage
V_{cm}	common-mode voltage
$V_{in,p}$	positive differential input
$V_{in,n}$	negative differential input
$\log_{10} I_{ref}$	reference current (log-scaled)

Table 3.5: Global context (entries 5–9 of x), broadcasted to every node, all normalized.

The remaining two parts are one-hot encodings. The node type encoding is a hierarchical one-hot, as listed in Table 3.6 and the net-role encoding labels the semantic function of each net, distinguishing input, output, internal, bias and supply nets, as listed in Table 3.7. The net-role encoding applies only to net nodes. Terminal node belong to a device carry an all-zero net-role vector.

Level	Dims	Encodes
Device class	6	M, R, C, V, I, VNode
Subclass	2	MOSFET: NMOS / PMOS; R,C,V,I: p / n; VNode: ground / non-ground
Terminal	4	drain, gate, source, bulk (MOSFET only, zero for others)

Table 3.6: Node type encoding, a 12-dimensional hierarchical one-hot. The subclass slot is reused across device classes and the terminal slot is filled only for MOSFETs.

Index	Role
0	input
1	output
2	internal
3	bias
4	supply

Table 3.7: Net-role encoding, one-hot applied only to net nodes.

Now for to the edges. Each edge carries a six-dimensional one-hot encoding identifying which terminal of the connected device it represents, shown in Table 3.8. An edge records not just

that a terminal connects to a net but which terminal it is, so a gate connection can be told apart from a drain connection. Each edge is undirected and both directions carry the same encoding.

Terminal kind	Index
gate	0
drain	1
source	2
bulk	3
p	4
n	5

Table 3.8: Edge feature encoding, a 6-dimensional one-hot depending on the terminal node it is connected to.

A compressed representation of each device’s IV surface was also explored as an additional input, but degraded the results and therefore was not adopted (Appendix A.9).

3.3 Data

This section covers how the training data is produced and what it looks like. Section 3.3.1 describes how the sizings are sampled, simulated and turned into labeled graphs. Section 3.3.2 shows the resulting data, the ranges of the target quantities and how they differ across the topologies. Section 3.3.3 describes different datasets used and their splits.

3.3.1 Data Generation

The training data is generated by sampling sizings for each topology, simulating each sizing and saving the resulting operating point as a labeled graph. Sizings are drawn with Latin hypercube sampling, which spreads the samples more evenly over the design space than independent uniform sampling. Each parameter is sampled on the unit interval and then mapped to its physical range on either a logarithmic or a linear scale. Device geometries, bias current and capacitances are sampled logarithmically, since they span several orders of magnitude, while the bias voltages are sampled linearly. The parameter ranges are listed in Table 3.9.

Parameter	Range	Scale
Width W	$0.42\mu\text{m}$ – $50\mu\text{m}$	log
Length L	$0.15\mu\text{m}$ – $20\mu\text{m}$	log
Multiplier M	1–100 (integer)	log
Bias current I_{BIAS}	$0.5\mu\text{A}$ – $100\mu\text{A}$	log
Compensation cap C_{COMP}	0.5 pF–30 pF	log
Common-mode voltage V_{CM}	0.7 V–0.9 V	linear

Table 3.9: Sampled parameter ranges, same across all topologies. Transistors that are grouped share one sampled (W, L, M) triple.

Rather than sampling every transistor independently, the transistors of each topology are clustered into groups that share a single sampled (W, L, M) triple. The grouping is taken from

the AnalogGym benchmark [15]. The number of groups varies with the topology, from seven for the simplest three-stage design to nine for the topologies with additional cascode and feedforward stages. The supply voltage is fixed at 1.8 V and the feedback resistors at 50 k Ω , so these are not sampled.

For each sampled sizing, the parameters are substituted into a SPICE netlist template and simulated with NGSPICE. A DC operating-point analysis and a separate AC analysis are run, and the resulting circuit is assembled into the graph representation described in Section 3.2. The graph sizes of the five three-stage topologies are listed in Table 3.10.

Each graph is labeled with the quantities the surrogate is trained to predict. From the DC analysis these are the voltage at every internal net, the drain and source currents of every transistor and the terminal currents of every resistor, together with the per-transistor small-signal parameters g_m and g_{ds} . From the AC analysis the circuit-level metrics are also recorded, the DC gain, the unity-gain bandwidth and the phase and gain margins (Figure 3.1). Of these only the DC gain is predicted in this work and the reason is elaborated in Section 4.3.1. The supply-rail voltages are fixed and not predicted, leaving the internal net voltages as the voltage targets. The transistor gate and bulk terminals and the capacitor terminals carry no current at the DC operating point, so their currents are zero and are not predicted.

Topology	MOSFETs	Passives	Nodes	Edges	Internal nets
fan_smc	24	7	129	522	14
sau_cfcc	24	7	129	522	14
peng_tcfc	32	8	170	688	21
leung_nmcf	24	10	136	540	14
leung_nmcnr	24	9	134	534	15

Table 3.10: Per-topology component and graph sizes. “Internal nets” are the net voltages the model predicts.

A sampled sizing is rejected only if the simulation fails to converge or if any internal node voltage falls outside the supply rails, below the ground rail or above 1.8 V, which is physically impossible. No filter is applied on operating region or on small-signal stability, so that the dataset does not consists only well-behaved sizings.

3.3.2 Dataset Statistics

This subsection looks at what the training data actually looks like, the spread of the target values across the five topologies. The reason to do this is that the design space turns out to hold many badly conditioned circuits, similar at the device level but quite different at the circuit level. Throughout, *fan_smc* is the topology the method was built on, and the other four were added later for the generalization study.

Table 3.11 sums up the data per topology and Figure 3.3 shows the distributions. The mean current and small-signal parameters are close across topologies, each spans many orders of magnitude, with the current distributions of the five sitting almost on top of each other (Figure 3.3a). The full ranges are in Appendix A.1.

The DC gain is where the topologies split. The last two columns of Table 3.11 show the mean

gain going from -77 dB for `leung_nmcnr` up to $+33$ dB for `leung_nmcf` and the share of circuits that actually amplify going from 8% to 77%. The gain distributions can be seen in Figure 3.3b. So the topologies are alike at the device level but differ on the circuit level.

Topology	V (V)	$\log_{10} I $	$\log_{10} g_m$	$\log_{10} g_{ds}$	Gain (dB)	Amplifying
<code>fan_smc</code>	0.61	-5.56	-4.43	-5.89	-56.9	17%
<code>sau_cfcc</code>	0.61	-5.67	-4.66	-6.11	-59.6	12%
<code>peng_tcfc</code>	0.87	-5.39	-4.24	-5.53	-35.9	24%
<code>leung_nmcf</code>	0.58	-5.42	-4.15	-5.96	$+33.0$	77%
<code>leung_nmcnr</code>	0.63	-5.47	-4.34	-5.83	-77.3	8%

Table 3.11: Per-topology summary of the target quantities. “Gain” is the mean DC gain in decibels and “Amplifying” the fraction of sampled circuits with a positive gain. The device-level quantities are comparable across the topologies, while the gain differs a lot. Full ranges and standard deviations are in Appendix A.1.

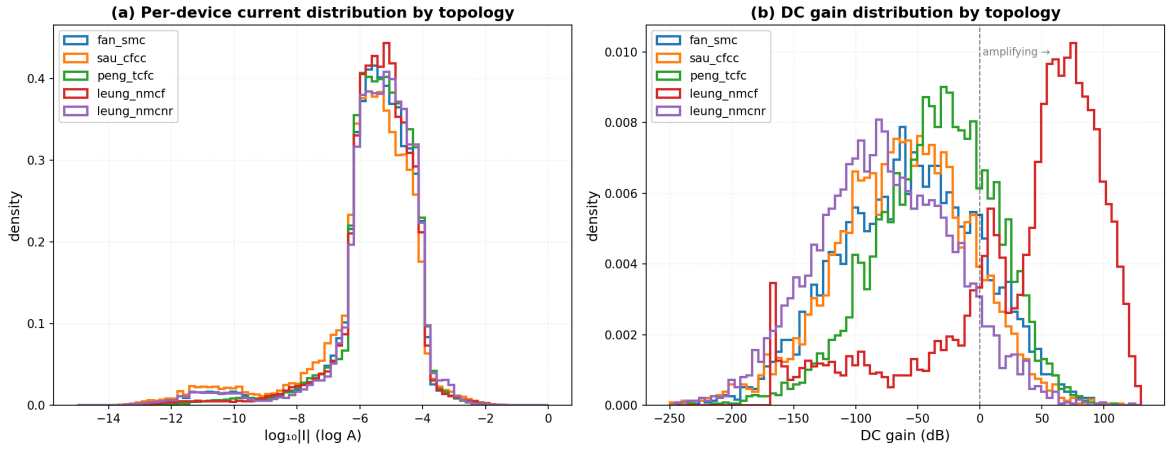


Figure 3.3: Distributions of two target quantities across the five topologies. (a) The per-device current ($\log_{10} |I|$) is distributed almost identically across the topologies. (b) The DC gain sits well below the amplifying threshold for most topologies, except `leung_nmcf`.

It also helps to look inside a single topology. Figures 3.4 break the distributions down by device for `fan_smc`, with the transistors grouped by the stage. Two things stand out. The bias network and the early devices barely move, their currents and small-signal parameters hardly change. The output devices are the opposite. Within each stage, the widest swing is always at that stage’s output device, whose current and conductances spread over many decades.

The targets are normalized before training with statistics taken from the training set, described in Section 3.5.1. Since the device-level quantities are close across topologies, one set of normalization statistics is enough for the multi-topology models.

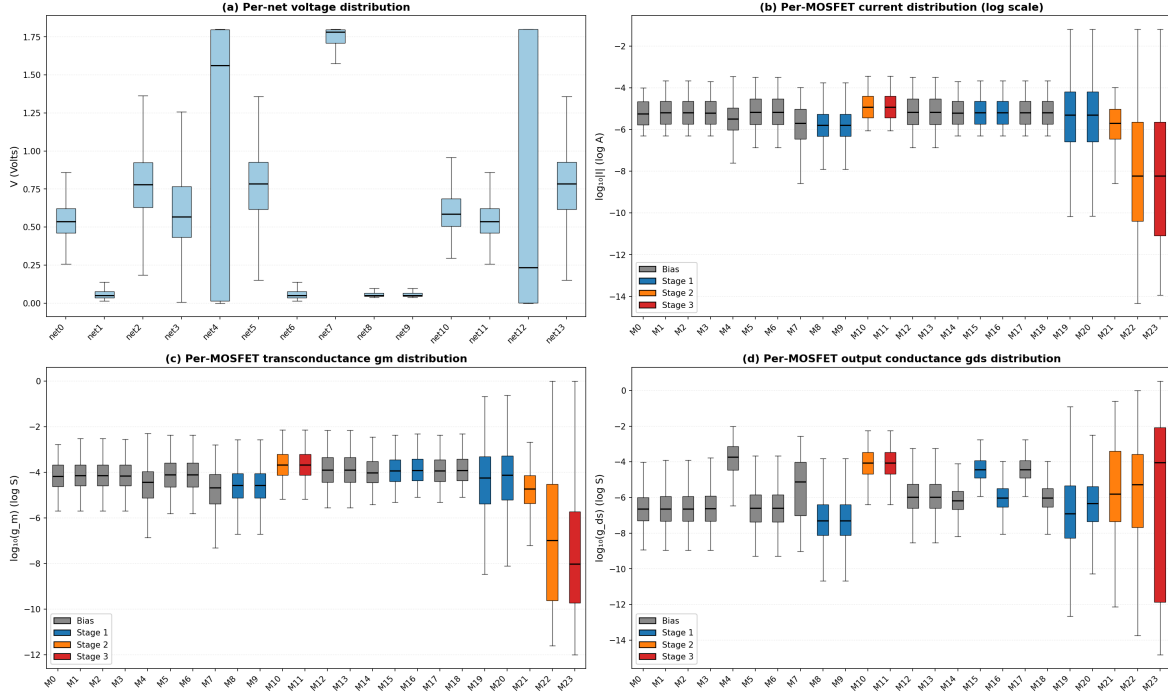


Figure 3.4: Per-device distributions for `fan_smc`, with the transistors grouped by stage. (a) The net voltages, some internal nets are pinned close to a rail while others swing over the full range. (b) The transistor currents, (c) the transconductance g_m and (d) the output conductance g_{ds} . The bias and early-stage devices stay tight, while the output device of each stage (the rightmost device in its group) spreads over many decades.

3.3.3 Dataset Splits

There are two important datasets. The method was first developed and tuned on a single topology, `fan_smc`, with 5,000 samples and the architecture and training choices were all settled on this set. To study generalization across topologies, the dataset was then extended to the five three-stage topologies (Section 3.3.1), each contributing 5,000 samples, for a combined of 25,000 samples. The samples of each topology are split into training and validation sets in an 80/20 ratio, performed independently within each topology before the per-topology sets are combined, so that every topology appears in both. This gives 20,000 training and 5,000 validation samples. No separate test set is held out.

To test whether the model generalizes to an unseen topology, it is trained in a leave-one-topology-out scheme. out of the five topologies, four are used for training and the fifth is held out. This is repeated five times so that each topology is the held-out one once. Each run trains on the combination of the four training splits, 16 000 samples and is then evaluated on the held-out topology, which is a zero-shot test and fine tuning with few examples from the held-out topology.

3.4 Model Architectures

The surrogate consists of a shared backbone that produces one representation per node, followed by a set of prediction heads on top of it.

The backbone combines three forms of communication at every layer. Message passing over

the device–net edges handles the local structure, an attention mechanism over the circuit’s loops mimics the KVL and a global self-attention mechanism gives every node a circuit-wide context. The outputs of all layers are then merged into a single per-node representation. Four heads use this representation for the node voltages, the terminal currents, the per-transistor small-signal parameters and the DC gain prediction.

Figure 3.5 shows the full pipeline. The subsections that follow explain the pipeline more in detail. The message-passing backbone in Section 3.4.1, the loop-attention component in Section 3.4.2, the global-context mechanism in Section 3.4.3 and the prediction heads in Section 3.4.4.

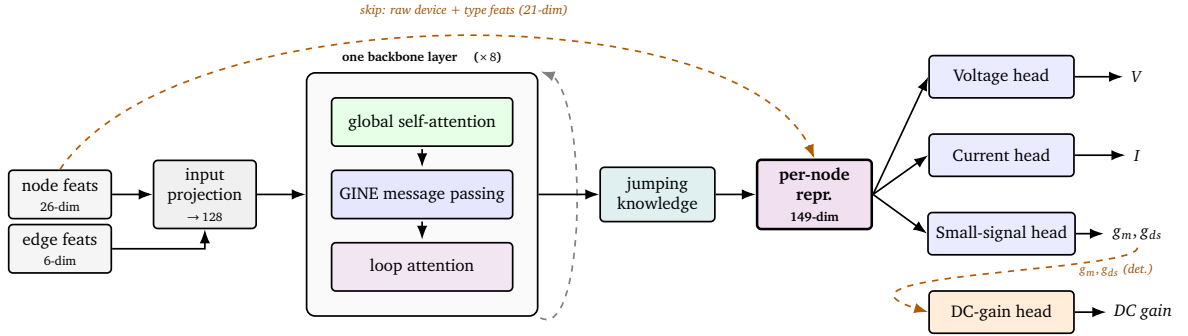


Figure 3.5: Overview of the architecture. The node and edge features are projected to the hidden width and passed through eight backbone layers, each applying global self-attention, GINE message passing and loop attention. A jumping-knowledge aggregation combines the layer outputs. Together with a skip connection that carries the raw device and type features, this forms a per-node representation. The four prediction heads read it off for the voltages, currents, small-signal parameters and DC gain.

3.4.1 Backbone and Message Passing

The backbone builds on the graph isomorphism operator (GIN) introduced in Section 2.3.1. In its original form, a GIN layer updates each node by summing its neighbours’ embeddings, weighting the node’s own embedding by a learnable factor and passing the result through a multi-layer perceptron:

$$h_v^{(k+1)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)}) h_v^{(k)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k)} \right). \quad (3.1)$$

The backbone extends this standard GIN, because each edge carries a feature vector, the edge-aware variant GINE is used, which adds the edge feature to each neighbour’s message before aggregation:

$$h_v^{(k+1)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)}) h_v^{(k)} + \sum_{u \in \mathcal{N}(v)} \text{ReLU}(h_u^{(k)} + W_e e_{vu}) \right), \quad (3.2)$$

where e_{vu} is the six-dimensional edge feature and W_e projects it to the hidden width. Each edge feature records which terminal of a device the edge connects to. Because this feature is added to the neighbor’s embedding before the ReLU, the same neighbor contributes differently through a gate connection than through a drain connection. The per-layer scalar $\epsilon^{(k)}$ is learnable and initialized with zero. The message passing of a GINE layer is illustrated in

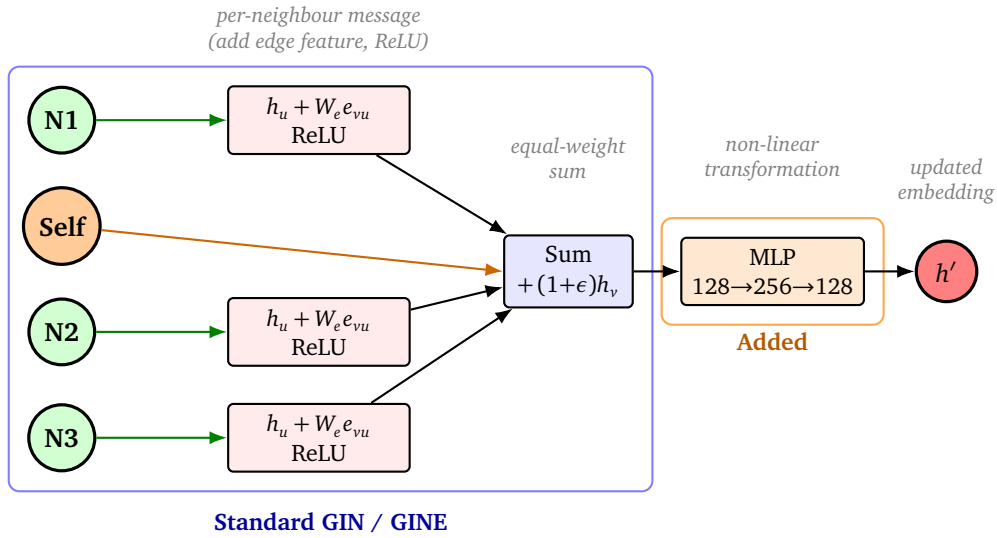


Figure 3.6: Message passing in the GINE layer. Each neighbour’s embedding is combined with the edge feature and passed through a ReLU. The neighbour messages are summed with equal weight and added to the self-embedding, which carries its own $(1 + \epsilon)$ factor and passed through a shared multi-layer perceptron to produce the updated embedding.

Figure 3.6.

The network uses eight such layers at a hidden width of 128. Before the first layer, the 26-dimensional node input is projected to this width by a single linear layer. Each message-passing layer is wrapped in a pre-activation residual block. The input is first layer-normalized and passed through a GELU nonlinearity, then through the GINE aggregation (3.2) and a post-aggregation MLP. The input is then added on top of the result, as shown in figure 3.7. The post-aggregation perceptron has three fully connected layers with a hidden expansion factor of two, batch normalization and GELU activations. Dropout is disabled. This residual, pre-normalized method keeps the network trainable at depth.

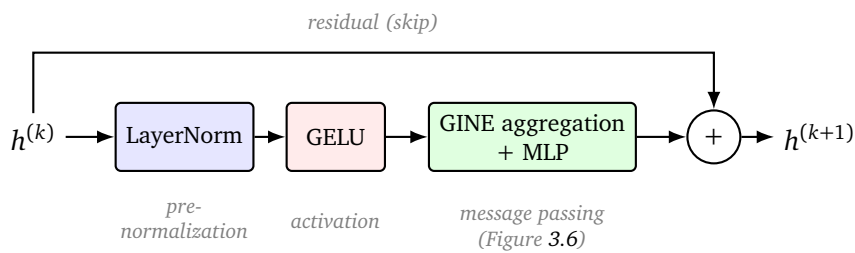


Figure 3.7: The pre-activation residual block wrapping each message-passing layer. The input is layer-normalized, passed through a GELU activation, then through the GINE aggregation and post-aggregation perceptron of Figure 3.6.

The outputs of all eight layers are combined rather than only the last. A jumping-knowledge aggregation concatenates the per-layer embeddings with learned attention weights and projects the result back to the hidden width, giving a 128-dimensional embedding per node, as shown in Figure 3.8. These node embeddings are the input to the prediction heads described in the following subsection 3.4.4.

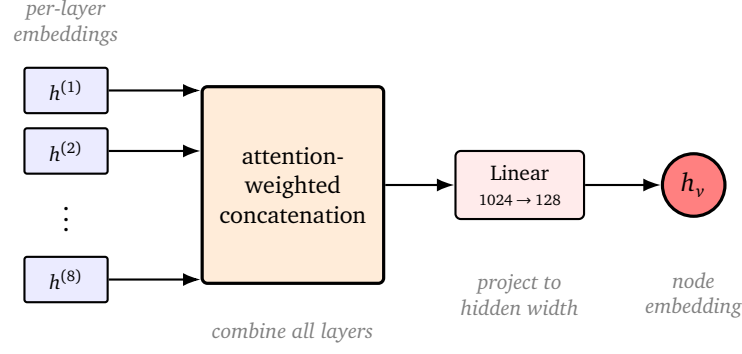


Figure 3.8: Jumping-knowledge aggregation of the layer outputs. The per-node embeddings from all eight message-passing layers are combined by an attention-weighted concatenation and projected back to the hidden width.

The choice of the GINE operator is empirical. On this task it reaches accuracy comparable to an attention-based operator such as GATv2 [23], shown in Figure 3.9, while using fewer parameters and running faster. A single GINE layer at this width has roughly 117k parameters against roughly 165k for a four-head GATv2 layer and the full eight-layer backbone has 1.21M parameters against an estimated 1.65M for the attention variant. The accuracy comparison is reported in Section 4.2.

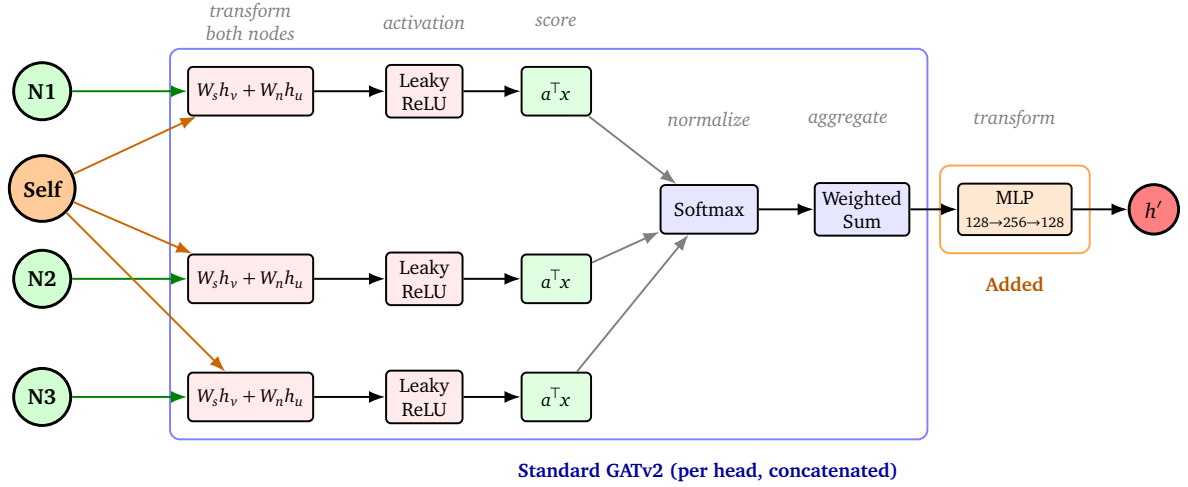


Figure 3.9: Message passing in the GATv2 layer. For every neighbour, the node and neighbour embeddings are linearly transformed, passed through a LeakyReLU and scored against a learned attention vector. The scores are normalized across neighbours and used to weight the aggregation.

3.4.2 Loop-Attention Component

Each loop in a circuit ties its devices together through a physical relation. The voltages around the loop must sum to zero. Nothing in the message passing makes them explicit. Message passing on the device-net graph is local, reaching one device from another only by traversing the nets between them two hops at a time and the devices in a loop can sit many hops apart depending on the topology.

This loop component follows [29], whose circuit surrogate lets the devices in a loop interact in order to represent the loop’s voltage relations. The motivation is physical, each cycle in the circuit corresponds to a Kirchhoff voltage law, so letting those devices attend to one another gives the network an approximate form of this relation. This is where the name loop attention comes from. The construction below follows the same idea, but applied at terminal level.

The loops are read off the circuit structure, as shown in Figure 3.10. The loops are computed with NetworkX, which gives a minimal set of fundamental cycles and for every cycle an edge is added between each pair of devices in it. The resulting structure is similar to the message-passing edges.

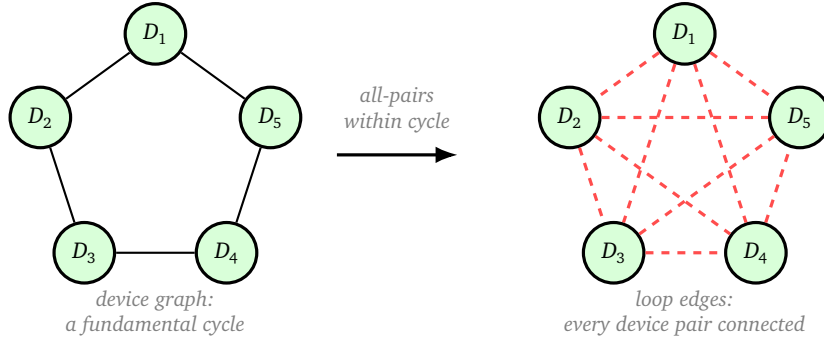


Figure 3.10: Construction of the loop edge. Each fundamental cycle of the device graph (left), where devices are linked if they share a net, is turned into an all-pairs set of loop edges among the devices in the cycle (right). Devices are thereby directly connected in the attention graph, but far apart in the device–net graph.

Over these loop edges the model applies multi-head attention at the level of devices, as illustrated in Figure 3.11. At each layer the terminal embeddings of a device are mean-pooled into a single device embedding. Queries, keys and values are formed from these device embeddings with four attention heads of dimension 32. A device attends only to the devices it shares a loop.

The update is then scattered back so that every terminal of the device receives it. It is added through a learned per-node gate, which lets the model decide where and how much the loop information helps. The gate is initialized with zero and the loop information is then gradually added in, the detailed training schedule is given in Section 3.6. The component is applied at each layers and adds roughly half a million parameters to the model. Its effect on accuracy is examined in Section 4.2.

3.4.3 Global Context

Message passing and loop attention are both restricted forms of communication. The first reaches only immediate graph neighbours, the second only devices in a shared loop. Some of the quantities depend on the whole circuit at once. The DC gain and AC metrics depend on the full signal path and how the stages are wired together. Loop attention does not solve this, since a device in the first stage and a device in the output stage do not share any loop.

One way to add such context is a virtual node, a single extra node connected to all others, which pools information from the whole graph and broadcasts it back, giving every node a global summary in one hop. Compressing all global communication through a single node loses a lot of information. The surrogate instead uses global self-attention, in which every node attends directly to every other node, following the graph transformer approach[30] and

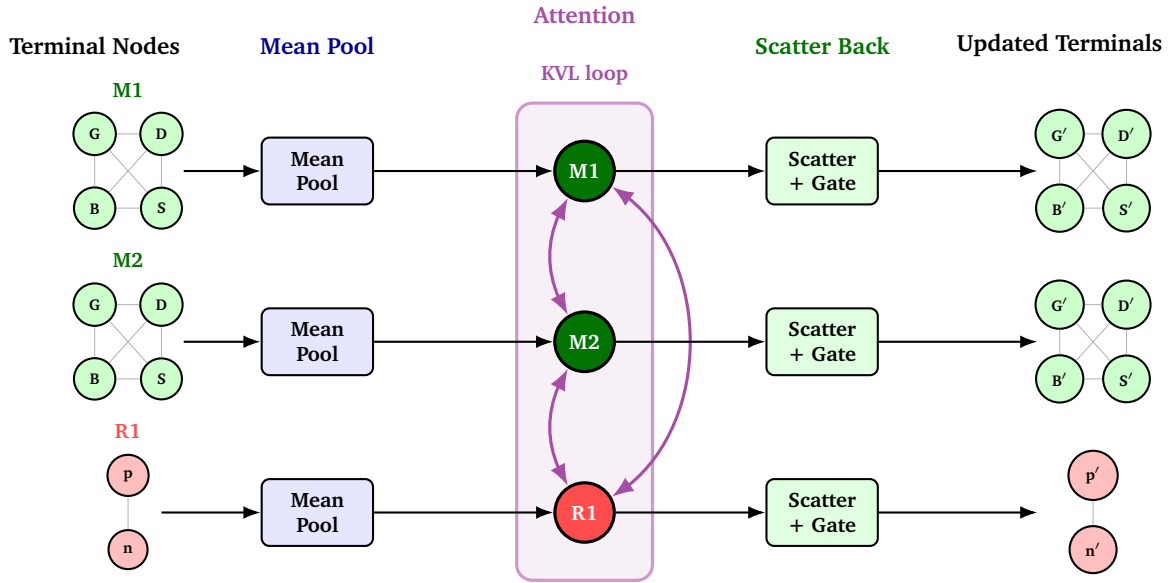


Figure 3.11: The loop-attention mechanism. The terminal embeddings of each device are mean-pooled to a device embedding, devices that share a loop attend to one another through multi-head attention, forming the Kirchhoff voltage law relation around the loop and the resulting update is scattered back to the device's terminals through a learned gate.

shown in Figure 3.12. At each backbone layer the node embeddings of a circuit are updated by multi-head self-attention with four heads of dimension 32. As with loop attention, the result is added to the node embedding through a learned per-layer gate.

The three mechanisms are applied in sequence at each layer. First global self-attention, giving each node circuit-wide context, then the GINE message passing over the device-net edges, then the loop attention over the loop edges. Each is a gated additive update, so the model can decide on the combination of local, loop and global communication. The global-attention mechanism reuses a single set of projections across all eight layers and adds only a small number of parameters. Its individual contribution to accuracy is examined in Section 4.2.

3.4.4 Prediction Heads

The backbone produces a per-node representation that the prediction heads read from. This representation is the jumping-knowledge embedding concatenated with a skip connection that carries the node's raw device and type features, which gives a 149-dimensional vector per node. Four heads use it, one per predicted quantity. Each head's structure is built to fit what the quantity needs (Figure 3.13).

The voltage head is the simplest. It is a single linear layer that maps the 149-dimensional node representation to one value, the operating-point voltage. There is no hidden layer, normalization or activation.

The current head works per device rather than per node. Instead of mapping each node's representation to a current, it concatenates the terminal representations of a device into one device-level vector and predicts a single current per device, which is then scattered back to its terminals. The reason for doing that is explained in Section 4.2.4. It has two separate MLPs, one for transistors and one for two-terminal devices, with a hidden width of 128. Keeping

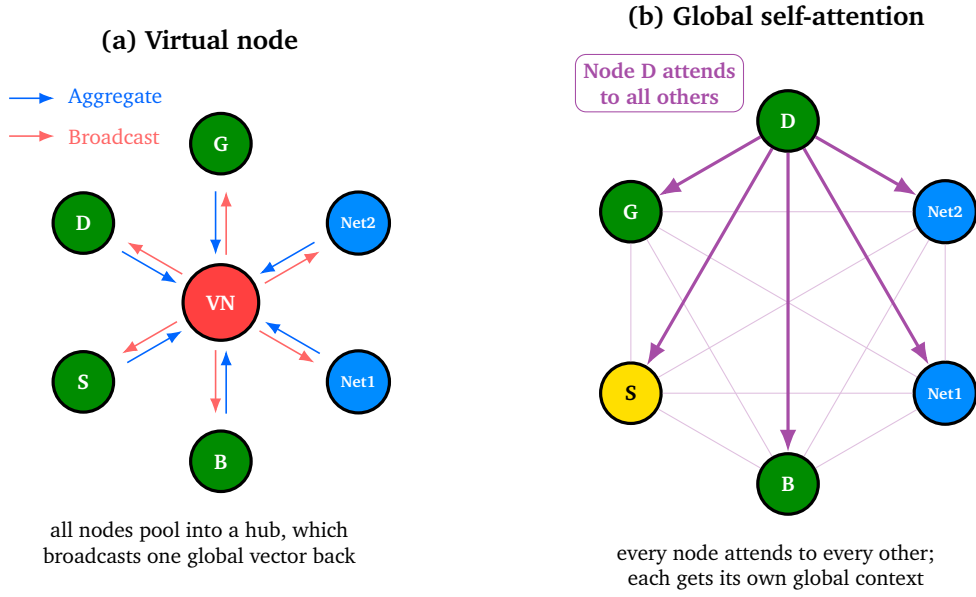


Figure 3.12: Two ways of providing global context. The virtual node (a) aggregates all nodes through a single node and broadcasts the vector back, a bottleneck that gives every node the same global summary and lose of information. Global self-attention (b) instead connects every pair of nodes directly, so each node receives a different context.

the branches separate lets the head treat devices differently instead of forcing one mapping to handle both.

The small-signal head predicts the transconductance and output conductance of each transistor. For each transistor it concatenates the representations of its gate, drain, source and bulk terminals together with three terminal voltages and the drain current as operating-point context, which gives a 600-dimensional input. This goes through an MLP and produces the two small-signal parameters.

The DC-gain head is the only physically structured one. It first builds an analytical estimate of the gain from the predicted transconductances and output conductances, packs these together with the estimate into a small feature vector and then refines it through an MLP. The analytical relationship and the full feature set are given in Appendix A.4. The gradient from the gain prediction does not flow back into the small-signal predictions. The small-signal head is trained only on its own targets and the gain head only learns how to combine its outputs. This keeps the gain objective from disturbing the device-level predictions.

All four heads predict in the normalized space in which the targets are stored and the outputs are de-normalized only for evaluation. The normalization scheme is described in Section 3.5.

3.5 Training Objective

The model predicts several different quantities at once, a voltage at each net, a current at each terminal, the small-signal parameters of each transistor and the DC gain of the whole circuit. For each quantity there is a loss that compares the prediction against the value from the SPICE simulation. On top of these, one physical law is added as a soft constraint, so that

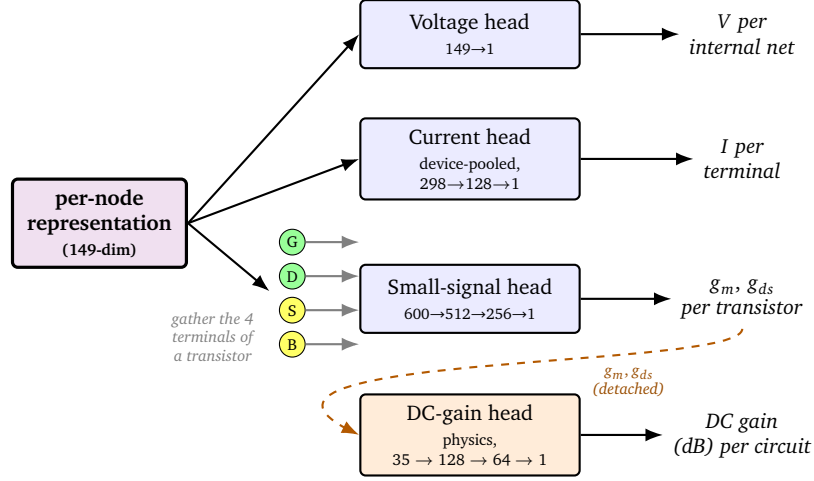


Figure 3.13: The four prediction heads. The voltage head is a single linear projection of the node representation. The current head pools each device’s terminal representations. The small-signal head gathers the four terminal representations of a transistor and predicts its transconductance and output conductance. The DC-gain head takes these small-signal predictions as fixed inputs, forms an analytical gain estimate and refines it.

the predictions are not only close to the targets one but also consistent with each other.

The rest of this section is in two parts. Section 3.5.1 describes the supervised losses and the used normalization. Section 3.5.2 describes how physical knowledge is added, the current conservation constraint and the physics that is built into the architecture itself.

3.5.1 Supervised Losses and Normalization

The model is trained by minimizing supervised loss per predicted quantity, together with one physics constraint:

$$\mathcal{L}_{\text{total}}(\theta; t) = \mathcal{L}_V + \mathcal{L}_I + \mathcal{L}_{g_m} + \mathcal{L}_{g_{ds}} + \mathcal{L}_{DC} + w_{\text{KCL}}(t) \mathcal{L}_{\text{KCL}}, \quad (3.3)$$

where θ are the model parameters and t is the training epoch. All five supervised terms have equal weight and are active from the start of training. The one term whose weight changes over time is the Kirchhoff current law loss $\mathcal{L}_{\text{KCL}}$, which is phased in as training proceeds, its weight schedule $w_{\text{KCL}}(t)$ and its formulation are given in Section 3.5.2.

The predicted quantities cover very different ranges, so each target is normalized before its loss is computed and the model is trained to predict in this normalized space. Voltages are standardized as they are. Currents and small-signal parameters span several orders of magnitude, so they are first put on a logarithmic scale and then standardized. For the currents the magnitude is used rather than the signed value. A current can be negative and its sign is not needed for the supervised loss but is handled separately by the physics term in Section 3.5.2. A small floor of 10^{-12} A (one picoampere) is added inside the logarithm so that terminals with negligible current do not produce a diverging value. This floor lies far below any current the model is trained to predict and so leaves the real values unchanged. The DC gain is already a logarithmic quantity and is standardized in decibels. Writing μ and σ for the mean and

standard deviation of each quantity. The normalized targets are

$$\begin{aligned} z_n^V &= (V_n - \mu_V)/\sigma_V, & z_t^I &= (\log_{10}(|I_t| + 10^{-12}) - \mu_I)/\sigma_I, \\ z_m^{gm} &= (\log_{10} g_{m,m} - \mu_{gm})/\sigma_{gm}, & z_m^{gds} &= (\log_{10} g_{ds,m} - \mu_{gds})/\sigma_{gds}, \\ z_g^{DC} &= (G_g^{dB} - \mu_{DC})/\sigma_{DC}, \end{aligned} \quad (3.4)$$

where V_n is the voltage at node n , I_t the current at terminal t , $g_{m,m}$ and $g_{ds,m}$ the transconductance and output conductance of transistor m , and G_g^{dB} the DC gain of circuit g in decibels.

Each loss is a mean squared error between the model's prediction and the target. A hat denotes the model's prediction. The predictions are mapped back to physical units only when metrics are reported. The four losses differ only in which positions they are summed over.

The voltage loss runs over the internal nets. The supply rails and the input nodes have fixed, known voltages and are never supervised, so with $\mathcal{N}_{\text{int}}$ the set of internal-net nodes,

$$\mathcal{L}_V = \frac{1}{|\mathcal{N}_{\text{int}}|} \sum_{n \in \mathcal{N}_{\text{int}}} (\hat{z}_n^V - z_n^V)^2. \quad (3.5)$$

The current loss runs over the terminals that carry a DC current. The drain and source of every transistor and the two terminals of every resistor. The transistor gates and bulks and capacitor, which carry no DC current are left out. With $\mathcal{T}_I$ the set of current-carrying terminals,

$$\mathcal{L}_I = \frac{1}{|\mathcal{T}_I|} \sum_{t \in \mathcal{T}_I} (\hat{z}_t^I - z_t^I)^2. \quad (3.6)$$

The small-signal losses run over the transistors, one value of g_m and one of g_{ds} per transistor. Passive devices have no such parameters. With M the number of transistors in the circuit,

$$\mathcal{L}_{gm} = \frac{1}{M} \sum_{m=1}^M (\hat{z}_m^{gm} - z_m^{gm})^2, \quad \mathcal{L}_{gds} = \frac{1}{M} \sum_{m=1}^M (\hat{z}_m^{gds} - z_m^{gds})^2. \quad (3.7)$$

The DC-gain loss runs once per circuit and only over circuits whose small-signal simulation converged to a positive gain. Samples that failed to converge or that returned a non-positive gain, are skipped. With $\mathcal{G}_{\text{valid}}$ the set of these circuits,

$$\mathcal{L}_{DC} = \frac{1}{|\mathcal{G}_{\text{valid}}|} \sum_{g \in \mathcal{G}_{\text{valid}}} (\hat{z}_g^{DC} - z_g^{DC})^2. \quad (3.8)$$

3.5.2 Physics-Informed Terms

Physical knowledge enters the model in two ways. One is as a soft constraint, an extra loss term that pushes the predictions toward a physical law. The other is as structure, a relation built into the network so that the prediction respects it by design. Which one is chosen depends on how well it actually holds and on what the architecture allows. A law that holds exactly can be enforced as a structure. One that only holds roughly is better built in as constraint, where the network can correct for its error.

The one conservation law that holds exactly here, whatever the transistor model, is Kirchhoff's current law, which says that the currents flowing into any net sum to zero. So it is true

to simulator precision. The simplified device-level relations such as the square-law transistor equation or ideal current-mirror ratios, do not hold this way. They differ from the BSIM4 reference enough that would fight the supervised targets. Current conservation is the only law enforced through the loss.

The KCL loss is applied per net. For an internal net n , let $\text{terms}(n)$ be the device terminals connected to it and give each terminal a sign for the direction of its current, +1 when current flows into the net and -1 when it flows out. The law to satisfy is

$$\sum_{t \in \text{terms}(n)} \text{sign}(t) |I_t| = 0, \quad (3.9)$$

where $|I_t|$ is the predicted current magnitude at terminal t . The sum runs over the terminals that carry a DC current (Figure 3.14). Capacitor terminals are kept in the sum but as zero entries, since a capacitor passes no DC current, including them forces the remaining terminals at the net to balance on their own. How each sign is set depends on the device. For a transistor it is fixed by the device type, because the drain and source currents keep their direction in normal operation. For every other device it follows the actual current direction, so it can flip, which is what lets the constraint handle a feedback resistor whose current reverses when the output stage changes state.

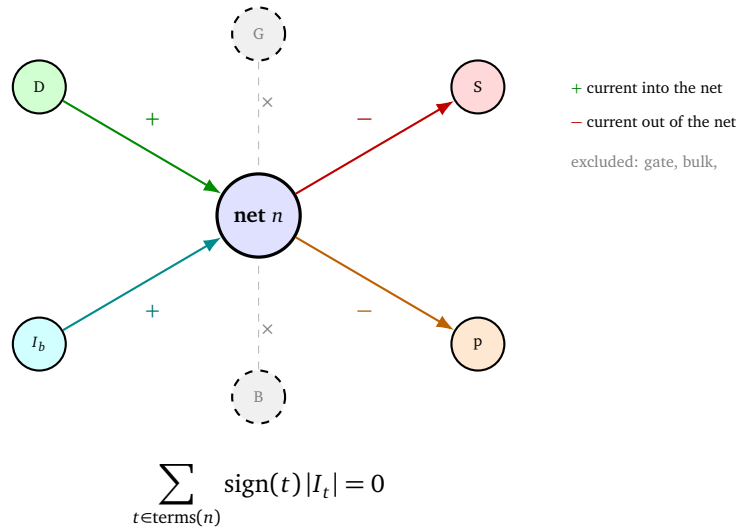


Figure 3.14: How current conservation is applied at an internal net. Transistor drains and sources, current-source terminals, and passive terminals contribute to the balance, each entering the sum with a sign that marks whether its current flows into or out of the net. The gate and bulk terminals carry no DC current are left out.

It is not applied at the supply rails or the input nodes. These sit at fixed, known voltages and the sources driving them deliver whatever current the circuit needs. A net is only worth constraining where a balance is actually possible, if all of its terminals carry current in the same direction so that inflow and outflow cannot balance or if the total current through it is below a nanoampere, where the simulator's own numerical noise dominates.

The model predicts currents as log-magnitudes, so the residual of (3.9) is formed in log space rather than by subtracting raw currents. For each net the inflowing and outflowing magnitudes are summed separately, each sum taken with a log-sum-exp so that the largest current does not dominate the smaller ones and the loss penalizes the squared gap between

the two log-sums:

$$\mathcal{L}_{\text{KCL}} = \frac{1}{|\mathcal{N}_{\text{KCL}}|} \sum_{n \in \mathcal{N}_{\text{KCL}}} (S_n^+ - S_n^-)^2, \quad (3.10)$$

where S_n^+ and S_n^- are the log-summed inflowing and outflowing magnitudes at net n , and $\mathcal{N}_{\text{KCL}}$ is the set of nets the constraint covers.

The constraint is gradually added. Early on the current predictions are still rough and penalizing before the individual currents are reasonable only adds noise to the gradient. The KCL weight is therefore held at zero for the first part of training and then ramped up linearly,

$$w_{\text{KCL}}(t) = \begin{cases} 0 & t < t_0, \\ (t - t_0)/T_r & t_0 \leq t < t_0 + T_r, \\ 1 & t \geq t_0 + T_r, \end{cases} \quad (3.11)$$

switched on at epoch t_0 and reaching full weight T_r epochs later. How much this constraint helps and how that changes with the amount of training data, is examined in Section 4.3.

Kirchhoff's voltage law is handled differently and never appears as a loss. Its loop relations are already the reason for the loop-attention component of Section 3.4.2, which is built to mimic them. On top of that, KVL is satisfied automatically by the voltage representation, predicting one voltage per net and letting every terminal inherit it makes the voltages around any loop sum to zero by construction. This is directly integrated into network.

KVL is not the only physics the architecture carries. The current head predicts one current per device and scatters it to that device's terminals, instead of predicting each terminal on its own (Figure 3.15). This uses the fact that the current through a two-terminal device is the same at both ends and ties a transistor's terminal currents together, so the predictions cannot break this per device consistency.

The DC-gain head goes further and uses analytical estimate of the gain. An expression assembled from the predicted transconductances and output conductances along the amplifier's signal path, which a small network then corrects. It is used because, unlike the device-level equations, this expression is accurate enough on its own to be a good starting point, its accuracy is shown in Section 4.2 and the full expression is given in Appendix A.4.

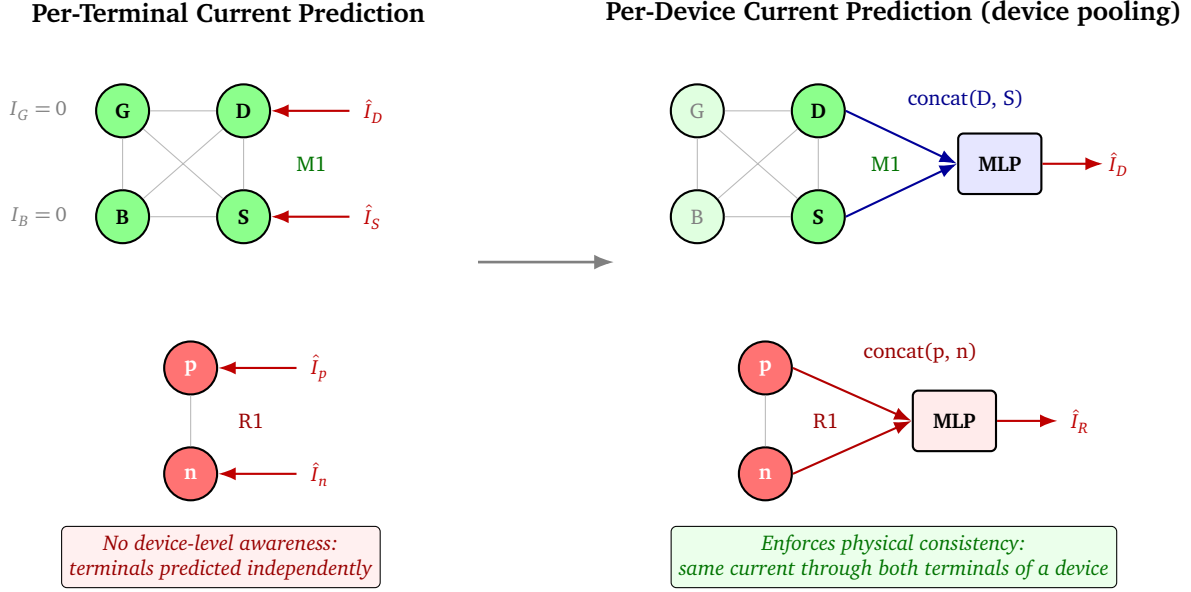


Figure 3.15: Per-terminal versus per-device current prediction. On the left, each terminal's current is predicted on its own. On the right, the device's terminal representations are pooled (the gate and bulk, which carry no DC current, are dropped) and a single current is predicted per device, so the same current is assigned to both terminals by construction.

3.6 Experimental Protocol

This section describes how the model is trained and evaluated, the optimization settings and the set of experiments used to test each part of the design. The aim is to make the runs behind the results reproducible.

Most of the experiments share the same setup, so a default is stated up front in Section 3.6.2. The development and most of the ablations were carried out on the `fan_smc` topology, which serves as the reference circuit. The experiments that differ from this default are the ones that test generalization across topologies.

The rest of this section is in two parts. Section 3.6.1 gives the training setup, the optimizer, the schedules and the warm-ups. Section 3.6.2 lays out the individual experiments, which parts of the model are switched on or off, how the loss terms change between them and the metrics everything is measured by.

3.6.1 Training Setup

The model is trained with Adam and no weight decay. The development and most of the ablations were carried out on the `fan_smc` topology, which serves as the reference and the setup described here is exactly that one. The learning rate is 3×10^{-3} , reached by a linear warm-up over the first 50 epochs and then reduced on a plateau. When the validation loss stops improving for 150 epochs the rate is halved, down to a floor of 10^{-6} . The same validation loss is used to pick the best checkpoint and to stop training early if it has not improved for 500 epochs. Gradients are clipped to a maximum norm of 0.3.

Each step processes a batch of 1024 circuit graphs and training runs for up to 5000 epochs. For the multi-topology runs the batches hold an equal share of every active topology and the validation set always contains all five topologies, even in the held-out runs where one topology is dropped from training. The full hyperparameters and the small differences between the training regimes are listed in Appendix A.2.¹

Three parts of the model are gradually added during training, as they should only begin to act once the predictions they build on are reasonable. The current conservation loss is held at zero until epoch 200 and then ramped to full weight over the next 100 epochs. The loop-attention component is kept inactive for the first 350 epochs and then ramped in over a further 400. The global-attention gates are not tied to an epoch schedule, instead each is initialized to zero, so the mechanism starts out doing nothing and opens up as training progresses.

The multi-topology runs follow the same recipe with a few changes. The learning rate is lowered to 10^{-3} , the plateau and early-stopping patience are longer and the two ramps run for longer, since these runs see more data and take longer to settle. The plateau, checkpoint and early stopping are driven by the average per-topology voltage error rather than the total validation loss, so that one topology or the current-conservation term coming in during training does not dominate the loss. The fine-tuning runs start from a pretrained model instead of a random initialization. The current-conservation term is active from the first fine-tuning epoch. The DC-gain head is only used in the single-topology runs, where the analytical gain formula is available and is switched off for all multi-topology runs. The exact settings for each regime are in Appendix A.2.

3.6.2 Experimental Conditions

Unless an experiment states otherwise, the reference configuration is the full model of Sections 3.4 and 3.5, a GINE backbone of eight layers at hidden width 128, with loop attention, global attention, the current conservation loss and the physics structured DC-gain head all active. Each experiment changes one part of this and keeps the rest fixed.

The supervised losses for the voltages, currents and small-signal parameters are present in every experiment. What changes between experiments is whether the current-conservation loss and physics built into the architecture are active or not. The DC gain loss is left out whenever is the multi-topology training. Table 3.12 summarizes which terms are active in each experiment.

¹All runs were carried out on a single NVIDIA H100 GPU, using Python 3.10 with PyTorch and PyTorch Geometric, and ngspice through PySpice for the ground-truth simulations.

Experiment	$\mathcal{L}_V$	$\mathcal{L}_I$	$\mathcal{L}_{gm}$	$\mathcal{L}_{gds}$	$\mathcal{L}_{KCL}$	$\mathcal{L}_{DC}$
Single-topology, with DC head	✓	✓	✓	✓	✓	✓
Joint pretraining	✓	✓	✓	✓	✓	
Held-out pretraining (physics)	✓	✓	✓	✓	✓	
Held-out pretraining (no physics)	✓	✓	✓	✓		
Fine-tuning (physics)	✓	✓	✓	✓	✓	
Fine-tuning (no physics)	✓	✓	✓	✓		

Table 3.12: Which loss terms are active in each experiment. The four supervised device losses are always present. The current conservation and DC gain terms are switched per experiment.

All metrics are computed on the validation set and broken down per topology. The voltage error is reported as a mean absolute error in millivolts and the current error as a mean absolute error in microamperes. Both are computed after mapping the predictions back to physical units. The small-signal parameters are reported as a mean absolute error in the $\log_{10}$ space and the DC gain as a mean absolute error in decibels over the circuits, which produce a valid positive gain.

Chapter 4

Results

This chapter tests the model first on the `fan_smc` amplifier and then across the other topologies. It has four parts. Section 4.1 shows how accurately the model predicts the operating point, the small-signal parameters and the DC gain, split by quantity and by operating region. Section 4.2.1 ablates the architecture, the message passing, the attention parts, the backbone depth and the prediction heads, to see which parts do the work. Section 4.3 is about the physics constraint, which one to use and why only Kirchhoff's current law helps, how much it helps when data is scarce and whether it makes the predictions more physically consistent. Section 4.4 is about generalization, how well one model handles several topologies at once and how well it adapts to a topology it has never seen.

Everything is measured against NGSPICE with the BSIM4 models as ground truth, on the unfiltered data described in Chapter 3. So the errors include the badly biased and edge-case circuits as well.

4.1 Prediction Accuracy

It is worth establishing that the model predicts the operating point and small-signal behavior accurately in the first place. This section reports the accuracy of the reference model on the `fan_smc` topology at full data, the configuration described in Section 3.6, evaluated on the validation set.

Table 4.1 collects the errors for each predicted quantity. The voltage is reported as a mean absolute error in millivolts, the current in microamperes and in the $\log_{10}$ space it is predicted in, the small-signal parameters in that same $\log_{10}$ space and the DC gain in decibels. The relative entry beside them put each error in context. For the current and small-signal parameters as a relative error per device and for the voltage as a fraction of the supply range. The ranges these quantities cover are the ones described in Section 3.3.2.

The node voltages are predicted tightly. A mean error of 6.42 mV is about a third of a percent of the 1.8V supply range and around one percent of the spread of the voltages themselves.

The current has two numbers. The $3.7\mu\text{A}$ is in physical units, but it is dominated by the few high-current devices. The currents span roughly twelve orders of magnitude, so a microampere-scale average says little about the small ones. The relative error is the more representative measure and it comes out at about 6% of each current's own value. This is the reason the currents are predicted in log space in the first place, as set out in Section 3.5.1.

Quantity	Error	Relative
Node voltage	6.42 mV	0.36% of supply range
Terminal current	$3.7 \mu\text{A} / 0.0268 (\log_{10})$	$\approx 6\%$ per device
g_m	$0.020 (\log_{10})$	$\approx 5\%$ per device
g_{ds}	$0.034 (\log_{10})$	$\approx 8\%$ per device
DC gain	4.36 dB	over all circuits

Table 4.1: Prediction accuracy of the reference model on `fan_smc` on the validation set. The current is given both in physical units and in $\log_{10}$ space. The relative column reports the per-device error for the current and small-signal parameters and a fraction of the 1.8 V supply range for the voltage.

The transconductances and output conductances, predicted the same way, reach a similar per-device accuracy.

The DC gain is the hardest of the quantities. Unlike the others, it is a circuit-level number that depends on the whole signal path at once. The sampled sizings cover everything from working amplifiers to circuits that do not amplify at all. The gain is taken straight from the simulator for every circuit, with no filtering, and goes from strongly negative values up to around 100 dB. The 4.36 dB mean error is over this full range. That the gain is less accurate than the device-level quantities is expected. It is the behavior of the entire circuit rather than a single node or device and there is less training signal for it, one value per circuit instead of one per device.

Figure 4.1 shows the predictions against the simulated targets directly, one panel per quantity. The points lie close to the diagonal across every quantity, which means the model is accurate not just on average but across the full spread of values.

The error is not spread evenly across the circuit. Figure 4.2 breaks the prediction error down by device and it lines up with the per-device spread seen in the data (Section 3.3.2). The bias and early-stage devices, whose quantities barely move, are predicted almost exactly, while the error concentrates on the output devices of the later stages, which are the ones that swing over the widest range. The same pattern shows up in the current, the transconductance and the output conductance. The hardest devices to predict are exactly the ones whose values vary the most.

The current accuracy also holds up across operating regions. Figure 4.3 splits the current plot by region. By the median relative error the model is tight everywhere and saturation has the lowest $\log_{10}$ error as expected. The mean relative error is reasonable in saturation and triode (3.8% and 7.3%) but jumps to 32.8% in cutoff. A small miss on a near-off current turns into a large relative error and a handful of these drag the mean up. By the median, the off devices are predicted as tightly as the rest. So the model handles all three regions well and the large cutoff mean is a result of dividing by near-zero currents.

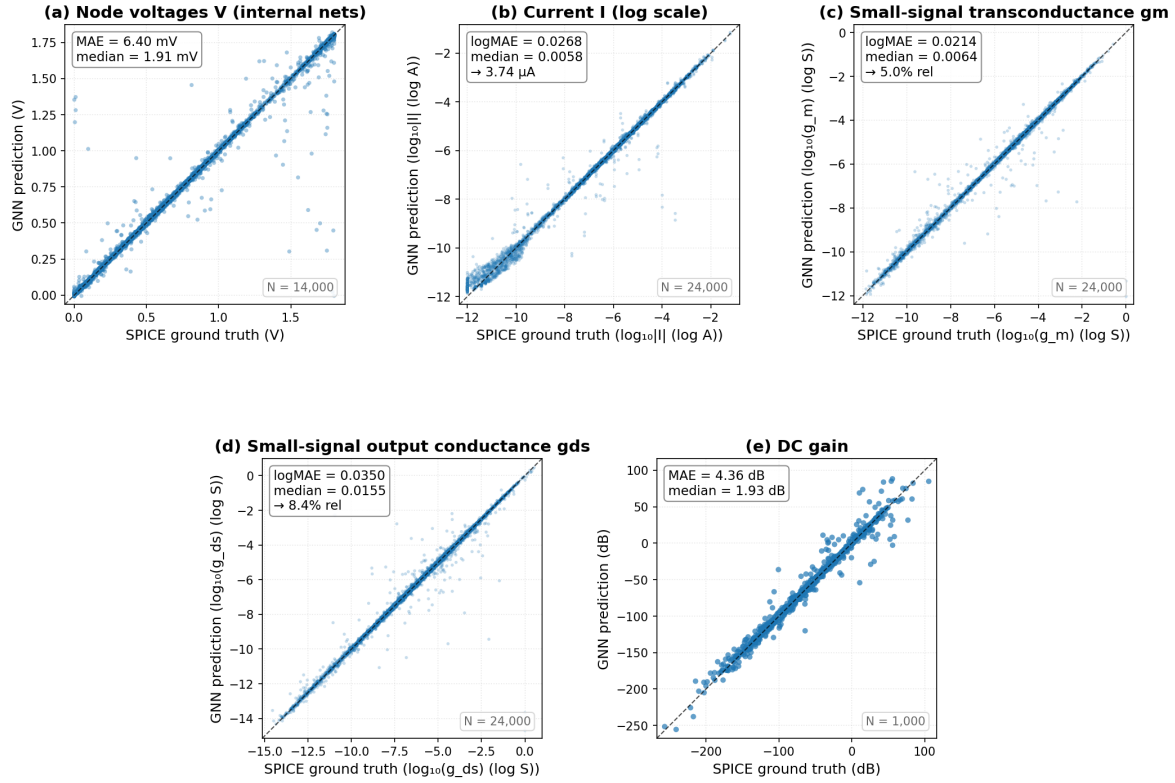


Figure 4.1: Predicted versus simulated values for the reference model on `fan_smc` on the validation set. The diagonal line marks ground truth. The voltages are shown in volts and the current and small-signal parameters in $\log_{10}$. The DC gain is in decibels. The predictions lie close to the diagonal across the full range of every quantity.

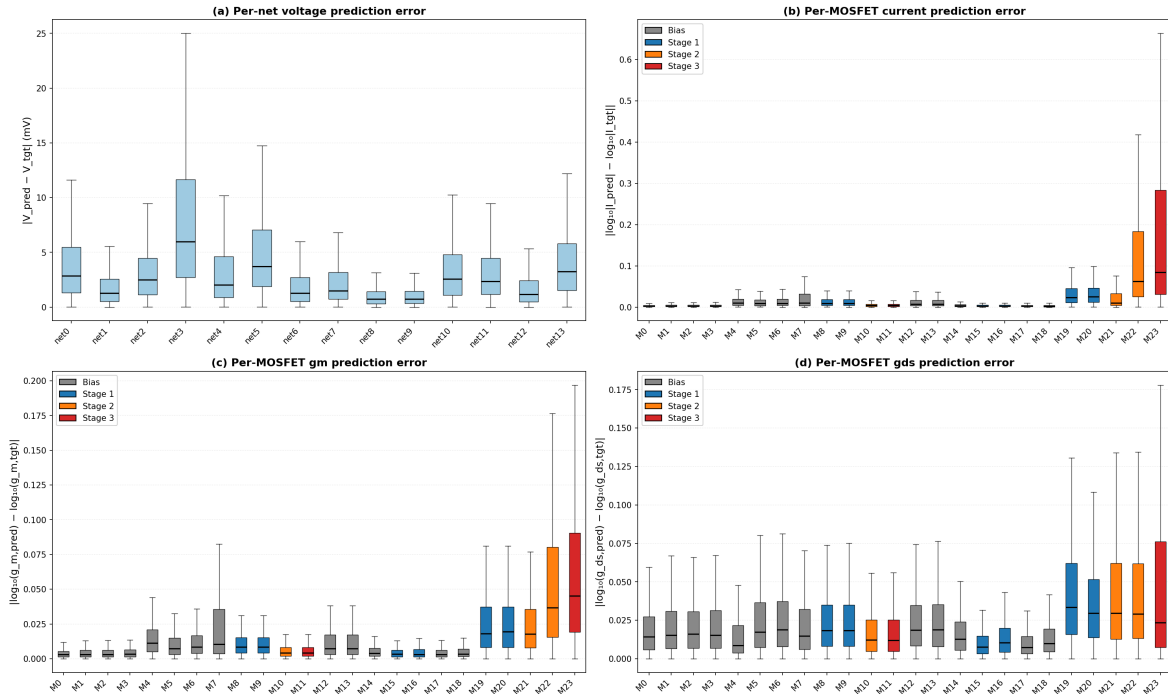


Figure 4.2: Per-device prediction error on `fan_smc`, with the transistors colored by stage. (a) net voltage error in millivolts, (b) current, (c) transconductance and (d) output conductance error in $\log_{10}$. The bias and early-stage devices are predicted almost exactly, while the error concentrates on the output devices of the later stages. The same pattern can be observed in the data spread (Figure 3.4).

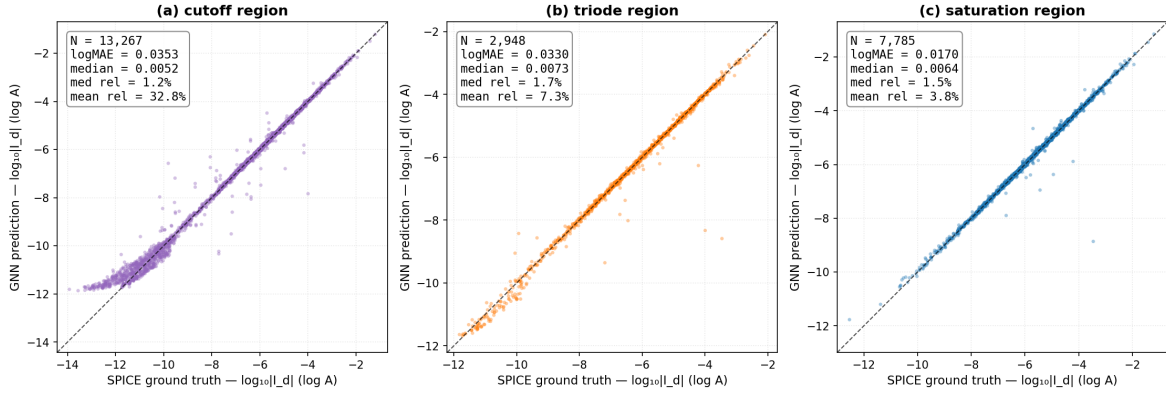


Figure 4.3: Predicted versus ground truth drain current on `fan_smc`, split by the device’s operating region. (a) cutoff, (b) triode, (c) saturation. Currents are in $\log_{10}$. By the median relative error the model is tight in every region. The cutoff mean is inflated because relative error blows up for the near-zero currents of off devices.

4.2 Architecture Ablations

The model has a few design choices that were settled by testing. This section goes through them one at a time, always changing one thing and keeping the rest at the reference setting, so each difference can be traced back.

There are four parts. The first is the message-passing operator. The second is the two attention mechanisms. The third is the backbone itself, how deep it should be and whether to split it into towers. The fourth is the prediction heads, which is also where the built-in hard physics constraints sit.

4.2.1 Message Passing

The backbone uses the GINE operator, which passes messages along the edges and adds in the edge features (Section 3.4.1). To check this is the right pick, it is compared against three alternatives. Plain GIN without edge features, the attention-based GATv2 and the generalized-aggregation GENConv [25]. All four are trained with the reference setup on `fan_smc`.

Table 4.2 gives the mean errors. GINE comes out best or tied on every quantity. GATv2 and GENConv, their voltage error is 20 to 30% higher and their gain error around 15% higher. GINE came out on the top without being the biggest model. GENConv has fewer parameters and loses on everything, so the advantage is not simply a matter of throwing capacity at the problem.

The clearest comparison is GINE against plain GIN, because the only thing separating them is whether the edge features are used. Dropping the edge features barely touches the voltage or the small-signal parameters, but the current error jumps by a third, from 3.74 to $4.97 \mu\text{A}$ and the gain error rises by about 15%. Edge features encode which terminal of a device an edge connects to and that is needed to tell a drain current from a source current.

It also helps to look at the share of predictions that land within a tight tolerance. Table 4.3 reports, how often the voltage is within 1% of the supply range, the current within 1% of its

Operator	V (mV)	I (μ A)	I (log)	g_m (log)	g_{ds} (log)	Gain (dB)	Params
GINE	6.42	3.74	0.027	0.020	0.034	4.36	2.93M
GIN	6.61	4.97	0.028	0.021	0.034	5.01	2.79M
GATv2	7.73	4.52	0.029	0.023	0.040	4.95	3.06M
GENConv	8.34	6.12	0.031	0.024	0.041	5.14	2.79M

Table 4.2: Message-passing operator comparison on `fan_smc`. GINE is best or tied on every quantity. “GIN” is the plain operator without edge features. The others use the same edge features as GINE.

own value, g_m and g_{ds} within 10% and the DC gain within 3 dB.

Operator	V @1%	I @1%	g_m @10%	g_{ds} @10%	Gain @3 dB
GINE	60.6	43.5	90.7	82.1	64.9
GIN	60.6	41.5	90.5	82.8	59.3
GATv2	53.1	35.4	89.2	77.7	58.6
GENConv	53.6	37.2	88.7	76.6	57.2

Table 4.3: Share of predictions (%) within a tight tolerance on `fan_smc`. Voltage within 1% of the supply range, current within 1% of its own value, g_m and g_{ds} within 10% and DC gain within 3 dB.

At these tight thresholds the same operator ranking holds. GINE is ahead on every quantity and the gap to both other operators is bigger. GINE puts about 61% of voltages within 1% of the supply range, while GATv2 and GENConv reach only about 53% and the difference shows up in the currents and the gain. Plain GIN is similar to GINE on the voltage and the small-signal parameters, but falls behind on the current and, more clearly, on the gain. The reason is again the edge features, helping most where terminal identity matters. The full set of thresholds for all four operators is given in Appendix A.3.

4.2.2 Attention Components

On top of the message passing there are two attention mechanisms. Loop attention, which attends over the device cycles of the circuit (Section 3.4.2) and a global virtual node, which mixes information across the whole graph (Section 3.4.3). Each is turned off one at a time from the full model, with everything else left at the reference setting. The virtual node also comes in two settings, a plain global node and the multi-head-attention version, which is used in the reference.

Table 4.4 gives the mean errors. Taking out loop attention is by far the worst setting. The voltage error more than doubles, from 6.42 to 15.51 mV, the current error more than triples and everything else gets worse by a similar amount. Loop attention is the most important part of the backbone. The virtual node matters too, without it the voltage error rises to 8.30 mV, but nothing like the collapse from dropping loop attention.

Switching from MHA to no virtual node costs about 1.9 mV in voltage error. The plain virtual node sits right on top of having no virtual node at all, 8.18 against 8.30 mV. So a plain global node adds basically nothing. Turning off both attention mechanisms at once lands in the same place as just dropping the virtual node, 8.51 against 8.30 mV. That says almost all of

Variant	V (mV)	I (μ A)	I (log)	g_m (log)	g_{ds} (log)	Gain (dB)	Params
Reference	6.42	3.74	0.027	0.020	0.034	4.36	2.93M
Loop off	15.51	12.30	0.054	0.045	0.065	7.26	2.40M
VN off	8.30	5.60	0.033	0.026	0.041	5.50	2.86M
VN plain	8.18	6.70	0.032	0.026	0.040	5.44	2.90M
Both off	8.51	5.80	0.033	0.026	0.040	5.65	2.33M

Table 4.4: Attention-component ablation on `fan_smc`. “Reference” is the MHA virtual node with loop attention on. “Both off” is the bare GINE backbone. Loop attention is by far the most important. The virtual node helps in its MHA form, but in the plain form it does not.

the attention benefit comes from loop attention.

One thing to look closer. The MHA virtual node only helps when loop attention is there to go with it. With loop attention off, turning the virtual node off as well nearly halves the error, from 15.51 to 8.51 mV, so the bare backbone does better than the virtual node on its own. A global attention with no loop attention harms the result.

The within-threshold numbers are shown in Table 4.5. Without loop-attention, the share of voltages within 1% of the supply range roughly halves, from 61% to 32%, and the share of currents within 1% of their own value falls from 44% to 16%. This shows that it is mimicking the KVL constraint, explained in Section 3.4.2. The virtual-node variants all sit close together.

Variant	V @1%	I @1%	g_m @10%	g_{ds} @10%	Gain @3 dB
Reference	60.6	43.5	90.7	82.1	64.9
Loop off	31.5	16.0	75.3	58.2	43.9
VN off	58.1	38.8	89.3	79.8	59.4
VN plain	58.2	41.0	89.1	79.2	56.9
Both off	60.0	40.4	88.4	80.3	57.4

Table 4.5: Share of predictions (%) within a tight tolerance for the attention ablation on `fan_smc`. Voltage within 1% of the supply range, current within 1% of its own value, g_m and g_{ds} within 10%, DC gain within 3 dB. Removing loop attention roughly halves the share of accurate voltages and currents.

It also helps to weigh each component against what it costs. Loop attention is the more expensive of the two, about 527k parameters, but it gives the biggest improvement. The MHA virtual node is cheap, about 67k parameters and gives a smaller gain. The plain virtual node costs almost nothing and gives almost nothing back. The takeaway is simple, keep loop attention in every model and use the virtual node only in its attention form.

Across all five variants the model tells amplifying circuits from non-amplifying ones about equally well, the validity classification (Table 4.6) stays near 97% even with loop attention gone. So the attention mainly improves the predicted gain *value*, not whether a circuit amplifies at all. The full set of thresholds is in Appendix A.3.

Variant	Validity accuracy (%)
Reference	97.9
Loop off	96.4
VN off	97.9
VN plain	97.0
Both off	97.3

Table 4.6: Accuracy of classifying whether a circuit amplifies (DC gain > 0), per attention variant. It stays near 97% throughout, even with loop attention removed, so the attention components affect only gain value.

4.2.3 Backbone Structure

Two things about the backbone are tested here. How deep it should be and whether splitting last layers into separate towers helps. The depth is swept from 4 to 16 layers and the tower variant keeps the same total depth but routes the last layers through two separate towers. Everything else is held at the reference setting.

Table 4.7 gives the depth sweep. Eight layers, the reference, is the best depth across the board, as it has lowest voltage error, lowest current error, lowest gain error and the best small-signal parameters. Four and six layers are too shallow and at twelve and sixteen layers every quantity is worse again. The familiar pattern of deeper graph networks becoming harder to train and starting to over-smooth.

Depth	V (mV)	I (μ A)	g_m (log)	g_{ds} (log)	Gain (dB)	Params
4 layers	8.55	6.00	0.026	0.039	5.73	2.00M
6 layers	7.21	7.90	0.024	0.038	5.32	2.46M
8 layers (ref)	6.42	3.74	0.020	0.034	4.36	2.93M
12 layers	8.52	6.20	0.027	0.045	5.17	3.86M
16 layers	8.38	5.40	0.025	0.040	4.96	4.79M

Table 4.7: Backbone depth sweep on fan_smc, single-tower, only the layer count changed. Eight layers is best on every quantity.

The within-threshold view in Table 4.8 tells the same story. Eight layers places the most predictions inside every tight tolerance and twelve and sixteen layers drop off clearly, the share of currents within 1% of their value falls from 44% at eight layers to around 31–32% at twelve and sixteen. The deeper models are not just worse on average, but they also get a smaller fraction of the predictions right.

Depth also costs parameters. The four-layer model has about 2.0M parameters and the sixteen-layer model about 4.8M, more than twice as many, while being clearly worse than the eight-layer reference at 2.9M. The extra capacity is not the bottleneck.

The other question is whether the backbone should be a single shared stack or split into towers. In a tower design, the backbone is shared for the first few layers and then branches into separate sub-networks, the towers, each refining the shared representation for a different group of outputs. The variant tested here keeps the eight-layer depth, six shared layers followed by two tower layers each. One tower predicts the small-signal parameters (g_m and g_{ds}), the other the operating-point values (the node voltages and the terminal currents).

Depth	V @1%	I @1%	g_m @10%	g_{ds} @10%	Gain @3 dB
4 layers	51.6	35.8	87.9	79.3	54.8
6 layers	55.7	39.6	89.2	79.6	57.2
8 layers (ref)	60.6	43.5	90.7	82.1	64.9
12 layers	48.4	30.8	86.2	71.1	54.2
16 layers	50.1	32.0	87.5	76.5	57.6

Table 4.8: Share of predictions (%) within a tight tolerance for the depth sweep on `fan_smc`. Voltage within 1% of the supply range, current within 1% of its own value, g_m and g_{ds} within 10%, DC gain within 3 dB. Eight layers is best throughout.

Table 4.9 compares it to the single-tower. The multi-tower is worse across all quantities. The clearest loss is in the current, which nearly doubles from 3.74 to $7.3\mu\text{A}$. The gain is barely affected. So splitting the backbone gives worse performance while adding unnecessary parameters.

Variant	V (mV)	I (μA)	g_m (log)	g_{ds} (log)	Gain (dB)
Single-tower (ref)	6.42	3.74	0.020	0.034	4.36
Multi-tower (6+2+2)	7.56	7.30	0.024	0.040	4.74

Table 4.9: Single-tower versus multi-tower backbone on `fan_smc`, at the same total depth of eight layers. The multi-tower is worse on every quantity.

A third option, refining the prediction over several passes instead of a single forward pass, was tried only on the simpler two-stage op-amp. It trained very unstably and was not carried over to the three-stage circuits, therefore it was not included in the comparison here.

4.2.4 Prediction Heads

The other quantities are read off local node or device representations, but the gain is a single circuit number, and there are many ways to produce it. This section first checks that the chosen analytical formula is accurate on its own, then compares eight different ways of building the head.

The physics head is built around an analytical expression for the gain, evaluated on the predicted g_m and g_{ds} , with a small MLP correcting whatever the formula misses (the full expression is in Appendix A.4). To see how good the formula is by itself, it can be evaluated on the ground-truth g_m and g_{ds} and compared to the ground-truth gain. This way it reaches a mean error of 2.70 dB, a median of 1.54 dB and a correlation of 0.995 with the true gain. So with correct small-signal inputs the formula already reproduces the gain closely, which is what makes it worth embedding. The head starts from a good estimate and only has to clean up the rest.

The exact form of the formula matters a lot. Figure 4.4 and Table 4.10 compare the chosen formulation against three simpler ones. Replacing the cascode output resistance of the first stage with a plain one (b) nearly doubles the error but still tracks the gain. Modelling the third stage as a simple class-A stage (c) [16], the usual textbook assumption, in which only

the output transistor carries the gain, is far worse. The third stage of this circuit is in fact push-pull [15], the feedforward path carries most of its transconductance and the class-A assumption drops that path. So the class-A formulation is completely wrong for this output stage. Only the formulation that keeps both the cascode boost and the push-pull path follows the simulator.

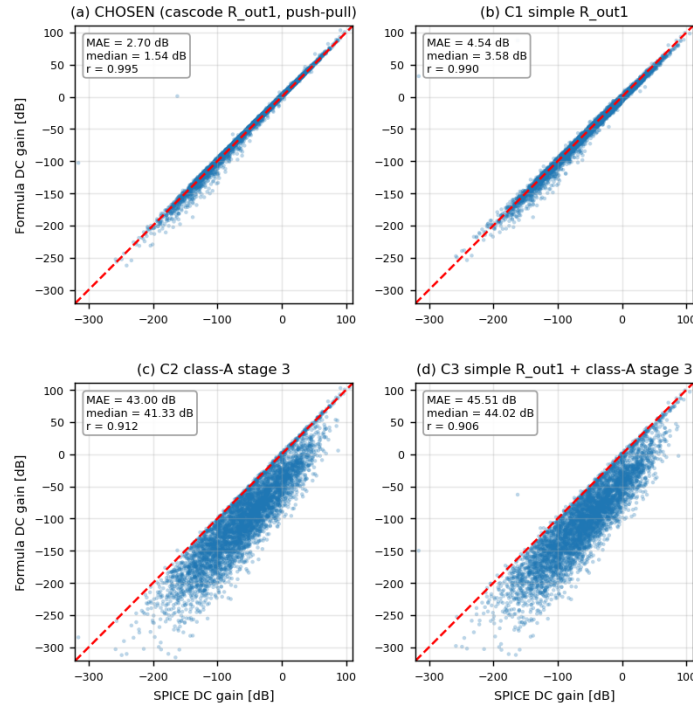


Figure 4.4: Accuracy of four candidate DC-gain formulas on `fan_smc`, evaluated on the ground-truth g_m and g_{ds} against the simulated gain. (a) The chosen formula and (b) the simpler R_{out1} variant tracks the diagonal. For (c) and (d) model the third stage as class-A, which under-predicting the gain because they omit the push-pull feedforward path.

Formula	MAE (dB)	Median (dB)
Chosen (cascode R_{out1} , push-pull stage 3)	2.70	1.54
Simple R_{out1}	4.54	3.58
Class-A stage 3	43.00	41.33
Simple + class-A (textbook cascade)	45.51	44.02

Table 4.10: Accuracy of the candidate DC-gain formulas on `fan_smc`, using ground-truth g_m and g_{ds} against the simulated gain. The chosen formulation is by far the most accurate.

With the formula settled, the question is how to combine it with the network or whether to use it at all. Eight versions of the head are compared in two families. The learned heads predict the gain straight from the graph, pooling or flattening node embeddings into an MLP with no physics built in. The physics heads start from the formula and add a learned correction in different ways.

Table 4.11 gives the result. The best learned head reaches about 9.9 dB, while all four physics heads do better. The best of all is `physics`, which feeds the formula’s estimate and its intermediate terms into a single MLP, at 4.36 dB, more than twice as accurate as the best learned

head. The cleanest comparison is physics against concat_mlp, the best learned head, because concat_mlp is handed the same signal-path devices the formula uses, just without the formula.

Among the learned heads the ordering follows how much device-specific information each one sees. The worst reads only a single global summary of the graph (vn_mlp). Adding a second global statistic helps a little (pool, which also pools the per-dimension maximum over all nodes). Pooling the devices per stage helps more, since the head then knows which stage each summary came from (stage_pool). Reading every signal-path device on its own slot is best (concat_mlp), as the head can then tell the individual transistors apart. The more device-level structure the head can see, the better it predicts get.

Mode	Family	MAE (dB)	Med. (dB)	@3 dB	@5 dB	@10 dB
vn_mlp	learned	13.75	6.75	26.5	40.7	64.3
pool	learned	11.36	5.04	32.0	49.8	73.0
stage_pool	learned	11.23	4.04	40.4	57.2	76.3
concat_mlp	learned	9.89	3.54	43.8	61.9	79.8
physics	physics	4.36	1.93	64.9	79.3	90.9
physics_residual	physics	5.64	2.66	54.0	71.0	86.6
physics_cross_attn	physics	7.37	4.10	41.0	57.1	79.6
physics_gated	physics	9.82	2.86	51.6	64.3	79.8

Table 4.11: DC-gain head comparison on fan_smc. The learned heads predict the gain from node embeddings, the physics heads start from the analytical formula and add a learned correction. “@x dB” is the share of circuits with gain error within x dB

With the formula settled, the question is how to combine it with the network or whether to use it at all. Eight versions of the head are compared in two families. The learned heads predict the gain straight from the graph, pooling or flattening node embeddings into an MLP with no formula involved. The physics heads all build on the same analytical formula, which takes the predicted g_m and g_{ds} of the signal-path devices to estimate gain. What differs is how that formula is used.

In physics the estimated gain, the per-stage gain terms and the device g_m and g_{ds} values all go into a single MLP that outputs the prediction in one shot (Appendix A.4). The other three keep the formula estimate as an explicit base and add a learned correction. physics_residual adds the output of a second MLP over the signal-path device embeddings. physics_gated blends the formula estimate and the learned prediction with a gate that decides how much to trust each. physics_cross_attn forms the correction by cross-attention over the signal-path device embeddings and adds it through a gate.

It is worth noting that the full head, at 4.36 dB, is less accurate than the formula on its own at 2.70 dB. That is expected, as the standalone number uses the true g_m and g_{ds} , while the head works from the model’s predicted values. The small-signal predictions are the bottleneck here.

One thing the head choice does not change much is whether the model can tell an amplifying circuit from a non-amplifying one. The validity classification stays between 94.5% and 97.9% across all eight heads, even the worst one. The full set of thresholds is in Appendix A.3.

The current head has a design choice worth checking too. A device carries one current through it. The current going into one terminal comes out the other, so the per-terminal currents are not independent. The pooled head uses this. Instead of predicting a current at each terminal on its own, it pools the device's terminals and predicts a single current per device, which is then shared across the terminals. This way the consistency is enforced by the architecture. A plain per-terminal head has no such guarantee and can predict currents that do not match, which the model would otherwise have to learn to avoid. Table 4.12 compares the two.

Pooling helps mostly on the current. The current error drops from 4.80 to 3.74 μA , about 28% and the share of currents within 1% goes from 40 to 44%. The other quantities get a bit better too, because a cleaner current feeds back through the shared backbone. The one place the two are even is the current log-error. This means, since lower linear (μA) MAE is dominated by the large values, pooling does better on the large currents.

Current head	V (mV)	I (μA)	I (log)	g_m (log)	g_{ds} (log)	Gain (dB)
Device-pooled (ref)	6.42	3.74	0.027	0.020	0.034	4.36
Per-terminal	6.99	4.80	0.026	0.021	0.037	5.08

Table 4.12: Device-pooled versus per-terminal current head on `fan_smc`. The pooled head predicts one current per device and shares it across the device's terminals. The per-terminal head predicts each terminal on its own. Pooling improves every metric.

4.3 Physics Constraints

This work builds physics into the model in two ways. Implicitly, through the architecture, which was covered in Section 4.2 and explicitly, through a constraint added to the loss. This section is about the explicit constraint. It first looks at which physical relationships are worth enforcing as a constraint and which are not (Section 4.3.1), then shows what the chosen constraint buys in terms of data efficiency (Section 4.3.2) and finally checks that it does what it is meant to, making the predictions physically consistent (Section 4.3.3).

4.3.1 Choosing the Constraint

In the reference model is only KCL active and the reason will be explained in this section. There are many other possible physics constraints. The current mirrors should match, the differential pair should split its tail current evenly, the devices should follow their square-law or sub-threshold relations, the bandwidth should follow from the first-stage transconductance.

The starting point is how well the analytical relations hold on their own. The per-region transconductance formulas are from the textbook. In saturation $g_m = 2I_D/V_{ov}$, in the triode region $g_m = I_D/(V_{ov} - V_{DS}/2)$ and in weak inversion $g_m = I_D/(nV_T)$, where $V_{ov} = V_{GS} - V_{th}$ is the overdrive voltage, V_T the thermal voltage and n the sub-threshold slope factor. Each holds only in its own region and each can be evaluated on the ground-truth values and compared

to the simulated g_m , with no model involved. Table 4.13 and Figure 4.5 show the result. Both the triode and cutoff have quite high median relative error. The saturation relation $g_m \approx 2I_D/V_{ov}$ looks reasonable only after heavy filtering. Its median relative error falls to a usable 8% only when restricted to the 15% of devices with an overdrive above 100 mV.

Formula	Region	Median rel	r
Triode g_m ($V_{ov} > V_{DS}$)	triode	15.8%	0.996
Cutoff g_m	cutoff	19.3%	0.98
Saturation g_m (no filter)	saturation	36.8%	0.88
Saturation g_m ($V_{ov} > 100$ mV)	saturation	8.3%	0.999
SMAXT (unified)	all	5.5%	0.997

Table 4.13: Accuracy of the analytical device formulas on `fan_smc`, evaluated on the ground-truth small-signal values. The per-region formulas hold poorly. The unified SMAXT formula is the most accurate.

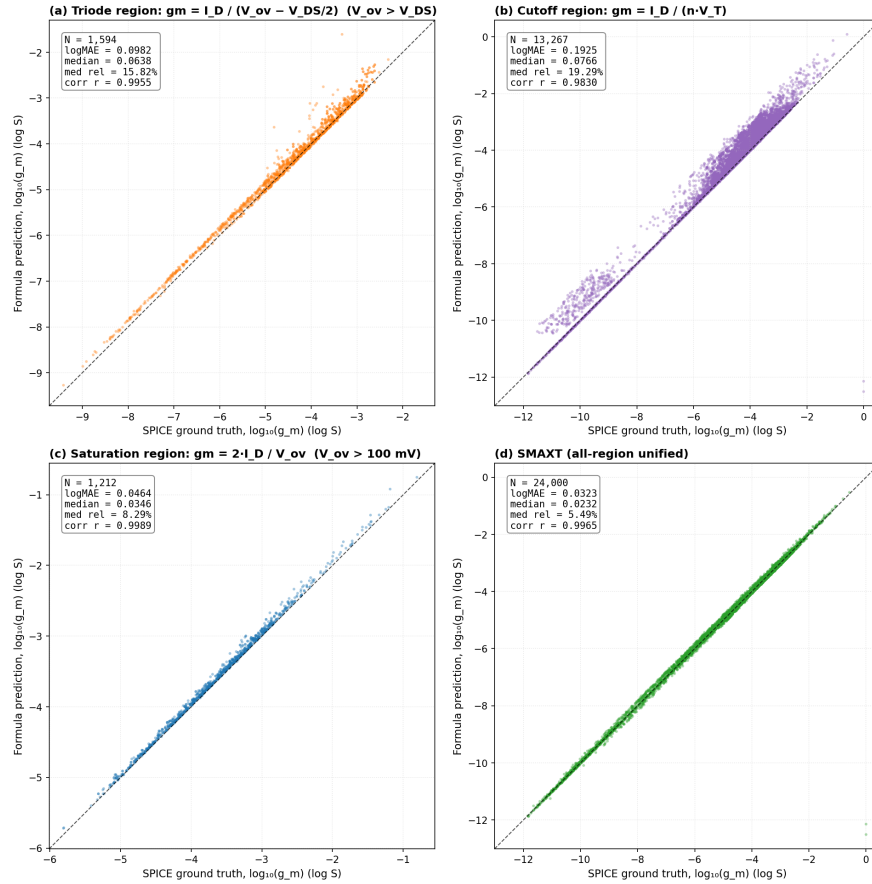


Figure 4.5: Analytical g_m formulas against the simulated g_m on `fan_smc` evaluated on ground-truth inputs. (a) triode, (b) cutoff, (c) saturation, (d) the unified SMAXT formula.

A big problem is the overdrive voltage $V_{ov} = V_{GS} - V_{th}$. The per-region formulas divide by it and near threshold V_{ov} goes to zero, so the formula blows up. This is why the unfiltered saturation relation has a high mean relative error, because of the near-threshold devices.

SMAXT avoids this. Continuous formula that describe the transistor across all operating regions in a single smooth expression, rather than switching between separate region formulas,

for example the EKV model [31] is valid from weak through strong inversion. SMAXT follows the same idea with a simpler construction. The three standard region relations [16] are joined with softplus, which approximates max and min without a kink:

$$g_{m,\text{SMAXT}} = \frac{I_D}{\text{denom}}, \quad \text{denom} = nV_T + nV_T \text{softplus}\left(\frac{V_{ov} - V_{DS,\text{eff}}/2 - nV_T}{nV_T}\right), \quad (4.1)$$

where nV_T , mV is the floor the denominator cannot drop below. It recovers each formula in its own limit, $2I_D/V_{ov}$ in saturation, $I_D/(V_{ov} - V_{DS}/2)$ in triode and $I_D/(nV_T)$ below threshold, as Figure 4.6 shows. Full derivation is in Appendix A.6.2.

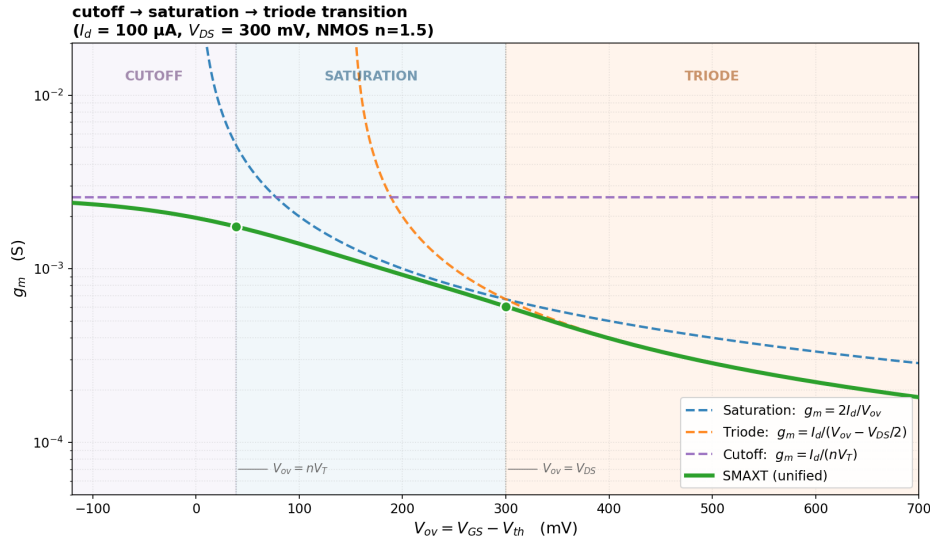


Figure 4.6: The SMAXT transconductance (green) against the three textbook equations. NMOS device as the overdrive V_{ov} is swept at fixed $V_{DS} = 300$ mV. In each region SMAXT follows the corresponding textbook formula and it stays smooth at the boundaries.

The problem is still overdrive $V_{ov} = V_{GS} - V_{th}$. The threshold voltage V_{th} is taken from the simulator, but the gate voltage V_{GS} is predicted, so any error in the prediction carries into V_{ov} . Table 4.14 tests how much this matters, by varying the overdrive voltage. SMAXT degrades by about a factor of two for a ± 50 mV error, much less than the standard saturation formula.

V_{ov} shift	SMAXT median rel	SMAXT $\log_{10}$ MAE
-50 mV	13.7%	0.065
-25 mV	8.0%	0.044
0 (truth)	5.5%	0.032
+25 mV	4.6%	0.032
+50 mV	5.9%	0.043

Table 4.14: Sensitivity of the SMAXT formula to an error in the overdrive voltage on `fan_smc`.

The bandwidth formula is much worse. The single-pole relation $UGBW \approx g_{m1}/(2\pi C_L)$ ignores the non-dominant poles and the Miller compensation. The Figure 4.7 shows it does not track the simulated value at all. This is why the unity-gain bandwidth is not among the predicted quantities. Several ways of predicting it were tried. A plain MLP head on the graph embedding, the analytical formula with a learned correction on top and a signal-flow

graph added to the model. The best approach still sat around 45% median relative error. The problem is that the model only learns the operating point, which holds no information about the frequency response, and the formula it would fall back on is too inaccurate to help. Not every sample has a UGBW either, as shown in Figure 3.1, which leaves less data to learn from.

Predicting the bandwidth caused another problem. It made other quantities worse. The voltage, current and small-signal errors roughly doubled. So the bandwidth could not be added on top of the model without hurting the rest, which is the second reason it was left out. The full set of approaches and numbers are in Appendix A.7.

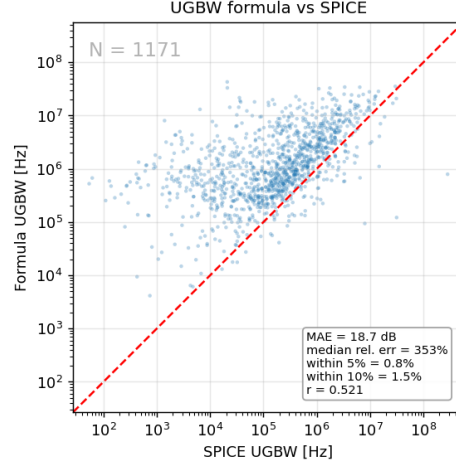


Figure 4.7: The analytical unity-gain bandwidth, $g_{m1}/(2\pi C_L)$, against the simulated value on `fan_smc`. The single-pole formula does not track the real bandwidth.

These constraints, other than the bandwidth formula, are applied only during training. Each one needs the ground-truth threshold voltage, which is not available at inference. The constraints therefore act as training-time guidance. Each constraint is added on top of the reference. Table 4.15 gives the result.

Added constraint	V (mV)	I (μ A)	g_m (log)	g_{ds} (log)	Gain (dB)
None (KCL only, ref)	6.42	3.70	0.020	0.034	4.36
Mirror / diff-pair	8.29	20.60	0.025	0.040	5.03
UGBW (single-pole)	13.77	9.70	0.036	0.056	6.13
Triode (square-law)	8.77	7.40	0.031	0.051	5.24
Cutoff (fixed slope)	7.08	13.90	0.065	0.035	4.67
g_m (saturation/SMAXT)	8.83	5.80	0.032	0.043	5.30
g_m/I_D consistency	7.62	5.30	0.025	0.038	4.70

Table 4.15: Effect of adding each physics constraint on top of the KCL reference, on `fan_smc`. Every constraint degrades accuracy.

The reason all of the constraints performs worse is the accuracy of the formula and the conflicting learning signal against the direct supervision. Take for example the current mirror. It forces the mirror current to follow the width ratio, $I_{\text{mirror}} = (W/L)I_{\text{ref}}$, which only holds for matched devices with infinite output resistance. Forcing the model toward the ideal ratio drags it away from the true currents. The triode and cutoff constraints fail as well, because the formulas behind them are inaccurate as seen from Table 4.13. Even the SMAXT-based g_m

constraint hurts, despite being the most accurate formula.

The g_m/I_D consistency term is not an approximation. It uses no formula, it only asks that the predicted g_m and I_D be as consistent with each other as the simulator's. It still hurts a little, but for a different reason. It adds a third gradient path through both heads at once and pulls them off their own best solutions. So even an exact extra constraint is not always worth adding.

KCL is the exception that works. It is exact, the currents into a node sum to zero with no approximation and no device parameters at all. Everything else here is either a simplification that disagrees with BSIM4 or an extra term that interferes with training. This is also why the KCL benefit is not just regularization, a generic penalty does not help and neither does an approximate physical one. Only the constraint that is physical exact improves the model.

There is a deeper reason the analytical relations struggle here, beyond being an approximation. the parameters they treat as constant are not constant in BSIM4. Figure 4.8 shows the spread of three of them across the dataset. The threshold voltage $|V_{th}|$ varies by hundreds of millivolts. The effective transconductance parameter μC_{ox} spans several orders of magnitude. And the sub-threshold slope factor n , which the formulas fix at 1.5 and 2.0, has a wide spread of its own.

The textbook saturation drain-current relation, $I_D = \frac{1}{2}\mu C_{ox}(W/L)V_{ov}^2$, depends on all of these at once, μC_{ox} , the overdrive $V_{ov} = V_{GS} - V_{th}$ and the device geometry. Worse, the overdrive enters squared, so the error is amplified. That is why no drain-current constraint was implemented at all. Only the small-signal relations, which need fewer of these and enter V_{ov} in first order, were tried, as the ablation showed, did not help.

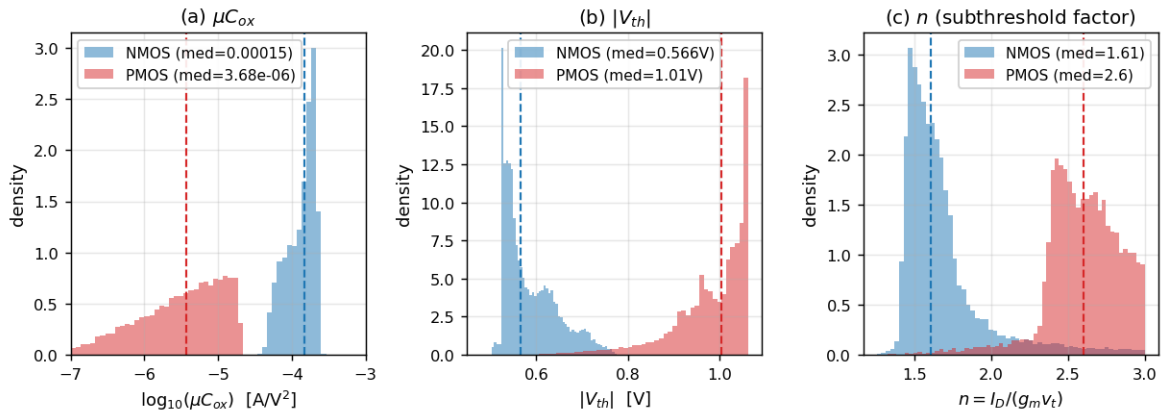


Figure 4.8: Spread of three parameters the textbook equations treat as constant, on fan_smc. (a) the effective transconductance parameter μC_{ox} , (b) the threshold voltage $|V_{th}|$, (c) the sub-threshold slope factor n . None of the three is constant in the BSIM4 model.

4.3.2 Data Efficiency

The main claim of this work is that the physics constraint lets the model reach a given accuracy with less data. The model is trained on fan_smc, from 100 samples up to the full training set of 4000, once with the KCL constraint on and once without. The accuracy is

always checked on the same 1000-sample validation set. Everything except the training-set size and the KCL flag is held at the reference setting, so the full-data end of the constrained curve is the reference model of Section 4.1.

Figure 4.9 shows the error against training-set size for each quantity. The first thing to say is that more data helps everything, both curves fall as the training set grows. The second is that the KCL curve sits below the no-KCL curve at every size and on every quantity.

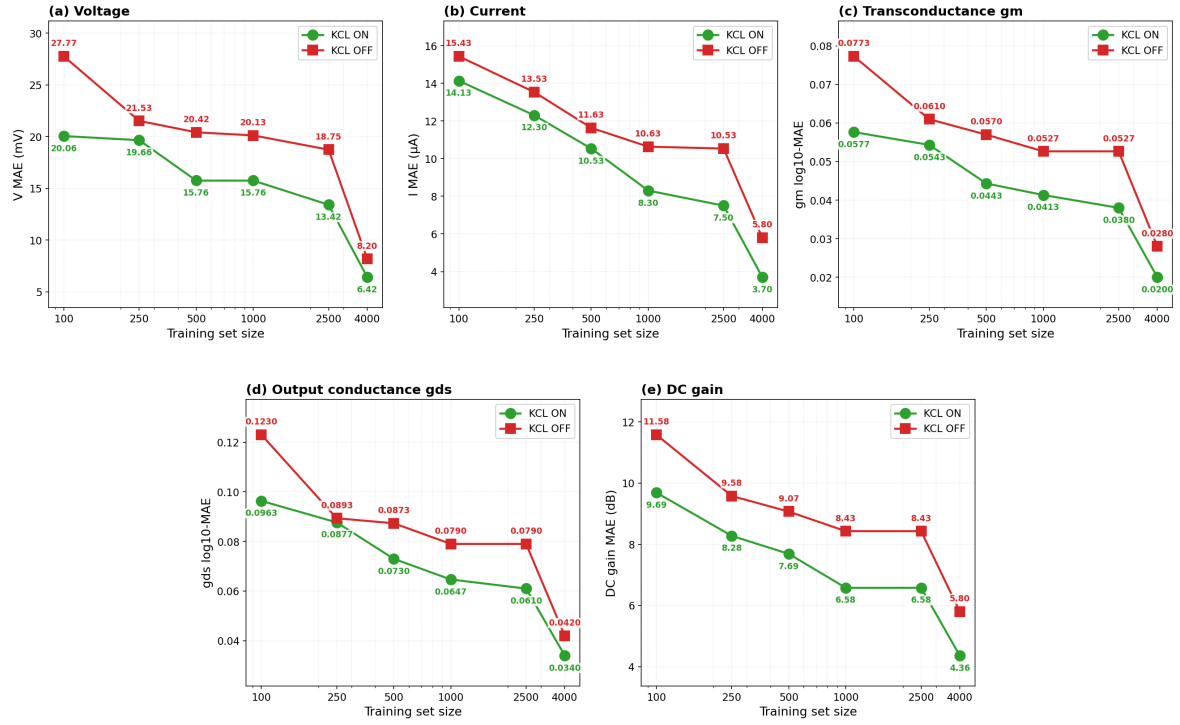


Figure 4.9: Mean absolute error against training-set size on `fan_smc` on the fixed 1000-sample validation set. KCL curve is below the no-KCL curve at every size on every quantity.

The within-threshold view agrees. Figure 4.10 plots, for each quantity, the share of predictions that land within a tight tolerance. The KCL curve is above the no-KCL curve at every size on every quantity. So the constraint does not just lower the average error, it also raises the share of predictions that are tight.

The data-efficiency of the model with KCL is higher. With 100 samples the constrained model reaches a voltage error of 20.1 mV, a level the unconstrained model still shows at 1000 samples, since its voltage error barely moves between 250 and 2500. At 2500 the constrained model is at 13.4 mV, which the unconstrained model only passes on the way to the full set. The other quantities show the same pattern with smaller factors, at the low end the constrained model gets by with roughly half to a quarter of the data.

The benefit does not fade with more data either. At the full 4000 samples the constrained errors are between about 19 and 36 percent lower depending on the quantity, the current benefits the most and the output conductance the least. The constraint helps in both ways, the model reaches a given accuracy with less data and it is still better when all the data is there. The exact numbers for every run are in Appendix A.5.

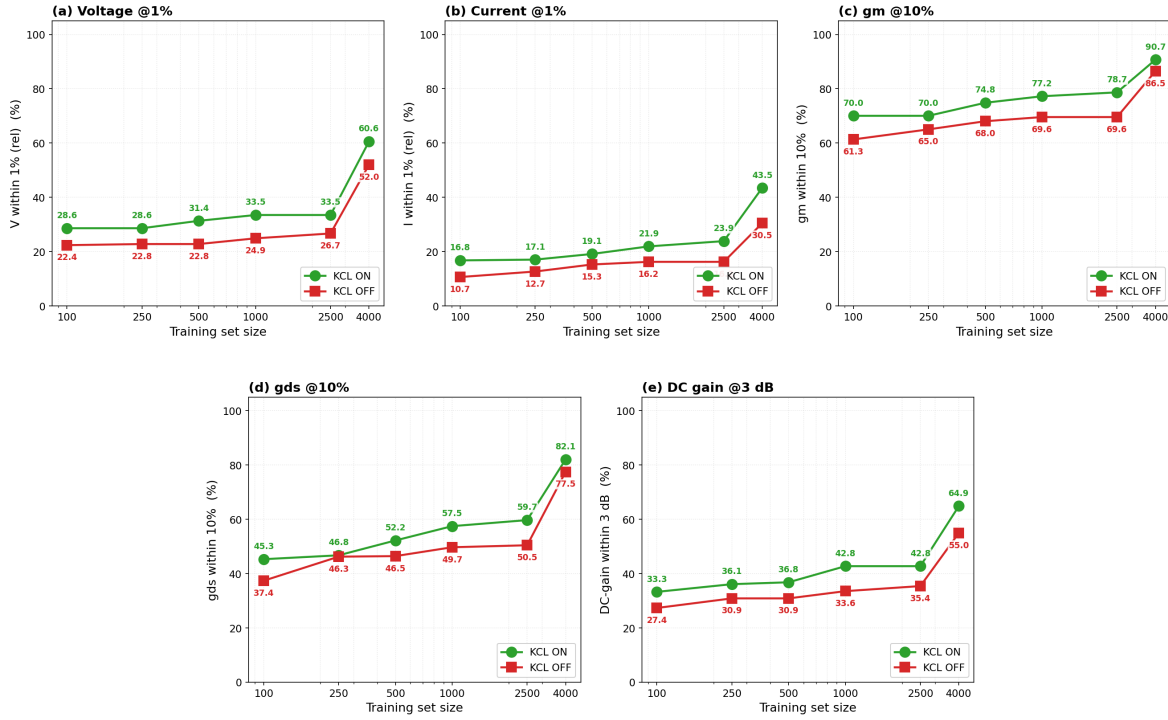


Figure 4.10: Within-threshold accuracy against training-set size on `fan_smc` showing the share of predictions inside a tight tolerance (voltage within 1% of the supply range, current within 1% of its value, g_m and g_{ds} within 10%, DC gain within 3 dB). The KCL curve is above the no-KCL curve at every size on every quantity.

4.3.3 Physics Consistency

The data-efficiency results showed that KCL makes the model more accurate. That is a different thing from whether the model actually obeys KCL. A model could be accurate on average and still predict currents that do not balance at the nodes.

At every internal net the currents flowing in should add up to zero. The residual is how far from zero they actually add up. It is measured two ways, as a raw current in amps and as a fraction of the total current through the net so that small and large currents can be compared. Running the same check on the simulated currents shows how well the simulator itself does.

Table 4.16 shows the numbers. The median imbalance is cut roughly in half, from 0.97% without KCL to 0.51% with it and the raw median drops from 112 to 58 nA. This proves that the constraint really do pull the predictions toward conserving current.

Two things are worth noting. The first is that the worst nets do not improve. At the 95th and 99th percentiles the residuals are large and about the same with KCL on or off, so the constraint helps the typical net but not the hardest ones. These carry a small total current where the terminal currents nearly cancel, hence a small absolute error still looks large. The second is that KCL is only a soft constraint, not a hard rule like the ones, explained in Section 4.2). The simulator conserves current about a thousand times better than the model, so the predictions still break conservation far more than the true physics does.

Normalized residual	KCL on	KCL off	GT floor
median	0.51%	0.97%	0.0002%
p90	15.8%	16.7%	0.009%
p95	56.0%	55.7%	0.028%
p99	88.3%	88.3%	0.284%
median (raw)	58 nA	112 nA	0.025 nA

Table 4.16: KCL residual on the `fan_smc`. The imbalance of the predicted currents at each net as a fraction of the current through it. The last row gives the raw median imbalance in amps. The constraint roughly halves the median imbalance but leaves the tail unchanged.

4.4 Generalization to Unseen Topologies

So far the model was trained and tested only on one topology. This section shows how well it carries over to other topologies. It is split into three parts. The first is training on all five topologies at once and seeing whether one model can handle them all (Section 4.4.1). The second is holding one topology out and adapting to it with little or no data from it (Section 4.4.2). The third looks at what role the KCL constraint plays when the model moves to a new topology (Section 4.4.3). As in the rest of this chapter the gain is left out here, since it needs a separate formula per topology, hence only the operating point and the small-signal parameters are predicted.

4.4.1 Joint Training Across Topologies

The simplest way to use more than one topology is to train on all of them at once. One model is trained on all five circuits together, 4000 samples from each or 20,000 in total, with every batch holding an equal share of each topology, and then tested on the remaining 5000 samples. The model is the same as before and it is never told which topology a sample is from. Gain is not predicted here, since each topology needs a different analytical gain formula and for some topologies that formula does not reflect the real behavior, so this part covers the operating point and the small-signal parameters only.

Table 4.17 compares this to training each topology on its own. Joint training helps every topology and on every quantity. The gap is small where the topology was already predicted well, a couple of millivolts on `fan_smc` and `peng_tcfcc`. The largest voltage improvements are on `leung_nmcfc` and `sau_cfcc`, where the error drops from 26.4 to 19.7 and from 26.8 to 21.6 mV. The currents improve the most, on `fan_smc` the error is more than halved.

This however is not a fair comparison. The joint model is trained on five times as much data, 20,000 samples against 4000, so part of the improvement could just be the extra data rather than the mix of topologies. To separate the two, the joint model was trained again on the same total budget as a single-topology run, 4000 samples split across the five, so 800 from each. Now the only difference from single-topology training is whether those 4000 samples come from one topology or are spread across five. Table 4.18 shows this comparison.

At equal data, joint training is slightly behind single-topology training on every topology, by a fraction of a millivolt on `fan_smc` and `peng_tcfcc` and by three to four millivolts on `sau_cfcc` and `leung_nmcfc`. That is expected, each topology only sees a fifth of the samples.

Topology	Training	V (mV)	I (μ A)	g_m (log)	g_{ds} (log)
fan_smc	single	14.13	9.43	0.036	0.056
	joint	11.53	4.25	0.030	0.041
sau_cfcc	single	26.79	26.57	0.063	0.081
	joint	21.58	14.27	0.054	0.068
peng_tcfc	single	11.57	5.25	0.035	0.043
	joint	9.45	4.26	0.022	0.032
leung_nmcfc	single	26.37	11.14	0.064	0.099
	joint	19.68	7.97	0.047	0.070
leung_nmcnr	single	12.89	7.34	0.036	0.050
	joint	11.19	4.64	0.024	0.035

Table 4.17: Per-topology accuracy with single-topology training (4000 samples of one topology) and with joint training on all five (20,000 in total) on the same 1000-sample test set per topology. Joint is better on every topology and every quantity.

Topology	Single (4000 of one)	Joint (800 $\times$ 5)	Joint (full, 4000 $\times$ 5)
fan_smc	14.13	14.35	11.53
sau_cfcc	26.79	29.78	21.58
peng_tcfc	11.57	12.19	9.45
leung_nmcfc	26.37	30.13	19.68
leung_nmcnr	12.89	14.60	11.19

Table 4.18: Single-topology uses 4000 samples of one topology. The middle column is joint training on 4000 total, split across five. The full joint model (20,000) is shown for reference. At equal data, spreading across topologies comes close to concentrating on one but stays slightly behind on every topology.

So the improvement of the full joint model over single-topology training comes from the extra data, not from the mix of topologies. What the mix buys is different, the joint model reaches nearly the per-topology accuracy of a dedicated model from a fifth of the per-topology data, while covering all five circuits at once.

This is a useful finding. A single model can handle all five topologies with about the same data budget as one model and stays close to the accuracy on each.

4.4.2 Adapting to an Unseen Topology

Joint training assumes all the topologies are available up front. The harder case is a new topology the model has never seen. This section looks at two ways to handle it. The first is zero-shot, the model trained on the other four is used directly on the new one. The second is to take the same pretrained model and fine-tune it on a small amount of data from the new topology. As before, only the operating point and the small-signal parameters are predicted.

Using the model with no fine-tuning does not work. Table 4.19 shows what happens when a model trained on four topologies is tested directly on the fifth. The voltage error is roughly three to thirteen times worse than a model trained on that topology alone. `fan_smc` holds up best. `leung_nmcnr` is the worst, its zero-shot error is the highest of the five while its own model trains among the best.

Held-out topology	Zero-shot V (mV)	Trained on it (mV)	Worse by
<code>fan_smc</code>	40.0	14.13	2.8×
<code>sau_cfcc</code>	137.3	26.79	5.1×
<code>peng_tcfc</code>	92.0	11.57	8.0×
<code>leung_nmcf</code>	133.5	26.37	5.1×
<code>leung_nmcnr</code>	171.5	12.89	13.3×

Table 4.19: Voltage error on a held-out topology when the model is trained on the other four and applied directly to the fifth, against a model trained on that topology alone. The error is roughly three to thirteen times worse.

Fine-tuning works much better. The model is first pretrained on the other four topologies with 16,000 samples in total and then fine-tuned on N samples of the new topology. The fine-tuned model has seen far more data overall, which is part of its advantage. Figure 4.11 shows the voltage error against N .

At small N the pretrained model is far ahead. With only 100 samples the voltage error is between about one and a half and nearly five times lower than training from scratch, since scratch does not have enough data to work with. At full data the pretrained model is still ahead on every topology. Across the whole range it wins or ties. Part of this is simply that the pretrained model has seen far more data. How much comes from the data and how much from the structure carried over from the other topologies cannot be separated here.

Two things to keep in mind. Outside the very low-data regime the saving is smaller, the pretrained model reaches the scratch model's full-data accuracy with roughly a quarter of the data on most topologies, and about two-thirds on `sau_cfcc`, the hardest case. And `leung_nmcf` is where the margin stays widest, scratch trains worst on it and is still well behind at the largest size, so fine-tuning keeps a clear lead there.

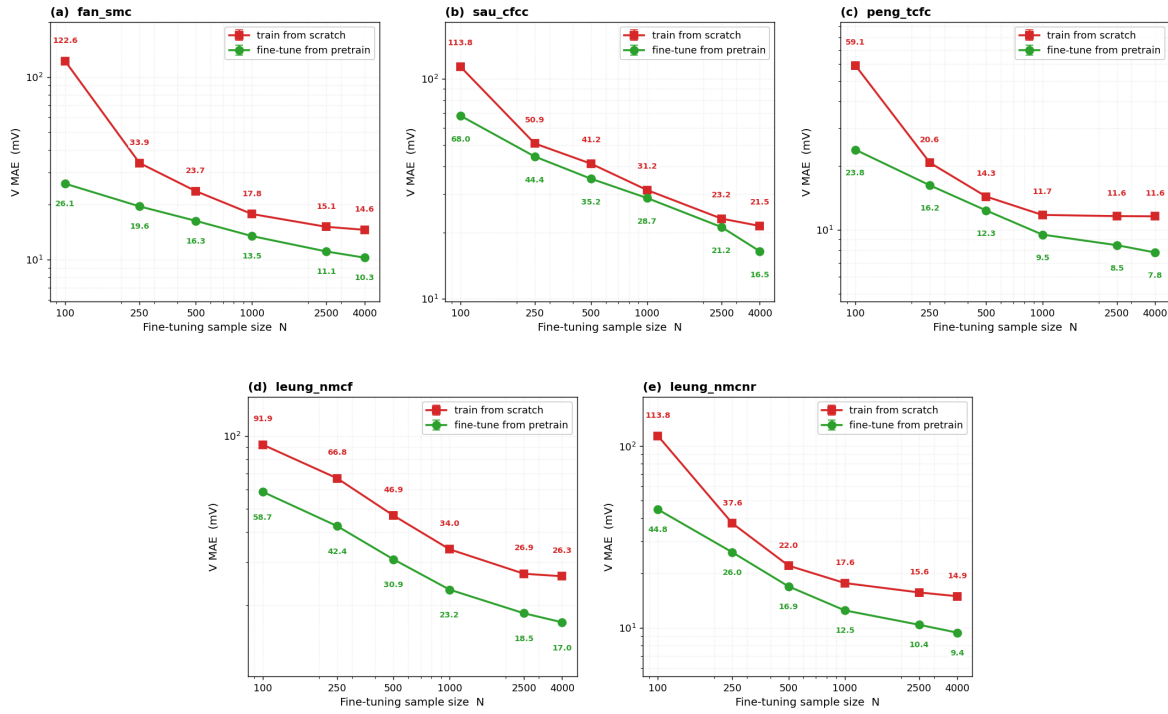


Figure 4.11: Fine-tuning from the pretrained model against training from scratch, voltage error at each sample size N . Fine-tuning is below scratch at every size in every topology and the gap is widest at small N . It narrows as N grows and comes close on *fan_smc* and *peng_tcfc*, but stays wide on *sau_cfcc* and *leung_nmcfc*.

Topology	N	V (mV)	I (μA)	g_m (log)	g_{ds} (log)
fan_smc	100	-96.5	-14.3	-0.229	-0.312
	4000	-4.3	-3.5	-0.007	-0.013
sau_cfcc	100	-45.7	-6.7	-0.112	-0.172
	4000	-5.0	+0.9	-0.009	-0.016
peng_tcfc	100	-35.4	-9.7	-0.079	-0.130
	4000	-3.7	-0.8	-0.013	-0.011
leung_nmcfc	100	-33.2	-6.1	-0.095	-0.148
	4000	-9.3	-3.3	-0.016	-0.023
leung_nmcnr	100	-69.1	-11.0	-0.124	-0.194
	4000	-5.5	-1.8	-0.010	-0.014

Table 4.20: Fine-tuning minus scratch on all four quantities, for all five held-out topologies, at the smallest and the full sample size. Negative means fine-tuning is better. Fine-tuning wins on every quantity for every topology. The full range is in Appendix A.8.

4.4.3 The Role of Physics in Transfer

The fine-tuned model in the previous section carried the KCL constraint through both the pretraining and the fine-tuning. This raises a question. Where does the constraint actually do the work. To find that out, KCL was switched on and off in each stage on its own and the transfer accuracy compared on `fan_smc`.

Table 4.21 shows the four combinations. Adding KCL during fine-tuning helps when the pretraining had none and does nothing when the pretraining already had it.

<i>N</i>	KCL in pretrain		No KCL in pretrain	
	no FT-KCL	FT-KCL	no FT-KCL	FT-KCL
100	20.6	21.0	22.6	20.0
500	16.1	15.9	15.7	14.4
1000	14.4	14.2	13.1	12.2

Table 4.21: Voltage error on `fan_smc` for the four ways of placing the KCL constraint.

This further shows that KCL holds for any circuit and it gives useful extra signal during training no matter the topology. The model can pick this up in pretraining or later in fine-tuning. The constraint is worth having somewhere in the pipeline, but there is no big harm from putting it in both stages.

This was only checked on `fan_smc`. The numbers show the direction of the effect rather than exact values.

Chapter 5

Discussion and Conclusion

This thesis set out to predict the DC operating point and small-signal behavior of analog op-amps straight from the netlist, using a graph neural network and physics constraints to keep the predictions consistent with circuit law. This chapter steps back from the individual results and gives a bigger picture. Where the approach works, where it does not and what the findings say beyond the one circuit studied here.

5.1 Summary of Findings

The model predicts the operating point and small-signal parameters of the `fan_smc` amplifier accurately. Node voltages come out to a few millivolts and the currents, transconductances and output conductances to a few percent at the median, across each quantity. The DC gain is the hardest target and the least accurate. It is a single circuit-level number that depends on the whole signal path. It is limited mainly by the small-signal parameters and the number of samples.

The central result is about the physics constraints. Out of all physical relations that could be added to the loss, only Kirchhoff's current law improves the model. Every approximate device relation tried degrades accuracy, the square-law and sub-threshold transconductance formulas, the current-mirror and differential-pair equalities, the single-pole bandwidth relation and the one auxiliary term that is exact but not a conservation law interferes with training. KCL works because it is exact and uses no device parameters at all. It rules out predictions that violate current conservation without ever pushing the model toward a simplified device model formula.

This help extends across the whole data range. In the lower-to-middle range the constrained model reaches the accuracy of the unconstrained one with a fraction of the data, on the voltage with up to ten times less and on the other quantities with roughly half to a quarter. The benefit does not disappear at full data either, there the constrained errors are still around twenty to thirty-five percent lower depending on the quantity.

With KCL active, the current imbalance at the net is about half what it is without it, so the model is not only more accurate but more consistent with current conservation. The effect is limited to the typical net, the worst nets stay about the same and even the improved predictions are far from the level the simulator conserves to.

The graph representation lets one model handle several topologies. Trained on all five at

once, it beats a separate model per topology on every quantity, but that comparison gives it five times the data. At equal data it stays close but slightly behind, what it buys is one model that covers all five circuits at nearly the same accuracy from a fifth of the per-topology data. For a topology held out entirely the model does not work without adaptation, but fine-tuning a pretrained model beats training from scratch at every data size and on nearly every quantity.

5.2 Correct Physics, Not Just Physics

The main lesson here was not the expected one. The idea behind physics-informed learning is that adding known equations helps the model. That turned out to be true for only one of them. For every relation that is just an approximation, adding it made things worse. The formula and the BSIM4 data do not agree, so forcing the model to follow the formula drags it away from the values it should be matching.

The reason for that is the ground truth being BSIM4. The difference between a textbook equation and the simulator is not just random noise that can be canceled out. It is a fixed error that the constraint carries with it. The square-law current ignores velocity saturation and channel-length modulation. The ideal mirror ignores finite output resistance. The single-pole bandwidth ignores the other poles and the Miller compensation that actually set it. Each one drops a real effect that BSIM4 keeps. Even SMAXT, the most accurate formula, hurts once it is enforced. And the parameters these formulas treat as fixed are not fixed in BSIM4. The threshold voltage, the transconductance parameter, and the sub-threshold slope factor all vary a lot, so even a formula with the right physics would still be using the wrong numbers.

KCL gets around all of this because it is not a device equation. It is just current conservation, that holds no matter the process, the region or the device parameters. That is why it is the only one being kept. The takeaway is simple. If the ground truth is a detailed simulator, the constraint has to be exact. An approximate law is not a weaker version of a good prior. It adds a fixed error and it is better to drop it.

There is a better place to put exact physics than the loss. When a relation is exact, it can be built into the architecture instead of added as a soft penalty, so the model cannot break it at all. The current head does this. It predicts one current per device and shares it across the terminals, making the device current consistent by construction. A constraint enforced this way costs the model nothing. Instead of spending that capacity to fit the law, it can use it to improve its own prediction. The soft KCL loss is actually a weaker form of the same idea. So the rule should be to make the exact physics impossible to violate rather than merely discouraged.

5.3 A Harder Problem than Filtered Sizing

A lot of the surrogate work for analog sizing trains on filtered design points, with transistors assumed to be in saturation and strong inversion or restricts the search to well-behaved circuits during optimization [27]. The training distribution here is left unfiltered on purpose. It includes poorly biased, edge-case and non-amplifying circuits.

By not filtering, the model has to handle the bad circuits, not just the good ones, hence its average error will look worse than a model tested only on well-behaved designs. However, this gives the designer more freedom to search the design space, instead of being limited to circuits that are already well behaved. It also differs from equation-based and constraint-optimization methods. Those rely on the simplified device equations holding well enough to enforce, which is exactly the assumption the constraint study here shows to fail. The surrogate makes no such assumption and learns the operating point directly.

5.4 Limitations

The main results are established on a single topology, `fan_smc`. How far they reach across op-amp families is what the generalization study tests and otherwise it is left to future work. The DC gain is supervised only in the single-topology setting and the analytical gain formula it is built on, is specific to this topology and does not transfer.

The scope is limited to the DC operating point and the small-signal parameters. The frequency-domain and stability metrics, the unity-gain bandwidth, the phase margin and the amplitude margin, are out of scope and the constraint study makes clear why. These depend on the full pole-zero structure of the circuit and no simple analytical relation holds for them. The single-pole bandwidth formula does not track the simulated bandwidth at all, hence it could neither be predicted nor enforced as a constraint, and the same goes for the other AC metrics.

One more limit is worth naming. The explicit-physics contribution rests on a single constraint. Out of all relations tried, only KCL survived, so the approach to explicit physics is narrow by result rather than by design.

5.5 Future Work

Many new directions can be explored. The most obvious one is the frequency domain. Predicting the unity-gain bandwidth and the phase margin would need either a learned AC model that does not lean on the single-pole approximation or a constraint derived from the full small-signal admittance rather than a single transconductance.

A second direction is breadth of topology. The generalization experiments here use a small set of op-amps from one benchmark suite. Extending the approach to a wider range of analog blocks and finding a gain formulation that carries across topologies instead of being rebuilt for each circuit, would test how general the graph with physics combination really is.

A third direction is to give the model more information about each device than its dimensions and bias. The operating point depends on the full current-voltage behavior of the transistor, which is currently encoded only indirectly through the sizing and the supply. A compressed representation of the device characteristic could carry more of that behavior. An attempt in this direction used an autoencoder to compress the simulated IV curve into a latent dimension and added it as an input feature (Appendix A.9). The idea itself is sound, the device curve holds many behaviors that analytical formulas miss and finding a representation that helps the model is still an open problem worth pursuing.

A fourth direction concerns the analytical formulas themselves. The DC-gain formula used here was specific to one topology. Candidate forms were proposed with the help of a language model and then checked against the ground truth before one was kept. This proved a practical way to search the space of plausible formulas. Turning this into a loop, where a language model proposes candidate formulas for a given topology, each is validated against the simulator and only the most accurate is used in training, would remove the need to derive a new formula by hand for every circuit. This would directly address the lack of transfer in the current gain formula. The human in the loop to validate the accuracy of the formulas is what makes this safe.

A fifth direction concerns the data regime in which the constraint study was run. The approximate constraints were all tested on the full training set. It is possible they behave differently when data is scarce, where the model has less to learn from and might tolerate an imperfect equation.

Finally, the constraint study. KCL is one exact, parameter-free law and others exist that share the same property of being true regardless of the device model, like Kirchhoff's voltage law or charge conservation. It is not clear whether adding more exact laws on top of KCL would help further or whether KCL alone already did most of the work.

5.6 Conclusion

This thesis combined a graph representation of the op-amp netlist with a physics-informed training signal to predict the DC operating point, the small-signal parameters and the DC gain of analog amplifiers. The higher-level performance numbers, like the gain, are then calculated from that state and the model corrects it. The model reaches that state accurately on a hard, unfiltered distribution and a single physics constraint, Kirchhoff's current law, makes the prediction more data-efficient.

The clearest finding is that adding physics to a learned surrogate helps only when the physics is exact, either hard constraint e.g. directly through the architecture or as soft constraint e.g. in the loss. Every other approximated relation tried here degraded the model, because the simplified equations conflict with the BSIM4 ground truth and even the parameters they treat as fixed are not fixed in the real device model. Knowing that approximate physics hurts is as useful as knowing that KCL helps. Both point to the same conclusion, if the ground truth is a detailed simulator, the only physics worth adding is the kind that is exactly true.

Appendix A

Appendix 1

A.1 Full Dataset Statistics

Table A.1 gives the complete per-topology statistics for each target quantity, the minimum, maximum, mean and standard deviation, over the full 5000 samples of each topology. Voltages are over each topology’s supervised internal nets (the rails and inputs are excluded, the per-topology counts are listed in Table 3.10). Currents and small-signal parameters are in $\log_{10}$, DC gain is in decibels. The summary means and the amplifying fraction are in Table 3.11 in the main text.

Node voltage V (V)					Current $\log_{10} I $ (log A)				
Topology	Min	Max	Mean	Std	Topology	Min	Max	Mean	Std
fan_smc	0.00	1.80	0.61	0.58	fan_smc	-14.92	-1.20	-5.56	1.44
sau_cfcc	0.00	1.80	0.61	0.58	sau_cfcc	-16.12	-1.31	-5.67	1.66
peng_tcfc	0.00	1.80	0.87	0.68	peng_tcfc	-14.54	-1.36	-5.39	1.16
leung_nmcf	0.00	1.80	0.58	0.49	leung_nmcf	-14.50	-1.37	-5.42	1.18
leung_nmcnr	0.00	1.80	0.63	0.61	leung_nmcnr	-14.33	-1.06	-5.47	1.44

Transconductance $\log_{10} g_m$ (log S)					Output conductance $\log_{10} g_{ds}$ (log S)				
Topology	Min	Max	Mean	Std	Topology	Min	Max	Mean	Std
fan_smc	-12.00	-0.58	-4.43	1.44	fan_smc	-14.82	0.54	-5.89	1.89
sau_cfcc	-14.61	-0.63	-4.66	1.60	sau_cfcc	-17.09	0.73	-6.11	2.06
peng_tcfc	-11.98	-0.76	-4.24	1.15	peng_tcfc	-15.23	0.77	-5.53	1.77
leung_nmcf	-11.99	-0.55	-4.15	1.02	leung_nmcf	-15.27	0.34	-5.96	1.64
leung_nmcnr	-11.99	-0.42	-4.34	1.40	leung_nmcnr	-14.71	0.64	-5.83	1.87

DC gain (dB)				
Topology	Min	Max	Mean	Std
fan_smc	-316.0	105.4	-56.9	57.0
sau_cfcc	-264.1	115.3	-59.6	51.5
peng_tcfc	-234.9	108.3	-35.9	47.8
leung_nmcf	-168.0	194.5	+33.0	70.6
leung_nmcnr	-242.9	121.8	-77.3	54.1

Table A.1: Full per-topology statistics for each target quantity, over the 5000 samples of each topology. Voltages are over each topology’s supervised internal nets (counts in Table 3.10), currents and small-signal parameters are in $\log_{10}$, DC gain is in decibels.

A.2 Training Details

This appendix lists the full training settings. The single-topology fan_smc run is taken as the reference and other training setups are described as deviations from it. Settings that are common to all runs are given once in Table A.2 and the per run differences in Table A.3.

Setting	Value
Optimizer	Adam
Weight decay	0
Peak learning rate	3×10^{-3}
Warm-up	linear, 50 epochs
LR schedule	halved on plateau, floor 10^{-6}
Plateau patience	150 epochs
Early-stop patience	500 epochs
Gradient clip (max norm)	0.3
Batch size	1024 graphs
Max epochs	5000
Plateau / checkpoint metric	validation loss
KCL loss ramp	start epoch 200, ramp 100 epochs
Loop-attention ramp	start epoch 350, ramp 400 epochs
Global-attention gates	initialized to 0, no schedule
$\mathcal{L}_V, \mathcal{L}_I, \mathcal{L}_{gm}, \mathcal{L}_{gds}$ weights	1.0 each
$\mathcal{L}_{KCL}$ weight	1.0
$\mathcal{L}_{DC}$ weight	1.0

Table A.2: Reference training settings, the single-topology fan_smc run. The other regimes differ only in the rows listed in Table A.3.

The multi-topology runs are the joint run on all five topologies, the held-out pretraining on four topologies with and without the physics terms and the fine-tuning on a held-out topology. They share the reference settings except for the entries in Table A.3. The learning rate is lower, the patience and the two ramps are longer and the plateau, checkpoint and early stopping are driven by the average per-topology voltage error rather than the validation loss. The gradient clip is 1.0 for these runs. The no-physics held-out runs switch only the current conservation loss off. The fine-tuning runs start from a pretrained model and the current conservation loss is active from the first. The DC-gain head and its loss are used only in the single-topology runs.

The held-out and fine-tuning runs with each batch holding an equal share of every active topology and the validation set always contains all five topologies. The fine-tuning sample counts are $N \in \{100, 250, 500, 1000, 2500, 4000\}$ per topology.

The runs were carried out with the following environment.

Setting	Single (ref)	Joint	Held-out (phys)	Held-out (no phys)	Fine-tune
Peak LR	3×10^{-3}	10^{-3}	10^{-3}	10^{-3}	10^{-3}
Plateau patience	150	200	200	200	200
Early-stop patience	500	800	800	800	800
Gradient clip	0.3	1.0	1.0	1.0	1.0
KCL ramp (start + dur.)	200+100	200+200	200+200	—	from ep. 0
Loop-attn ramp (start + dur.)	350+400	350+800	350+800	off	350+800
KCL weight	1.0	1.0	1.0	0	1.0
DC-gain head	on	off	off	off	off
Selection metric	val. loss	per-topo V	per-topo V	per-topo V	per-topo V

Table A.3: Per run training settings. All other settings are as in Table A.2. “per-topo V” is the average per-topology voltage error on the validation set. The no-physics held-out runs switch off the current conservation loss. The fine-tuning runs start from a pretrained model and the current conservation loss is active from the first epoch.

Component	Version
Python	3.10
PyTorch	2.5.1 (CUDA 12.1)
PyTorch Geometric	2.7.0
PySpice	1.5
GPU	single NVIDIA H100

Table A.4: Software and hardware environment. NGSPICE is used through PySpice for the ground truth simulations at dataset generation.

A.3 Detailed Ablation Accuracy

The body reports only the tightest threshold for each quantity, since that is where the variants separate most. This appendix gives for every architecture ablation, the share of predictions that fall within a range of absolute and relative tolerances. The first group of tables is for the message-passing operators of Section 4.2.1, the second for the attention components of Section 4.2.2 and the third for the backbone depth sweep of Section 4.2.3. All numbers are on `fan_smc` on the validation set (1000 samples).

Voltage					Current				
Op.	@10mV	@50mV	@1%	@10%	Op.	@2μA	@20μA	@1%	@10%
GINE	92.0	98.9	60.6	94.5	GINE	93.6	98.7	43.5	92.2
GIN	91.8	98.9	60.6	94.3	GIN	93.3	98.6	41.5	91.6
GATv2	88.2	98.9	53.1	92.5	GATv2	92.4	98.5	35.4	89.9
GENConv	88.3	98.9	53.6	92.8	GENConv	92.4	98.5	37.2	90.2

Small-signal					DC gain				
Op.	$g_m@10$	$g_m@20$	$g_{ds}@10$	$g_{ds}@20$	Op.	@3dB	@5dB	@10dB	valid
GINE	90.7	96.8	82.1	94.5	GINE	64.9	79.3	90.9	97.9
GIN	90.5	96.7	82.8	94.6	GIN	59.3	75.4	89.1	97.7
GATv2	89.2	96.2	77.7	92.8	GATv2	58.6	73.8	88.8	98.2
GENConv	88.7	95.8	76.6	92.3	GENConv	57.2	73.0	88.9	97.3

Table A.5: Within-threshold accuracy (%) for the attention components. Thresholds and units as in Table A.6.

<i>Voltage</i>					<i>Current</i>				
Variant	@10mV	@50mV	@1%	@10%	Variant	@2 μ A	@20 μ A	@1%	@10%
Reference	92.0	98.9	60.6	94.5	Reference	93.6	98.7	43.5	92.2
Loop off	62.9	96.7	31.5	84.3	Loop off	86.4	97.4	16.0	74.9
VN off	89.8	98.6	58.1	93.6	VN off	92.5	98.5	38.8	90.1
VN plain	90.0	98.6	58.2	93.7	VN plain	93.0	98.4	41.0	90.4
Both off	90.3	98.5	60.0	93.2	Both off	93.0	98.5	40.4	89.7

<i>Small-signal</i>					<i>DC gain</i>				
Variant	g_m @10	g_m @20	g_{ds} @10	g_{ds} @20	Variant	@3dB	@5dB	@10dB	valid.
Reference	90.7	96.8	82.1	94.5	Reference	64.9	79.3	90.9	97.9
Loop off	75.3	88.4	58.2	81.0	Loop off	43.9	60.8	83.3	96.4
VN off	89.3	95.9	79.8	93.5	VN off	59.4	74.8	87.7	97.9
VN plain	89.1	95.7	79.2	92.8	VN plain	56.9	74.6	88.2	97.0
Both off	88.4	95.2	80.3	93.3	Both off	57.4	73.3	88.2	97.3

<i>Voltage</i>					<i>Current</i>				
Depth	@10mV	@50mV	@1%	@10%	Depth	@2 μ A	@20 μ A	@1%	@10%
4	87.5	98.8	52.7	92.4	4	92.4	98.5	35.7	89.7
6	89.4	99.0	55.7	93.1	6	92.6	98.5	39.6	90.3
8	92.0	98.9	60.6	94.5	8	93.6	98.7	43.5	92.2
12	84.1	98.7	48.4	91.6	12	91.2	98.3	30.8	87.5
16	85.8	98.8	50.1	92.4	16	91.6	98.4	32.0	88.0

<i>Small-signal</i>					<i>DC gain</i>				
Depth	g_m @10	g_m @20	g_{ds} @10	g_{ds} @20	Depth	@3dB	@5dB	@10dB	valid.
4	88.8	95.7	79.4	93.5	4	57.2	72.4	88.6	97.3
6	89.2	96.0	79.6	93.6	6	57.2	72.0	88.3	97.9
8	90.7	96.8	82.1	94.5	8	64.9	79.3	90.9	97.9
12	86.2	94.6	71.1	90.1	12	54.2	70.1	88.8	97.7
16	87.5	95.5	76.5	92.4	16	57.6	74.3	89.2	98.2

Table A.6: Within-threshold accuracy (%) for the message-passing operators. Voltage thresholds are absolute (mV) and relative. Current thresholds are absolute (μ A) and relative. The small-signal thresholds are relative. The DC-gain thresholds are absolute (dB) and “valid” is the accuracy of classifying whether a circuit amplifies.

A.4 DC-Gain Formula

This appendix gives the analytical DC-gain formula embedded in the physics head of Section 4.2.4. The formula is not a loss term, it is a analytical formula evaluated on the predicted g_m and g_{ds} to produce a first estimate of the gain, which a small MLP that corrects.

The expression follows standard small-signal analysis of a multi-stage amplifier [16], where the overall gain is the product of the per-stage gains. Each stage contributes a transconductance g_m times an output resistance R_{out} . Here g_m is the transconductance of a transistor and g_{ds} its output conductance. The device labels (M6, M8, and so on) are the transistor names of the fan_smc topology [15]. The formula uses ten transistors on the signal path: M6, M8, M16, and M20 in the first stage; M7, M10, M21, and M22 in the second; and M11 and M23 in the third, push-pull output stage. What is specific to this topology is which devices sit on the path and how the third stage is modelled, these were worked out from the circuit rather than taken from a textbook.

The first stage uses a cascode output resistance, which keeps the boost from the cascode devices rather than collapsing it to a single transistor's output conductance:

$$R_{out1} = \frac{1}{g_{ds,M6} + \frac{g_{ds,M16} g_{ds,M20}}{g_{m,M16}}}, \quad A_1 = g_{m,M8} R_{out1}, \quad (A.1)$$

where A_1 is the first-stage gain. The second stage contributes a gain A_2 along the signal path:

$$A_2 = \frac{g_{m,M10}}{g_{m,M21} + g_{ds,M21} + g_{ds,M10}} \cdot \frac{g_{m,M22}}{g_{ds,M7} + g_{ds,M22}}. \quad (A.2)$$

The third stage is the part where a simple model fails. It is a push-pull stage [15], where two devices drive the output, the output transistor M23 and a feedforward path through M11, hence its effective transconductance is the sum of both contributions rather than that of the output device alone. Its output resistance is

$$R_{out3} = \frac{1}{g_{ds,M11} + g_{ds,M23}}, \quad (A.3)$$

and the total gain is

$$A_{dc} = A_1 \cdot R_{out3} \cdot (g_{m,M11} + A_2 g_{m,M23}), \quad A_{dc}[\text{dB}] = 20 \log_{10}(A_{dc}). \quad (A.4)$$

Dropping the $g_{m,M11}$ term treats the third stage as a simple class-A stage [16], one in which only the output transistor contributes to the gain. That is the textbook assumption, but it is wrong for this circuit. The feedforward path through M11 carries most of the third stage's transconductance, so leaving it out throws away most of the gain. This is why the class-A formulations are so far off in Section 4.2.4.

The whole expression is evaluated in $\log_{10}$ space, with sums handled by a numerically stable log-sum, so that the very wide range of g_m and g_{ds} does not cause overflow or underflow.

The head does not receive only the ten devices that appear in the formula. The predicted g_m and g_{ds} of all 14 signal-path MOSFETs are passed to the head, together with the formula's intermediate terms, so the network also sees the devices that the closed-form expression

Quantity	Count
MOSFETs in the analytical formula	10
Signal-path MOSFETs fed to the head	14
g_m and g_{ds} values (14×2)	28
Intermediate formula scalars	7
Total head input dimension	35

Table A.7: Input to the physics-based DC-gain head. Each of the 14 signal-path MOSFETs contributes a predicted g_m and g_{ds} (28 values) and the seven intermediate terms of the formula (the per-stage gains, the output resistances and the formula's own decibel estimate) are added, for a total input dimension of 35.

leaves out. Table A.7 lists the breakdown.

This expression is specific to the `fan_smc` topology. The device labels and the signal path are those of this circuit and the formula does not transfer unchanged to the other topologies.

A.5 Data Efficiency Runs

This appendix gives the exact numbers behind the data-efficiency curves of Section 4.3.2. All runs use the reference architecture on `fan_smc`, with the same 1000-sample validation set. Only the training-set size and the KCL constraint change. The left block is the mean absolute error, the right block the share of predictions within a tight tolerance (voltage within 1% of the supply range, current within 1%, g_m/g_{ds} within 10%, gain within 3 dB).

n	KCL	MAE					Within threshold (%)				
		V (mV)	I (μ A)	g_m (log)	g_{ds} (log)	Gain (dB)	V @1%	I @1%	g_m @10%	g_{ds} @10%	Gain @3dB
100	on	20.06	14.1	0.058	0.096	9.69	28.6	16.8	70.0	45.3	33.3
100	off	27.77	15.4	0.077	0.123	11.58	22.4	10.7	61.3	37.4	27.4
250	on	19.66	12.3	0.054	0.088	8.28	28.6	17.1	70.0	46.8	36.1
250	off	21.53	13.5	0.061	0.089	9.58	22.8	12.7	65.0	46.3	30.9
500	on	15.76	10.5	0.044	0.073	7.69	31.4	19.1	74.8	52.2	36.8
500	off	20.42	11.6	0.057	0.087	9.07	22.8	15.3	68.0	46.5	30.9
1000	on	15.76	8.3	0.041	0.065	6.58	33.5	21.9	77.2	57.5	42.8
1000	off	20.13	10.6	0.053	0.079	8.43	24.9	16.2	69.6	49.7	33.6
2500	on	13.42	7.5	0.038	0.061	6.58	33.5	23.9	78.7	59.7	42.8
2500	off	18.75	10.5	0.053	0.079	8.43	26.7	16.2	69.6	50.5	35.4
4000	on	6.42	3.7	0.020	0.034	4.36	60.6	43.5	90.7	82.1	64.9
4000	off	8.20	5.8	0.028	0.042	5.80	52.0	30.5	86.5	77.5	55.0

Table A.8: Data-efficiency runs on `fan_smc`. Mean absolute error (left) and within-threshold accuracy (right) by training-set size, KCL on/off.

A.6 Approximate Physics Constraints

This appendix shows the exact form of the analytical constraints discussed in Section 4.3.1, the full SMAXT derivation and the detailed accuracy of each formula against the BSIM4 ground truth.

A.6.1 Constraint Definitions

Each constraint adds a loss term comparing a model-predicted quantity to an analytical target. The per-region transconductance formulas are the textbook small-signal relations, applied to the devices the simulator places in that region:

$$\text{saturation: } g_m = \frac{2I_D}{V_{ov}}, \quad (A.5)$$

$$\text{triode: } g_m = \frac{I_D}{V_{ov} - V_{DS}/2}, \quad g_{ds} = \frac{I_D}{V_{DS}} \cdot \frac{V_{ov} - V_{DS}}{V_{ov} - V_{DS}/2}, \quad (A.6)$$

$$\text{weak inversion: } g_m = \frac{I_D}{nV_T}, \quad (A.7)$$

with the overdrive $V_{ov} = V_{GS} - V_{th}$, the thermal voltage $V_T = 25.85 \text{ mV}$ at 300 K, and the sub-threshold slope factor $n = 1.5$ (NMOS) or 2.0 (PMOS). The current-mirror and differential-pair constraints enforce the ideal device relations:

$$I_{\text{mirror}} = \frac{(W/L)_{\text{mirror}}}{(W/L)_{\text{ref}}} I_{\text{ref}}, \quad I_{M8} + I_{M9} = I_{\text{tail}}, \quad (A.8)$$

the bandwidth constraint enforces the single-pole relation

$$\text{UGBW} \approx \frac{g_{m1}}{2\pi C_L}, \quad (A.9)$$

and the g_m/I_D consistency term matches the predicted ratio g_m/I_D to the simulated one, with no analytical formula involved. Each loss is computed in $\log_{10}$ space and for the region-dependent formulas, applied only to the devices in the corresponding region.

A.6.2 The SMAXT Transconductance Formula

SMAXT replaces the three separate per-region [16] with one smooth, region-independent formulas. Continuous all-region MOSFET models like the EKV model [31] describe the transistor with a single expression valid from weak to strong inversion. SMAXT does the same thing with a simpler construction built directly from the textbook region relations. The smoothing uses $\text{softplus}(x) = \log(1 + e^x)$ to build smooth versions of the min and max that the piecewise formulas would otherwise switch between.

Voltages are first normalized so that both device types are handled with positive overdrive,

$$V_{GS} = \begin{cases} V_G - V_S & \text{NMOS} \\ V_S - V_G & \text{PMOS} \end{cases}, \quad V_{DS} = \begin{cases} V_D - V_S & \text{NMOS} \\ V_S - V_D & \text{PMOS} \end{cases}, \quad (A.10)$$

The overdrive is $V_{ov} = V_{GS} - V_{th}$, which may be negative in cutoff. The drain-source voltage is first capped at the overdrive, a smooth $\min(V_{DS}, V_{ov})$, to mark the saturation–triode boundary:

$$V_{DS,\text{eff}} = V_{DS} - nV_T \text{softplus}\left(\frac{V_{DS} - V_{ov}}{nV_T}\right). \quad (\text{A.11})$$

The denominator is then a smooth $\max(V_{ov} - V_{DS,\text{eff}}/2, nV_T)$, which prevents it from collapsing as $V_{ov} \rightarrow 0$:

$$\text{denom} = nV_T + nV_T \text{softplus}\left(\frac{V_{ov} - V_{DS,\text{eff}}/2 - nV_T}{nV_T}\right), \quad (\text{A.12})$$

and the unified transconductance is

$$g_{m,\text{SMAXT}} = \frac{I_D}{\text{denom}}. \quad (\text{A.13})$$

The constraint is the mean-squared error between $\log_{10} g_{m,\text{SMAXT}}$ and the $\log_{10} g_m$ from the small-signal head. The floor nV_T is what keeps the formula finite near threshold, where the per-region formulas diverge.

The point of the smooth construction is that it reduces to each textbook formula, as summarised in Table A.9. SMAXT is therefore the same physics as the three per-region relations, written as one expression.

Region	Condition	Limit of $g_{m,\text{SMAXT}}$
Saturation (strong inv.)	$V_{DS} \geq V_{ov} \gg nV_T$	$2I_D/V_{ov}$
Triode (strong inv.)	$V_{DS} < V_{ov}, V_{ov} \gg nV_T$	$I_D/(V_{ov} - V_{DS}/2)$
Weak inversion / cutoff	$V_{ov} \ll nV_T$	$I_D/(nV_T)$

Table A.9: The three limits of the SMAXT formula. In each regime the smooth expression reduces to the corresponding textbook transconductance relation.

A.6.3 Formula Accuracy in Detail

Table A.10 gives the full accuracy of each formula against the simulated values, including the effect of filtering the saturation relation by overdrive and the per-region breakdown of SMAXT. All numbers use ground-truth inputs, so they measure the accuracy of the formula itself.

A.7 Predicting the Unity-Gain Bandwidth

The unity-gain bandwidth was the one frequency-domain quantity that a serious effort went into predicting. It did not work and this appendix records what was tried and why none of it reached a usable accuracy. All results are on `fan_smc` and the error is reported as the mean absolute error of $\log_{10}(\text{UGBW})$, together with the fraction of samples whose predicted bandwidth lands within a given percentage of the ground truth value.

Formula	Region	N	$\log_{10}$ MAE	Median rel	Mean rel	r
Saturation g_m (no filter)	saturation	2507	0.305	36.8%	365%	0.881
Saturation g_m ($V_{ov} > 10$ mV)	saturation	2334	0.228	29.6%	107%	0.945
Saturation g_m ($V_{ov} > 50$ mV)	saturation	1718	0.097	13.9%	28%	0.992
Saturation g_m ($V_{ov} > 100$ mV)	saturation	1212	0.046	8.3%	11.7%	0.999
Triode g_m ($V_{ov} > V_{DS}$)	triode	1594	0.098	15.8%	32.9%	0.996
Triode g_{ds} ($V_{ov} > V_{DS}$)	triode	1594	0.078	2.7%	71.7%	0.983
Cutoff g_m	cutoff	13267	0.193	19.3%	—	0.983
SMAXT (all regions)	all	24000	0.032	5.5%	—	0.997
SMAXT (cutoff slice)	cutoff	13267	0.024	4.5%	—	0.995
SMAXT (triode slice)	triode	2948	0.064	12.4%	16.7%	0.999
SMAXT (saturation slice)	saturation	7785	0.034	6.6%	8.3%	0.999

Table A.10: Accuracy of each analytical formula on `fan_smc`, on ground-truth inputs. The saturation relation is shown at several overdrive filters. The triode relations are the least accurate. SMAXT is the most accurate both overall and within each region.

A.7.1 The Data Problem

Only about 23% of the `fan_smc` samples have an unity-gain bandwidth, because it the frequency where the gain crosses 0 dB, so it only exists if the gain actually crosses it.

A.7.2 The Approaches

Table A.11 lists the approaches that were run, grouped by the kind of head used.

Approach	UGBW MAE (log)	< 5%	< 10%	< 20%
<i>Plain neural head (no physics)</i>				
MLP on the graph embedding	0.474	3.3%	6.0%	11.6%
with a larger learning rate	0.431	3.6%	6.5%	13.2%
virtual-node pooling	0.704	1.1%	1.7%	5.6%
<i>Analytical formula with a learned correction</i>				
$g_m/(2\pi C_C)$ plus an MLP	0.254	8.7%	16.2%	28.2%
<i>Signal-flow graph (DAG)</i>				
DAG only, no formula	0.403	3.3%	5.5%	12.4%
DAG plus the formula	0.265	7.1%	15.2%	29.9%
DAG as a residual on the formula	0.296	4.8%	9.3%	19.0%

Table A.11: Unity-gain bandwidth accuracy for the approaches that were run, on `fan_smc`. The error is the mean absolute error of $\log_{10}(\text{UGBW})$ and the last three columns are the fraction of samples within 5, 10 and 20 percent of the ground truth value. The formula with a learned correction is the best, but still far from usable.

The plain neural head learns almost nothing useful. The analytical formula with a learned correction does best. The model fits the bandwidths in the training set well, around 0.04 to 0.05, but on validation samples the error is several times higher. The formula it tries to fit here is quite inaccurate. On the ground-truth values the bare $g_m/(2\pi C_C)$ has a median relative error of over two thousand percent, because it assumes a single dominant pole and ignores the other poles. With the leaned correction the median error is reduced to about 45%.

The signal-flow graph was added to give the model a sense of how the signal flow from one stage to another, since the bandwidth depends on the path from the input through the compensation to the output. The DAG on top of the formula is no better than the formula alone. Furthermore, the DAG on its own is as poor as the plain neural head. A cross-attention and a multi-head pooling variant for the bandwidth was considered but never trained on, as at that point the approach was already abandoned.

A.7.3 The Cost to the Other Quantities

Adding a bandwidth head also made the rest of the model worse. Table A.12 shows a matched comparison where the only change is the extra bandwidth head. The bandwidth head competes with the operating-point task for the same capacity. This is the second reason the bandwidth was left out of the final model.

Model	V (mV)	I (μ A)	g_m (log)	g_{ds} (log)	UGBW (log)
Without the UGBW head	12.34	7.6	0.042	0.065	—
With the UGBW head	22.54	11.3	0.076	0.117	0.272
Change	+83%	+49%	+81%	+80%	—

Table A.12: The effect of adding a unity-gain bandwidth head, on a matched comparison where the only difference is the extra head. All the quantities got worse. The bandwidth head is not a free addition.

A.8 Full Fine-Tuning Results Across Topologies

This appendix gives the full fine-tuning results for the Section 4.4.2. Each entry is the difference between fine-tuning and training from scratch on the same data, fine-tune minus scratch, so a negative number means fine-tuning is better and a positive number means scratch is better. All values are on the held-out topology’s 1000-sample validation set.

Topology	N					
	100	250	500	1000	2500	4000
fan_smc	−96.5	−14.3	−7.4	−4.3	−4.1	−4.3
sau_cfcc	−45.7	−6.5	−6.0	−2.5	−2.0	−5.0
peng_tcfc	−35.4	−4.5	−2.0	−2.3	−3.1	−3.7
leung_nmcf	−33.2	−24.4	−16.0	−10.8	−8.5	−9.3
leung_nmcnr	−69.1	−11.6	−5.1	−5.2	−5.3	−5.5

Table A.13: Voltage error, fine-tuning minus scratch (mV), per topology and sample size. Negative means fine-tuning is better.

Quantity	Topology	$N=100$	500	1000	4000
I (μA)	fan_smc	-14.3	-9.9	-0.5	-3.5
	sau_cfcc	-6.7	+0.6	-5.6	+0.9
	peng_tcfc	-9.7	-0.8	-1.9	-0.8
	leung_nmcfc	-6.1	-5.0	-9.8	-3.3
	leung_nmcnr	-11.0	-5.7	-0.1	-1.8
g_m (log)	fan_smc	-0.229	-0.049	-0.012	-0.007
	sau_cfcc	-0.112	-0.011	+0.004	-0.009
	peng_tcfc	-0.079	-0.005	-0.005	-0.013
	leung_nmcfc	-0.095	-0.036	-0.048	-0.016
	leung_nmcnr	-0.124	-0.036	-0.019	-0.010
g_{ds} (log)	fan_smc	-0.312	-0.062	-0.031	-0.013
	sau_cfcc	-0.172	-0.018	-0.006	-0.016
	peng_tcfc	-0.130	-0.021	-0.009	-0.011
	leung_nmcfc	-0.148	-0.047	-0.067	-0.023
	leung_nmcnr	-0.194	-0.036	-0.025	-0.014

Table A.14: Current and small-signal error, fine-tuning minus scratch, per topology and sample size. Negative means fine-tuning is better.

A.9 Explored Alternatives: Compressed IV-Surface Embeddings as Device Features

One direction explored for giving the model more device information was to feed it a compressed form of each transistor’s full current-voltage behavior.

For each device archetype an IV surface was built from the SKY130 BSIM models, $\log_{10} |I_D|$ over a grid of V_{GS} and V_{DS} at the typical corner, about 65,000 points per surface. Figure A.1 shows one such surface. These were generated once as a lookup table over device geometry, 20 widths and 80 lengths across the sampled range for both polarities, giving 3,200 surfaces. A convolutional autoencoder was trained offline to compress each surface into a latent vector, with seven downsampling layers into a small bottleneck and a mirrored decoder. Bottleneck sizes of 64, 96 and 128 were tried, along with a single-channel (I_D) and a six-channel (I_D , g_m , g_{ds} , V_{th} , V_{dsat} , g_{mbs}) variant. The encoder was then frozen and its latent vector attached as an extra per-device input feature, with the rest of the model unchanged.

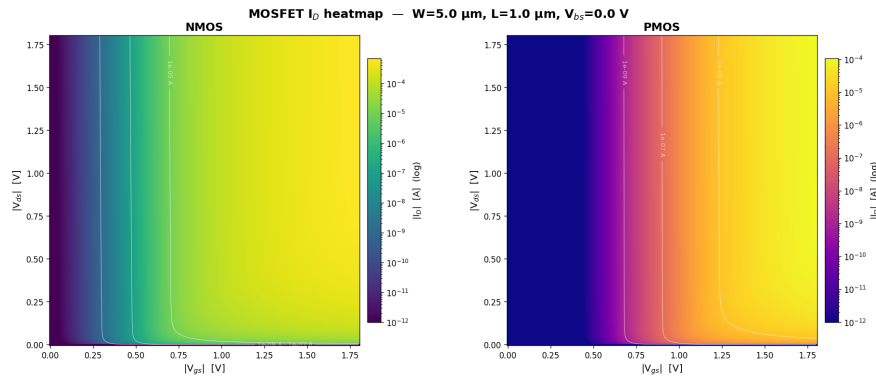


Figure A.1: The IV surface that was compressed, $\log_{10} |I_D|$ over V_{GS} and V_{DS} for an NMOS and a PMOS device at $V_{BS} = 0$. The current depends almost entirely on V_{GS} and barely on V_{DS} , so the surface has little structure to encode.

The autoencoder compressed the surfaces almost perfectly. Figure A.2 shows several devices with their reconstructions and the residual, which is near-zero everywhere except a thin band at the cutoff boundary.

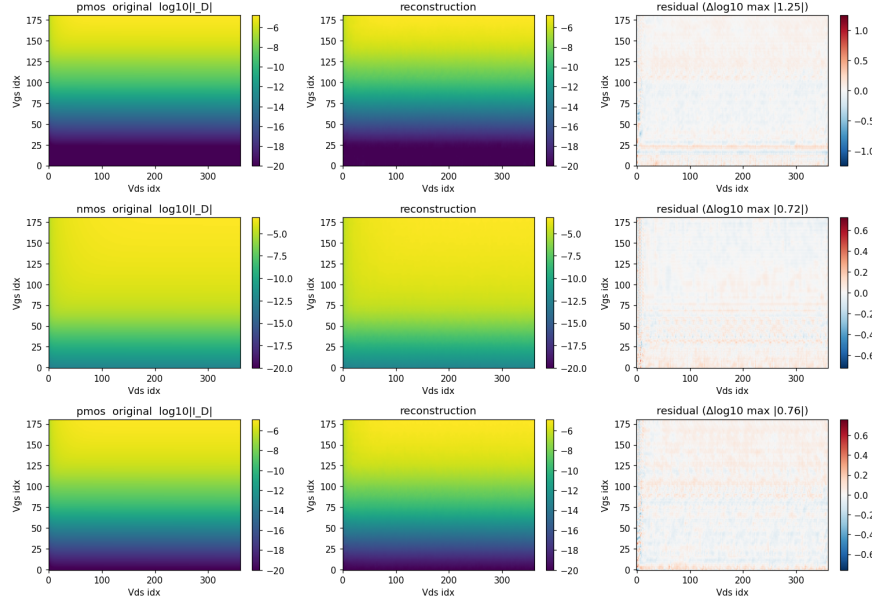


Figure A.2: Original IV surfaces, their autoencoder reconstructions and the residual $\Delta \log_{10} |I_D|$. The reconstruction is near perfect, the residual is close to zero apart from a thin band at the cutoff boundary.

The embedding did not improve the model. An MLP trained on the embedding alone could recover the device width, length, polarity and small-signal parameters at a reference point with $R^2 > 0.99$. But an MLP given the embedding on top of the raw geometry did no better than one given the geometry alone. The embedding carried nothing that the width, length and polarity did not already imply.

The reason is structural. At a fixed bulk bias, corner and temperature, the IV surface is completely determined by the device geometry and polarity. Knowing W , L and whether the device is NMOS or PMOS fixes the entire surface. So compressing the surface into an embedding can only recover those same three things and the model already has all three as inputs.

The two figures show this from both sides. In Figure A.3 the embeddings organise themselves only by polarity and by channel length. And the near perfect reconstruction at a tiny bottleneck (Figure A.2) says the surfaces are simple to begin with. There was no extra information for the embedding to add.

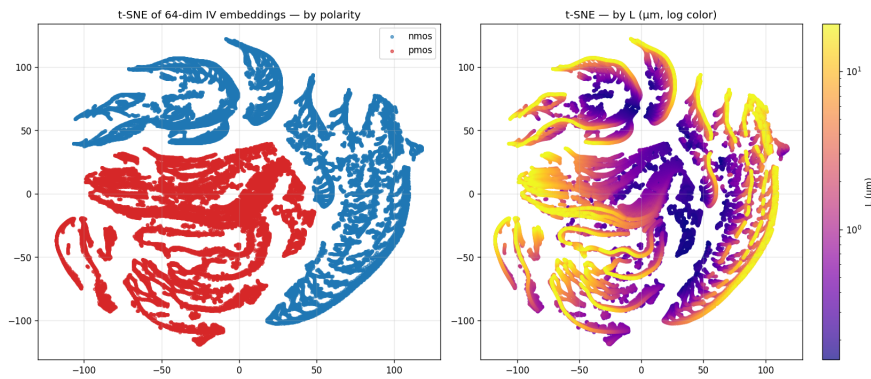


Figure A.3: A t-SNE projection of the 64-dimensional IV embeddings, coloured by polarity (left) and by channel length (right). The embeddings are organised entirely by polarity and length, the same geometry the model already has as input, which is why they add no information.

The approach was dropped for this reason. It added real complexity, the lookup table, the autoencoder and its tuning, and the per-device surface lookup, for no measurable gain over letting the model learn from the geometry directly. The only versions that could add genuine information would vary the bulk bias or the corner, where the geometry no longer fixes the surface and that was not pursued.

Bibliography

- [1] Synopsys, Inc. *Overcoming Analog Circuit Design Challenges at Advanced Semiconductor Nodes*. Accessed: 2026-05-04. 2024. URL: <https://www.synopsys.com/blogs/chip-design/overcoming-analog-circuit-design-challenges-at-advanced-semiconductor-nodes.html>.
- [2] Rashid, R., Krishna, K., George, C. P., and Nambath, N. “Machine Learning Driven Global Optimisation Framework for Analog Circuit Design”. In: *Integration, the VLSI Journal* (2024). DOI: 10.1016/j.vlsi.2024.102216. arXiv: 2404.02911.
- [3] Mehradfar, A., Zhao, X., Niu, Y., Babakniya, S., Alesheikh, M., Aghasi, H., and Avestimehr, S. *Supervised Learning for Analog and RF Circuit Design: Benchmarks and Comparative Insights*. 2025. arXiv: 2501.11839 [cs.LG].
- [4] Budak, A. F., Gandara, M., Shi, W., Pan, D. Z., Sun, N., and Liu, B. “An Efficient Analog Circuit Sizing Method Based on Machine Learning Assisted Global Optimization”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41.5 (2022), pp. 1209–1221. DOI: 10.1109/TCAD.2021.3081405.
- [5] Yu, X., Torbunov, D., Mandal, S., and Ren, Y. *AutoSizer: Automatic Sizing of Analog and Mixed-Signal Circuits via Large Language Model (LLM) Agents*. 2026. arXiv: 2602.02849 [cs.AR].
- [6] Dutta, S., Amin, F., Panda, S., Rabe, J., Wen, Y., and Franzon, P. *Can an Actor-Critic Optimization Framework Improve Analog Design Optimization?* 2026. arXiv: 2603.24714 [cs.LG].
- [7] Bloem, R., Larrauri, A., Lengfeldner, R., Mateis, C., Ničković, D., and Ziegler, B. “Industry Paper: Surrogate Models for Testing Analog Designs under Limited Budget – a Bandgap Case Study”. In: *International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*. IEEE, 2022. DOI: 10.1109/CODES-ISSS55005.2022.00016.
- [8] Uhlmann, Y., Brunner, M., Bramlage, L., Scheible, J., and Curio, C. “Procedural- and Reinforcement-Learning-Based Automation Methods for Analog Integrated Circuit Sizing in the Electrical Design Space”. In: *Electronics* 12.2 (2023), p. 302. DOI: 10.3390/electronics12020302.
- [9] Dong, Z., Cao, W., Zhang, M., Tao, D., Chen, Y., and Zhang, X. “CktGNN: Circuit Graph Neural Network for Electronic Design Automation”. In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2308.16406. URL: <https://openreview.net/forum?id=NE2911Kq1sp>.
- [10] Liu, M., Turner, W. J., Kokai, G. F., Khailany, B., Pan, D. Z., and Ren, H. “Parasitic-Aware Analog Circuit Sizing with Graph Neural Networks and Bayesian Optimization”. In: *Design, Automation and Test in Europe Conference (DATE)*. 2021, pp. 1372–1377. DOI: 10.23919/DATE51398.2021.9474253.

- [11] Li, Y., Lin, Y., Madhusudan, M., Sharma, A. K., Xu, W., Sapatnekar, S. S., Harjani, R., and Hu, J. "A Customized Graph Neural Network Model for Guiding Analog IC Placement". In: *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 2020, pp. 1–9. DOI: 10.1145/3400302.3415624.
- [12] Raissi, M., Perdikaris, P., and Karniadakis, G. E. "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Non-linear Partial Differential Equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. DOI: 10.1016/j.jcp.2018.10.045.
- [13] Shahbaz, I., Abdel-Rahman, M. J., and Hammad, E. *G-PIFNN: A Generalizable Physics-Informed Fourier Neural Network Framework for Electrical Circuits*. 2025. arXiv: 2512.02712.
- [14] Taj, R. *Physics-Informed Neural Networks for Electrical Circuit Analysis: Applications in Dielectric Material Modeling*. 2024. arXiv: 2411.10483 [cs.LG].
- [15] Li, J., Cao, H., Sun, S., Liu, B., Hu, Y., Zhang, Z., Wei, C., Yu, N., Yang, X., Tang, H., Wang, R., Yang, F., Zeng, X., and Zhou, D. "AnalogGym: An Open and Practical Testing Suite for Analog Circuit Synthesis". In: *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. New York, NY, USA, Oct. 2024. DOI: 10.1145/3676536.3697117.
- [16] Razavi, B. *Design of Analog CMOS Integrated Circuits*. 2nd ed. McGraw-Hill Education, 2017.
- [17] Gray, P. R., Hurst, P. J., Lewis, S. H., and Meyer, R. G. *Analysis and Design of Analog Integrated Circuits*. 6th ed. Wiley, 2024.
- [18] Vogt, H., Hendrix, M., Nenzi, P., and Warning, D. *NGSPICE Open Source Mixed-Mode Mixed-Level Circuit Simulator*. <https://ngspice.sourceforge.io/>. 2026.
- [19] Liu, W., Cheng, X., and Hu, C. *BSIM4.6.1 MOSFET Model User Manual*. University of California, Berkeley. 2007.
- [20] Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. "Neural Message Passing for Quantum Chemistry". In: *Proceedings of the 34th International Conference on Machine Learning*. PMLR, 2017, pp. 1263–1272.
- [21] Xu, K., Hu, W., Leskovec, J., and Jegelka, S. "How Powerful are Graph Neural Networks?" In: *International Conference on Learning Representations*. 2019.
- [22] Kipf, T. N. and Welling, M. "Semi-Supervised Classification with Graph Convolutional Networks". In: *International Conference on Learning Representations*. 2017.
- [23] Brody, S., Alon, U., and Yahav, E. "How Attentive are Graph Attention Networks?" In: *International Conference on Learning Representations*. 2022.
- [24] Hamilton, W. L., Ying, R., and Leskovec, J. "Inductive Representation Learning on Large Graphs". In: *Advances in Neural Information Processing Systems* 30. 2017, pp. 1024–1034.
- [25] Li, G., Xiong, C., Thabet, A., and Ghanem, B. "DeeperGCN: All You Need to Train Deeper GCNs". In: *arXiv preprint*. arXiv:2006.07739. 2020.
- [26] Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., and Yang, L. "Physics-Informed Machine Learning". In: *Nature Reviews Physics* 3.6 (2021), pp. 422–440. DOI: 10.1038/s42254-021-00314-1.
- [27] Abel, I., Neuner, M., and Graeb, H. "COPRICSI: CONstraint-PROgrammed Initial Circuit Sizing". In: *Integration, the VLSI Journal* 76 (2021), pp. 148–158. DOI: 10.1016/j.vlsi.2020.10.002.

- [28] Ghosh, S., Gebru, E. Y., Kashyap, C. V., Harjani, R., and Sapatnekar, S. S. “Accelerating OTA Circuit Design: Transistor Sizing Based on a Transformer Model and Precomputed Lookup Tables”. In: *Design, Automation and Test in Europe Conference (DATE)*. EDAA, 2025.
- [29] Lu, H., Fan, S., Zhang, S., Cao, N., Zhang, X., and Li, J. “Towards Generalizable and Efficient Circuit Topology Design: A Graph-Transformer-based Surrogate Model with Curriculum Learning”. In: *ACM Transactions on Design Automation of Electronic Systems* 31.4 (2026), pp. 1–29. DOI: 10.1145/3768167.
- [30] Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., and Beaini, D. “Recipe for a General, Powerful, Scalable Graph Transformer”. In: *Advances in Neural Information Processing Systems* 35. 2022.
- [31] Enz, C. C., Krummenacher, F., and Vittoz, E. A. “An Analytical MOS Transistor Model Valid in All Regions of Operation and Dedicated to Low-Voltage and Low-Current Applications”. In: *Analog Integrated Circuits and Signal Processing* 8.1 (1995), pp. 83–114. DOI: 10.1007/BF01239381.

Disclaimer

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Use of AI tools. During the preparation of this thesis, I used generative AI tools (Anthropic Claude) as assistive tools, for language and grammar editing of text I wrote, for feedback on structure and phrasing, for assistance with $\text{\LaTeX}$ formatting, for extracting and verifying configuration details from my own codebase. In addition, as described in Section 5.5, a language model was used to propose candidate analytical DC-gain formulas, which I validated against the simulator before one was used. All experiments, analyses and conclusions are my own. I reviewed all AI-assisted output and take full responsibility for the content of this thesis.

Garching, July 10, 2026

(Signature)